BevIQ / beverage_db
Code Explanation Study Guide

Yeh separate study PDF hai. Project ke source code mein koi feature add ya change nahi kiya gaya. Hinglish mein file-by-file, function-by-function aur line-by-line reading guide diya gaya hai.

1. Project ko ek line mein samjho

Browser question bhejta hai -> Flask app.py receive karta hai -> rag_orchestrator intent decide karta hai -> SQL/comparison/document/vector/UPC engine data nikalta hai -> optional Ollama language answer banata hai -> JSON frontend ko milta hai -> index.html chat/table dikhata hai.

- Flask: web server aur REST API routes.
- SQLite: offline local indexed database; CSV ko har query par scan nahi karna padta.
- MySQL: optional DBeaver/shared database; SQLite fallback available hai.
- RAG: relevant records/chunks retrieve karke unke basis par answer.
- Ollama: local LLM; optional conversational synthesis, exact truth source nahi.
- Vector index: yahan BM25-style token scoring hai; FAISS/dense embeddings nahi.
- UPC normalization: scientific notation/decimal/prefix se clean 13-digit string, leading zero safe.

2. RAG aur Ollama kyun?

RAG ka matlab Retrieval Augmented Generation hai: pehle apne database/document se relevant context, phir us context par answer. Exact UPC, count aur comparison code/SQL se calculate hote hain. Ollama local llama3.2 jaise model ko call karta hai, taaki retrieved facts ko natural English/Hinglish mein explain kiya ja sake. Ollama down ho tab deterministic offline answer available rehta hai.

3. Core source files: line-by-line

File: app.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
   1: import os
```
Import: library/module ke functions yahan available karata hai.
```
   2: import io
```
Import: library/module ke functions yahan available karata hai.
```
   3: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
   4: from datetime import datetime, timezone
```
Import: library/module ke functions yahan available karata hai.
```
   5: from pathlib import Path
```
Import: library/module ke functions yahan available karata hai.
```
   6: from flask import Flask, render_template, request, jsonify, send_file, send_from_directory
```
Import: library/module ke functions yahan available karata hai.
```
   7: from config import BASE_DIR, REPORTS_DIR, MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_DB, OLLAMA_HOST, OLLAMA_MODEL,
CSV_FILES
```
Import: library/module ke functions yahan available karata hai.
```
   8: from database.db_manager import db_manager
```
Import: library/module ke functions yahan available karata hai.
```
   9: from database.mysql_migrator import migrate_data_to_mysql
```
Import: library/module ke functions yahan available karata hai.
```
  10: from engines.sql_engine import sql_engine
```
Import: library/module ke functions yahan available karata hai.
```
  11: from engines.comparison_engine import comparison_engine
```
Import: library/module ke functions yahan available karata hai.
```
  12: from engines.vector_engine import vector_engine
```
Import: library/module ke functions yahan available karata hai.
```
  13: from engines.rag_orchestrator import rag_orchestrator
```
Import: library/module ke functions yahan available karata hai.
```
  14: from data_ingestion.loader import ingest_all_to_sqlite
```
Import: library/module ke functions yahan available karata hai.
```
  15: from document_rag import ingest as ingest_document, list_documents
```
Import: library/module ke functions yahan available karata hai.
```
  16: from engines.upc_match_engine import upc_match_engine
```
Import: library/module ke functions yahan available karata hai.
```
  17:
```
Blank line: code ko readable banane ke liye.
```
  18: app = Flask(__name__, template_folder="templates", static_folder="static")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  19: app.config['SECRET_KEY'] = 'beverage-data-assistant-secret-key-2026'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  20:
```
Blank line: code ko readable banane ke liye.
```
  21: @app.route('/')
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
  22: def index():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
  23:     return render_template('index.html',
```
Return: calculated result caller ko wapas bhejta hai.
```
  24:                            mysql_host=MYSQL_HOST,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  25:                            mysql_port=MYSQL_PORT,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  26:                            mysql_user=MYSQL_USER,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  27:                            mysql_db=MYSQL_DB,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  28:                            ollama_model=OLLAMA_MODEL)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  29:
```
Blank line: code ko readable banane ke liye.
```
  30: @app.route('/api/chat', methods=['POST'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
  31: def api_chat():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
  32:     data = request.form if request.form else (request.get_json() or {})
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
  33:     message = data.get('message', '').strip()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  34:     session_id = data.get('session_id', 'default-session')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
35:     if not message:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:         return jsonify({'error': 'Message cannot be empty'}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
37:
```
Blank line: code ko readable banane ke liye.
```
38:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
39:         uploaded = None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
40:         if 'file' in request.files and request.files['file'].filename:
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
41:             uploaded = ingest_document(request.files['file'], session_id)
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
42:         response = rag_orchestrator.route_and_execute(message, session_id=session_id)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
43:         if uploaded:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
44:             response['uploaded_document'] = uploaded
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
45:         return jsonify(response)
```
Return: calculated result caller ko wapas bhejta hai.
```
46:     except ValueError as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
47:         return jsonify({'error': str(e)}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
48:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
49:         return jsonify({
```
Return: calculated result caller ko wapas bhejta hai.
```
50:             'answer': f"### ?? Error processing query\n\n`{str(e)}`",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
51:             'source_files': [],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
52:             'record_count': 0,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
53:             'records': [],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
54:             'download_token': None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
55:         }), 500
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
56:
```
Blank line: code ko readable banane ke liye.
```
57: @app.route('/api/documents', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
58: def api_documents():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
59:     return jsonify({'documents': list_documents(request.args.get('session_id', 'default-session'))})
```
Return: calculated result caller ko wapas bhejta hai.
```
60:
```
Blank line: code ko readable banane ke liye.
```
61: @app.route('/api/upc-audit', methods=['POST'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
62: def api_upc_audit():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
63:     data = request.get_json() or {}
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
64:     category = data.get('category')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
65:     limit = None if data.get('full_review') else max(1, min(int(data.get('limit', 100)), 1000))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
66:     records = upc_match_engine.audit(category, limit)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
67:     import uuid
```
Import: library/module ke functions yahan available karata hai.
```
68:     token = str(uuid.uuid4())[:8]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
69:     columns = list(records[0].keys()) if records else [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
70:         'category', 'status', 'current_upc', 'suggested_upc', 'alternate_upc',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
71:         'confidence', 'source_file', 'source_row', 'source_brand', 'source_product',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
  72:         'source_flavour', 'source_size', 'source_category', 'master_brand',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  73:         'master_product', 'master_size', 'master_row', 'remarks'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  74:     ]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  75:     import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
  76:     pd.DataFrame(records, columns=columns).to_csv(REPORTS_DIR / f'report_{token}.csv', index=False)
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
  77:     return jsonify({'records': records, 'record_count': len(records), 'download_token': token,
```
Return: calculated result caller ko wapas bhejta hai.
```
  78:                     'columns': columns, 'read_only': True,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  79:                     'message': 'Review-only report. No master product or UPC was changed.'})
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  80:
```
Blank line: code ko readable banane ke liye.
```
  81: @app.route('/api/compare', methods=['POST'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
  82: def api_compare():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
  83:     data = request.get_json() or {}
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
  84:     comp_type = data.get('type', 'all_missing')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  85:     limit = int(data.get('limit', 250))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  86:     search = (data.get('search') or '').strip()[:100]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  87:
```
Blank line: code ko readable banane ke liye.
```
  88:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
  89:         if comp_type == 'all_missing':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  90:             res = comparison_engine.compare_all_missing_in_master(limit_per_category=limit, search=search or None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  91:         elif comp_type == 'master_vs_beer':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  92:             res = comparison_engine.compare_master_vs_categorization('cat_beer', limit=limit, search=search or
None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  93:         elif comp_type == 'master_vs_rtd':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  94:             res = comparison_engine.compare_master_vs_categorization('cat_rtd', limit=limit, search=search or
None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  95:         elif comp_type == 'master_vs_spirits':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  96:             res = comparison_engine.compare_master_vs_categorization('cat_spirits', limit=limit, search=search or
None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  97:         elif comp_type == 'master_vs_wine':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  98:             res = comparison_engine.compare_master_vs_categorization('cat_wine', limit=limit, search=search or
None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  99:         elif comp_type == 'allbrand_vs_supplier':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 100:             res = comparison_engine.compare_allbrand_vs_supplier(limit=limit)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 101:         elif comp_type == 'beer_vs_rtd':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 102:             res = comparison_engine.compare_beer_vs_rtd(limit=limit)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 103:         elif comp_type == 'duplicate_upcs':
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 104:             res = comparison_engine.find_duplicate_upcs_across_all(limit=limit)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 105:         else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 106:             return jsonify({'error': f"Unknown comparison type: {comp_type}"}), 400
```
Return: calculated result caller ko wapas bhejta hai.

```
107:
```
Blank line: code ko readable banane ke liye.
```
108:         # Create download token
```
Comment: developer ka note; Python ise execute nahi karta.
```
109:         import uuid, pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
110:         token = str(uuid.uuid4())[:8]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
111:         if res.get('records'):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
112:             df = pd.DataFrame(res['records'])
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
113:             df.to_csv(REPORTS_DIR / f"report_{token}.csv", index=False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
114:         res['download_token'] = token
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
115:
```
Blank line: code ko readable banane ke liye.
```
116:         return jsonify(res)
```
Return: calculated result caller ko wapas bhejta hai.
```
117:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
118:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
119:
```
Blank line: code ko readable banane ke liye.
```
120: @app.route('/api/stats', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
121: def api_stats():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
122:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
123:         overview = sql_engine.get_dataset_overview()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
124:         total_records = sum(item['total_rows'] for item in overview)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
125:         total_upcs = sum(item['unique_upcs'] for item in overview)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:         return jsonify({
```
Return: calculated result caller ko wapas bhejta hai.
```
127:             'total_records': total_records,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
128:             'total_unique_upcs': total_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
129:             'datasets': overview
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
130:         })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
131:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
132:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
133:
```
Blank line: code ko readable banane ke liye.
```
134: @app.route('/api/dashboard/overview', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
135: def api_dashboard_overview():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
136:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
137:         overview = sql_engine.get_dataset_overview()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
138:         total_records = sum(int(item.get('total_rows') or 0) for item in overview)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
139:         total_upcs = sum(int(item.get('unique_upcs') or 0) for item in overview)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
140:         return jsonify({
```
Return: calculated result caller ko wapas bhejta hai.
```
141:             'backend': db_manager.backend_name(),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
142:             'kpis': {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
143:                 'total_records': total_records,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
144:                 'total_unique_upcs': total_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
145:                 'dataset_count': len(overview),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
146:             },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
147:             'datasets': overview,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
148:             'generated_at': datetime.now(timezone.utc).isoformat(),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
149:         })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
150:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
151:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
152:
```
Blank line: code ko readable banane ke liye.
```
153: @app.route('/api/dashboard/metrics', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
154: def api_dashboard_metrics():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
155:     """Return live category, UPC quality, and Master Brand dashboard metrics."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
156:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
157:         return jsonify({
```
Return: calculated result caller ko wapas bhejta hai.
```
158:             'backend': db_manager.backend_name(),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
159:             **sql_engine.get_dashboard_metrics(),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
160:         })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
161:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
162:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
163:
```
Blank line: code ko readable banane ke liye.
```
164: @app.route('/api/master-brands/search', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
165: def api_master_brand_search():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
166:     master_id = (request.args.get('id') or '').strip()
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
167:     brand = (request.args.get('brand') or '').strip()
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
168:     upc = (request.args.get('upc') or '').strip()
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
169:     product = (request.args.get('product') or '').strip()
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
170:     query = (request.args.get('q') or '').strip()
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
171:     if not any((master_id, brand, upc, product, query)):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
172:         return jsonify({'error': 'Provide a Master Brand ID, brand, UPC, product, or search term'}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
173:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
174:         limit = max(1, min(int(request.args.get('limit', 100)), 500))
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
175:     except (TypeError, ValueError):
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
176:         return jsonify({'error': 'limit must be a valid number'}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
177:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
178:         return jsonify(sql_engine.search_master_brands(
```
Return: calculated result caller ko wapas bhejta hai.
```
179:             query=query, master_id=master_id or None, brand=brand or None,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
180:             upc=upc or None, product=product or None, limit=limit,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
181:         ))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
182:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
183:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
184:
```
Blank line: code ko readable banane ke liye.
```
185: @app.route('/api/master-brands/export', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
186: def api_master_brand_export():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
187:     response = api_master_brand_search()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
188:     if isinstance(response, tuple):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
189:         return response
```
Return: calculated result caller ko wapas bhejta hai.
```
190:     if response.status_code != 200:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
191:         return response
```
Return: calculated result caller ko wapas bhejta hai.
```
192:     payload = response.get_json()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
193:     token = str(__import__('uuid').uuid4())[:8]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
194:     frame = pd.DataFrame(payload.get('records') or [], columns=payload.get('columns') or None)
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
195:     frame.to_csv(REPORTS_DIR / f'report_{token}.csv', index=False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
196:     return jsonify({**payload, 'download_token': token})
```
Return: calculated result caller ko wapas bhejta hai.
```
197:
```
Blank line: code ko readable banane ke liye.
```
198: @app.route('/api/dashboard/categories', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
199: def api_dashboard_categories():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
200:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
201:         return jsonify({'backend': db_manager.backend_name(), 'categories': sql_engine.count_by_category()})
```
Return: calculated result caller ko wapas bhejta hai.
```
202:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
203:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
204:
```
Blank line: code ko readable banane ke liye.
```
205: @app.route('/api/dashboard/missing-upc', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
206: def api_dashboard_missing_upc():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
207:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
208:         limit = max(1, min(int(request.args.get('limit', 100)), 500))
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
209:         search = (request.args.get('search') or '').strip()[:100]
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
210:         result = comparison_engine.compare_all_missing_in_master(limit_per_category=limit, search=search or None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
211:         return jsonify({'backend': db_manager.backend_name(), **result})
```
Return: calculated result caller ko wapas bhejta hai.
```
212:     except (TypeError, ValueError):
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
213:         return jsonify({'error': 'limit must be a valid number'}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
214:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
215:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
216:
```
Blank line: code ko readable banane ke liye.
```
217: @app.route('/api/dashboard/export/<dataset>', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.

```
218: def api_dashboard_export(dataset):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
219:     if dataset not in CSV_FILES:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
220:         return jsonify({'error': f'Unknown dataset: {dataset}'}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
221:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
222:         limit = max(1, min(int(request.args.get('limit', 50000)), 50000))
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
223:         table = CSV_FILES[dataset]['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
224:         rows = db_manager.query(f"SELECT * FROM `{table}` LIMIT {limit}")
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
225:         frame = pd.DataFrame(rows)
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
226:         content = frame.to_csv(index=False).encode('utf-8')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
227:         return send_file(io.BytesIO(content), as_attachment=True,
```
Return: calculated result caller ko wapas bhejta hai.
```
228:                          download_name=f'beviq_{dataset}.csv', mimetype='text/csv')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
229:     except (TypeError, ValueError):
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
230:         return jsonify({'error': 'limit must be a valid number'}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
231:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
232:         return jsonify({'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
233:
```
Blank line: code ko readable banane ke liye.
```
234: @app.route('/api/db-status', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
235: def api_db_status():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
236:     database = db_manager.database_status()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
237:     ollama_ok = rag_orchestrator.is_ollama_available()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
238:     return jsonify({
```
Return: calculated result caller ko wapas bhejta hai.
```
239:         **database,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
240:         'mysql_port': MYSQL_PORT,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
241:         'mysql_db': MYSQL_DB,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
242:         'dbeaver_url': f"jdbc:mysql://{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DB}",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
243:         'ollama_connected': ollama_ok,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
244:         'ollama_model': OLLAMA_MODEL
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
245:     })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
246:
```
Blank line: code ko readable banane ke liye.
```
247: @app.route('/api/configure-db', methods=['POST'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
248: def api_configure_db():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
249:     data = request.get_json() or {}
```
Browser/API request se input, JSON, form ya uploaded file read ho rahi hai.
```
250:     password = data.get('password', '')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
251:     migrate = data.get('migrate', False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
252:
```
Blank line: code ko readable banane ke liye.
```
253:     db_manager.mysql_password = password
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
254:     ok, msg = db_manager.test_mysql_connection(password=password)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
255:
```
Blank line: code ko readable banane ke liye.
```
256:     if not ok:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
257:         db_manager.set_mysql_backend(False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
258:         return jsonify({'success': False, 'message': msg}), 400
```
Return: calculated result caller ko wapas bhejta hai.
```
259:
```
Blank line: code ko readable banane ke liye.
```
260:     db_manager.set_mysql_backend(True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
261:
```
Blank line: code ko readable banane ke liye.
```
262:     if migrate:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
263:         mig_ok, mig_msg = migrate_data_to_mysql(password=password)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
264:         return jsonify({'success': mig_ok, 'message': mig_msg, 'backend': db_manager.backend_name()})
```
Return: calculated result caller ko wapas bhejta hai.
```
265:
```
Blank line: code ko readable banane ke liye.
```
266:     return jsonify({'success': True, 'message': f"Connected to MySQL on port {MYSQL_PORT} successfully!",
```
Return: calculated result caller ko wapas bhejta hai.
```
267:                     'backend': db_manager.backend_name()})
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
268:
```
Blank line: code ko readable banane ke liye.
```
269: @app.route('/api/rebuild-index', methods=['POST'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
270: def api_rebuild_index():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
271:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
272:         count = vector_engine.build_from_database(limit_per_table=10000)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
273:         return jsonify({'success': True, 'indexed_documents': count})
```
Return: calculated result caller ko wapas bhejta hai.
```
274:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
275:         return jsonify({'success': False, 'error': str(e)}), 500
```
Return: calculated result caller ko wapas bhejta hai.
```
276:
```
Blank line: code ko readable banane ke liye.
```
277: @app.route('/api/export-csv/<token>', methods=['GET'])
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
278: def api_export_csv(token):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
279:     csv_file = REPORTS_DIR / f"report_{token}.csv"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
280:     if not csv_file.exists():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
281:         return "Report file not found or expired", 404
```
Return: calculated result caller ko wapas bhejta hai.
```
282:             return send_file(csv_file, as_attachment=True, download_name=f"beverage_report_{token}.csv",
mimetype="text/csv")
```
Return: calculated result caller ko wapas bhejta hai.
```
283:
```
Blank line: code ko readable banane ke liye.
```
284: if __name__ == '__main__':
```
Entry-point guard: file direct run hone par startup code chalata hai.
```
285:     port = int(os.getenv("PORT", 5000))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
286:     print(f"Starting BevIQ on http://127.0.0.1:{port}")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
287:     app.run(host='0.0.0.0', port=port, debug=False, threaded=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
288:
```
Blank line: code ko readable banane ke liye.

File: config.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import os
```
Import: library/module ke functions yahan available karata hai.
```
 2: from pathlib import Path
```
Import: library/module ke functions yahan available karata hai.
```
 3:
```
Blank line: code ko readable banane ke liye.
```
 4: # Base Paths
```
Comment: developer ka note; Python ise execute nahi karta.
```
 5: BASE_DIR = Path(__file__).resolve().parent
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 6: DATA_DIR = BASE_DIR
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 7: DB_DIR = BASE_DIR / "db_storage"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 8: INDEX_DIR = BASE_DIR / "vector_indices"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 9: REPORTS_DIR = BASE_DIR / "generated_reports"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
10:
```
Blank line: code ko readable banane ke liye.
```
11: # Load .env file if present
```
Comment: developer ka note; Python ise execute nahi karta.
```
12: env_path = BASE_DIR / ".env"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
13: if env_path.exists():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
14:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
15:         with open(env_path, "r", encoding="utf-8") as f:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
16:             for line in f:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
17:                 line = line.strip()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18:                 if line and not line.startswith("#") and "=" in line:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:                     k, v = line.split("=", 1)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:                     os.environ.setdefault(k.strip(), v.strip())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:     except Exception:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
22:         pass
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
23:
```
Blank line: code ko readable banane ke liye.
```
24: # Ensure directories exist
```
Comment: developer ka note; Python ise execute nahi karta.
```
25: DB_DIR.mkdir(exist_ok=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
26: INDEX_DIR.mkdir(exist_ok=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
27: REPORTS_DIR.mkdir(exist_ok=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
28:
```
Blank line: code ko readable banane ke liye.
```
29: # Database Configurations
```
Comment: developer ka note; Python ise execute nahi karta.
```
30: # Primary Target: MySQL on localhost:3307 (as requested for DBeaver)
```
Comment: developer ka note; Python ise execute nahi karta.
```
31: MYSQL_HOST = os.getenv("MYSQL_HOST", "127.0.0.1")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
32: MYSQL_PORT = int(os.getenv("MYSQL_PORT", 3307))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
33: MYSQL_USER = os.getenv("MYSQL_USER", "root")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34: MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD", "")  # Loaded from .env or environment
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
35: MYSQL_DB = os.getenv("MYSQL_DB", "beverage_db")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:
```
Blank line: code ko readable banane ke liye.
```
37: # High-Performance Local SQLite Fallback & Cache (Works 100% offline with zero setup)
```
Comment: developer ka note; Python ise execute nahi karta.
```
38: SQLITE_PATH = DB_DIR / "beverage_data.sqlite3"
```
SQLite: local offline file database; fast indexed queries ke liye.
```
39:
```
Blank line: code ko readable banane ke liye.
```
40: # Ollama / LLM Configurations
```
Comment: developer ka note; Python ise execute nahi karta.
```
41: OLLAMA_HOST = os.getenv("OLLAMA_HOST", "http://localhost:11434")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
42: OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
43:
```
Blank line: code ko readable banane ke liye.
```
44: # CSV File Mappings
```
Comment: developer ka note; Python ise execute nahi karta.
```
45: CSV_FILES = {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
46:     "all_brands": {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
47:         "file": "$ALLBrand_202610291524.csv",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
48:         "delimiter": "|",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
49:         "table": "all_brands",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
50:         "upc_col": "UPCCode",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
51:         "alt_upc_col": "AlternateUPCCode",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
52:         "brand_col": "BrandName",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
53:         "supplier_col": "SupplierName",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
54:         "category_col": "ProductCategory"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
55:     },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
56:     "suppliers": {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
57:         "file": "$Supplier_202610291521.csv",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
58:         "delimiter": "|",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
59:         "table": "suppliers",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
60:         "upc_col": None,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
61:         "alt_upc_col": None,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
62:         "brand_col": None,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
63:         "supplier_col": "SupplierName",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
64:         "category_col": None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
65:     },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
66:     "ai_master": {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
67:         "file": "AIMasterBrandData.csv",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
68:         "delimiter": "|",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
69:         "table": "ai_master",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
70:         "upc_col": "UPCCode",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
71:         "alt_upc_col": "AlternateUPCCode",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
72:         "brand_col": "BrandName",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
73:         "supplier_col": "SupplierName",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
74:         "category_col": "ProductCategory"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
75:     },
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
76:     "cat_beer": {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
77:         "file": "Categorization 2026 - Beer.csv",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
78:         "delimiter": ",",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
79:         "table": "cat_beer",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
80:         "upc_col": "UPC 13 digit",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
81:         "alt_upc_col": None,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
82:         "brand_col": "Beer Brands Value",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
83:         "supplier_col": "Beer Vendors Value",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
84:         "category_col": "Beer Category Value"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
85:     },
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
86:     "cat_rtd": {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
87:         "file": "Categorization 2026 - RTDs.csv",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
88:         "delimiter": ",",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
89:         "table": "cat_rtd",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
90:         "upc_col": "UPC 13 digit",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
91:         "alt_upc_col": None,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
92:         "brand_col": "Brands Value",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
93:         "supplier_col": None,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
94:         "category_col": "RTD Category Value"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
95:     },
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
96:     "cat_spirits": {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
97:         "file": "Categorization 2026 - Spirits.csv",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
98:         "delimiter": ",",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
99:         "table": "cat_spirits",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
100:         "upc_col": "UPC 13 digit",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
101:         "alt_upc_col": None,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
102:         "brand_col": "Spirits Brands Value",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
103:         "supplier_col": "Spirits Suppliers Value",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
104:         "category_col": "Spirits Category Value"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
105:     },
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
106:     "cat_wine": {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
107:         "file": "Categorization 2026 - Wine.csv",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
108:         "delimiter": ",",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
109:         "table": "cat_wine",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
110:         "upc_col": "UPC 13 digit",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
111:         "alt_upc_col": None,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
112:         "brand_col": "Brands Value",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
113:         "supplier_col": "Wine Suppliers Value",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
114:         "category_col": "Categories Value"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
115:     }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
116: }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
117:
```

Blank line: code ko readable banane ke liye.

File: run.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import os
```
Import: library/module ke functions yahan available karata hai.
```
 2: import sys
```
Import: library/module ke functions yahan available karata hai.
```
 3: from pathlib import Path
```
Import: library/module ke functions yahan available karata hai.
```
 4: from config import SQLITE_PATH, INDEX_DIR
```
Import: library/module ke functions yahan available karata hai.
```
 5: from data_ingestion.loader import ingest_all_to_sqlite
```
Import: library/module ke functions yahan available karata hai.
```
 6: from engines.vector_engine import vector_engine
```
Import: library/module ke functions yahan available karata hai.
```
 7:
```
Blank line: code ko readable banane ke liye.
```
 8: def setup_environment():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
 9:     print("=" * 65)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
10:     print("              BEVIQ - INITIALIZATION SEQUENCE")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
11:     print("=" * 65)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
12:
```
Blank line: code ko readable banane ke liye.
```
13:     # 1. Check if SQLite cache exists and has records
```
Comment: developer ka note; Python ise execute nahi karta.
```
14:     db_needs_init = not SQLITE_PATH.exists() or SQLITE_PATH.stat().st_size < 1000000
```
SQLite: local offline file database; fast indexed queries ke liye.
```
15:
```
Blank line: code ko readable banane ke liye.
```
16:     if db_needs_init:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
17:         print("[*] Local database not found or incomplete. Ingesting 7 CSV files...")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18:         print("[*] Performing UPC recovery, leading zero preservation, and schema indexing...")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:         stats = ingest_all_to_sqlite()
```
SQLite: local offline file database; fast indexed queries ke liye.
```
20:         print("[+] Ingestion complete:", stats)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:     else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
22:         print(f"[+] Local SQLite database verified: {SQLITE_PATH} ({SQLITE_PATH.stat().st_size / (1024*1024):.2f}
MB)")
```
SQLite: local offline file database; fast indexed queries ke liye.
```
23:
```
Blank line: code ko readable banane ke liye.
```
24:     # 2. Check if vector index exists
```
Comment: developer ka note; Python ise execute nahi karta.
```
25:     index_file = INDEX_DIR / "beverage_vector_index.pkl"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
26:     if not index_file.exists():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
27:         print("[*] Building persistent local semantic vector index...")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
28:         count = vector_engine.build_from_database(limit_per_table=10000)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
29:         print(f"[+] Vector index built successfully with {count:,} semantic documents.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:     else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
31:         print(f"[+] Vector index verified: {index_file}")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
32:
```
Blank line: code ko readable banane ke liye.
```
33:     print("=" * 65)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:     print("  Application ready! Starting Flask Web Server on http://127.0.0.1:5000")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
35:     print("  DBeaver Target: MySQL on localhost:3307 (Database: beverage_db)")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:     print("=" * 65)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
37:
```
Blank line: code ko readable banane ke liye.
```
38: if __name__ == '__main__':
```
Entry-point guard: file direct run hone par startup code chalata hai.
```
39:     setup_environment()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
40:     from app import app
```
Import: library/module ke functions yahan available karata hai.
```
41:     port = int(os.getenv("PORT", 5000))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
42:     app.run(host='0.0.0.0', port=port, debug=False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
43:
```
Blank line: code ko readable banane ke liye.

File: document_rag.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: """Local document ingestion and retrieval for chat-scoped RAG conversations."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 2:
```
Blank line: code ko readable banane ke liye.
```
 3: from __future__ import annotations
```
Import: library/module ke functions yahan available karata hai.
```
 4:
```
Blank line: code ko readable banane ke liye.
```
 5: import csv
```
Import: library/module ke functions yahan available karata hai.
```
 6: import hashlib
```
Import: library/module ke functions yahan available karata hai.
```
 7: import os
```
Import: library/module ke functions yahan available karata hai.
```
 8: import re
```
Import: library/module ke functions yahan available karata hai.
```
 9: import sqlite3
```
Import: library/module ke functions yahan available karata hai.
```
10: import uuid
```
Import: library/module ke functions yahan available karata hai.
```
11: from pathlib import Path
```
Import: library/module ke functions yahan available karata hai.
```
12:
```
Blank line: code ko readable banane ke liye.
```
13: from werkzeug.utils import secure_filename
```
Import: library/module ke functions yahan available karata hai.
```
14:
```
Blank line: code ko readable banane ke liye.
```
15: from config import DB_DIR, REPORTS_DIR
```
Import: library/module ke functions yahan available karata hai.
```
16:
```
Blank line: code ko readable banane ke liye.
```
17: UPLOAD_DIR = REPORTS_DIR / "uploads"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18: UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19: DOC_DB = DB_DIR / "document_rag.sqlite3"
```
SQLite: local offline file database; fast indexed queries ke liye.
```
20: MAX_UPLOAD_BYTES = int(os.getenv("MAX_UPLOAD_BYTES", 25 * 1024 * 1024))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21: SUPPORTED_EXTENSIONS = {".pdf", ".doc", ".docx", ".txt", ".md", ".log", ".json", ".xml",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
22:                         ".csv", ".xlsx", ".xls", ".png", ".jpg", ".jpeg", ".webp", ".bmp", ".tiff"}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
23:
```
Blank line: code ko readable banane ke liye.
```
24:
```
Blank line: code ko readable banane ke liye.
```
25: def _connection():
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
26:     conn = sqlite3.connect(str(DOC_DB))
```
SQLite: local offline file database; fast indexed queries ke liye.
```
27:     conn.row_factory = sqlite3.Row
```
SQLite: local offline file database; fast indexed queries ke liye.
```
28:     conn.execute("""CREATE TABLE IF NOT EXISTS documents (
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
29:         document_id TEXT PRIMARY KEY, session_id TEXT NOT NULL, filename TEXT NOT NULL,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:         content_hash TEXT NOT NULL, created_at TEXT DEFAULT CURRENT_TIMESTAMP
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
31:     )""")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
32:     conn.execute("""CREATE TABLE IF NOT EXISTS document_chunks (
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
33:         id INTEGER PRIMARY KEY AUTOINCREMENT, document_id TEXT NOT NULL,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:         session_id TEXT NOT NULL, chunk_index INTEGER NOT NULL, page TEXT,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
35:         content TEXT NOT NULL
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:     )""")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
37:     conn.execute("CREATE INDEX IF NOT EXISTS idx_doc_chunks_session ON document_chunks(session_id)")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
38:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
39:         conn.execute("""CREATE VIRTUAL TABLE IF NOT EXISTS document_chunks_fts USING fts5(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
40:             document_id UNINDEXED, session_id UNINDEXED, filename UNINDEXED,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
41:             page UNINDEXED, content
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
42:         )""")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
43:     except sqlite3.OperationalError:
```
SQLite: local offline file database; fast indexed queries ke liye.
```
44:         pass
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
45:     conn.commit()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
46:     return conn
```
Return: calculated result caller ko wapas bhejta hai.
```
47:
```
Blank line: code ko readable banane ke liye.
```
48:
```
Blank line: code ko readable banane ke liye.
```
49: def _clean(text):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
50:     return re.sub(r"\s+", " ", str(text or "")).strip()
```
Return: calculated result caller ko wapas bhejta hai.
```
51:
```
Blank line: code ko readable banane ke liye.
```
52:
```
Blank line: code ko readable banane ke liye.
```
53: def _extract(path: Path):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
54:     suffix = path.suffix.lower()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
55:     if suffix == ".doc":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
56:         raise ValueError("DOC files require conversion to DOCX before upload; legacy binary DOC extraction is not
installed.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
57:     if suffix == ".pdf":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
58:         from pypdf import PdfReader
```
Import: library/module ke functions yahan available karata hai.
```
59:         pages = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
60:         for number, page in enumerate(PdfReader(str(path)).pages, 1):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
61:             text = _clean(page.extract_text())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
62:             if text:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
63:                 pages.append((str(number), text))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
64:         return pages
```
Return: calculated result caller ko wapas bhejta hai.
```
65:     if suffix == ".docx":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
66:         from docx import Document
```
Import: library/module ke functions yahan available karata hai.
```
67:         text = _clean("\n".join(p.text for p in Document(str(path)).paragraphs))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
68:         return [("1", text)] if text else []
```
Return: calculated result caller ko wapas bhejta hai.
```
69:     if suffix in {".txt", ".md", ".log", ".json", ".xml"}:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
70:         return [("1", _clean(path.read_text(encoding="utf-8", errors="ignore")))]
```
Return: calculated result caller ko wapas bhejta hai.
```
71:     if suffix == ".csv":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
72:         with path.open("r", encoding="utf-8-sig", errors="ignore", newline="") as handle:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
73:             rows = list(csv.reader(handle))[:5000]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
74:         return [("1", _clean("\n".join(" | ".join(row) for row in rows)))]
```
Return: calculated result caller ko wapas bhejta hai.
```
75:     if suffix in {".xlsx", ".xls"}:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
76:         import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
77:         sheets = pd.read_excel(path, sheet_name=None, nrows=5000)
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
78:         return [(str(name), _clean(df.fillna("").to_csv(index=False))) for name, df in sheets.items()]
```
Return: calculated result caller ko wapas bhejta hai.
```
79:     if suffix in {".png", ".jpg", ".jpeg", ".webp", ".bmp", ".tiff"}:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
80:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
81:             import pytesseract
```
Import: library/module ke functions yahan available karata hai.
```
82:             from PIL import Image
```
Import: library/module ke functions yahan available karata hai.
```
83:             text = _clean(pytesseract.image_to_string(Image.open(path)))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
84:             return [("image", text)] if text else []
```
Return: calculated result caller ko wapas bhejta hai.
```
85:         except Exception as exc:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
86:             raise RuntimeError("Image OCR needs Tesseract installed and available on PATH.") from exc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
87:     raise ValueError("Supported uploads: PDF, DOCX, TXT, MD, CSV, XLSX, and common image formats.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
88:
```
Blank line: code ko readable banane ke liye.
```
89:
```
Blank line: code ko readable banane ke liye.
```
90: def validate_upload(file_storage, max_bytes=MAX_UPLOAD_BYTES):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
91:     """Validate upload name, extension, and size before writing it to disk."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
92:     filename = secure_filename(file_storage.filename or "")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
93:     suffix = Path(filename).suffix.lower()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
94:     if not filename or suffix not in SUPPORTED_EXTENSIONS:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
95:         raise ValueError("Supported uploads: PDF, DOC/DOCX, TXT, MD, CSV, XLS/XLSX, and common image formats.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
96:     if max_bytes < 1:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
97:         raise ValueError("Upload size limit must be positive.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
98:     stream = file_storage.stream
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
99:     position = stream.tell()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
100:     stream.seek(0, os.SEEK_END)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
101:     size = stream.tell()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
102:     stream.seek(position)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
103:     if size > max_bytes:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
104:         raise ValueError(f"Uploaded file is too large; maximum allowed size is {max_bytes // (1024 * 1024)} MB.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
105:
```
Blank line: code ko readable banane ke liye.
```
106:
```
Blank line: code ko readable banane ke liye.
```
107: def _chunks(text, size=1200, overlap=180):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
108:     words = text.split()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
109:     if not words:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
110:         return []
```
Return: calculated result caller ko wapas bhejta hai.
```
111:     result = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
112:     start = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
113:     while start < len(words):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
114:         result.append(" ".join(words[start:start + size]))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
115:         start += max(1, size - overlap)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
116:     return result
```
Return: calculated result caller ko wapas bhejta hai.
```
117:
```
Blank line: code ko readable banane ke liye.
```
118:
```
Blank line: code ko readable banane ke liye.
```
119: def ingest(file_storage, session_id):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
120:     filename = secure_filename(file_storage.filename or "")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
121:     if not filename:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
122:         raise ValueError("Uploaded file has no valid filename.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
123:     validate_upload(file_storage)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
124:     document_id = uuid.uuid4().hex[:12]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
125:     path = UPLOAD_DIR / f"{document_id}_{filename}"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:     UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
127:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
128:         file_storage.save(path)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
129:         if path.stat().st_size > MAX_UPLOAD_BYTES:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
130:             raise ValueError(f"Uploaded file is too large; maximum allowed size is {MAX_UPLOAD_BYTES // (1024 *
1024)} MB.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
131:         pages = _extract(path)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
132:         chunks = [(page, chunk) for page, text in pages for chunk in _chunks(text)]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
133:         if not chunks:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
134:             raise ValueError("No readable text was found in the uploaded document.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
135:         digest = hashlib.sha256(path.read_bytes()).hexdigest()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
136:         conn = _connection()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
137:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
138:             conn.execute("INSERT INTO documents(document_id,session_id,filename,content_hash) VALUES(?,?,?,?)",
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
139:                          (document_id, session_id, filename, digest))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
140:                 conn.executemany("INSERT INTO document_chunks(document_id,session_id,chunk_index,page,content)
VALUES(?,?,?,?,?)",
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
141:                                  [(document_id, session_id, index, page, content) for index, (page, content) in
enumerate(chunks)])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
142:             try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
143:                 conn.executemany("INSERT INTO document_chunks_fts(document_id,session_id,filename,page,content)
VALUES(?,?,?,?,?)",
```
SQL: database se data read/write ya schema/index tayyar hota hai.

```
144:                                 [(document_id, session_id, filename, page, content) for page, content in chunks])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
145:             except sqlite3.OperationalError:
```
SQLite: local offline file database; fast indexed queries ke liye.
```
146:                 pass
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
147:             conn.commit()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
148:         finally:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
149:             conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
150:         return {"document_id": document_id, "filename": filename, "chunks": len(chunks)}
```
Return: calculated result caller ko wapas bhejta hai.
```
151:     except Exception:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
152:         path.unlink(missing_ok=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
153:         raise
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
154:
```
Blank line: code ko readable banane ke liye.
```
155:
```
Blank line: code ko readable banane ke liye.
```
156: def _tokens(text):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
157:     return set(re.findall(r"[a-z0-9]{2,}", text.lower()))
```
Return: calculated result caller ko wapas bhejta hai.
```
158:
```
Blank line: code ko readable banane ke liye.
```
159:
```
Blank line: code ko readable banane ke liye.
```
160: def search(query, session_id, top_k=6):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
161:     wanted = _tokens(query)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
162:     if not wanted:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
163:         return []
```
Return: calculated result caller ko wapas bhejta hai.
```
164:     conn = _connection()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
165:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
166:         fts_query = " AND ".join(f'"{token}"' for token in sorted(wanted))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
167:         rows = conn.execute("""SELECT document_id, session_id, page, content, filename
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
168:             FROM document_chunks_fts WHERE session_id=? AND document_chunks_fts MATCH ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
169:             LIMIT ?""", (session_id, fts_query, max(1, int(top_k)))).fetchall()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
170:         if rows:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
171:             conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
172:             return [{"document_id": row["document_id"], "filename": row["filename"], "page": row["page"],
```
Return: calculated result caller ko wapas bhejta hai.
```
173:                      "content": row["content"], "score": 1.0} for row in rows]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
174:     except sqlite3.OperationalError:
```
SQLite: local offline file database; fast indexed queries ke liye.
```
175:         pass
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
176:     rows = conn.execute("""SELECT c.*, d.filename FROM document_chunks c
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
177:         JOIN documents d ON d.document_id=c.document_id WHERE c.session_id=?""", (session_id,)).fetchall()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
178:     conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
179:     scored = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
180:     for row in rows:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
181:         tokens = _tokens(row["content"])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
182:         score = len(wanted & tokens) / max(1, len(wanted))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
183:         if score > 0:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
184:             scored.append((score, row))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
185:     scored.sort(key=lambda item: item[0], reverse=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
186:     return [{"document_id": r["document_id"], "filename": r["filename"], "page": r["page"],
```
Return: calculated result caller ko wapas bhejta hai.
```
187:              "content": r["content"], "score": round(score, 4)} for score, r in scored[:top_k]]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
188:
```
Blank line: code ko readable banane ke liye.
```
189:
```
Blank line: code ko readable banane ke liye.
```
190: def list_documents(session_id):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
191:     conn = _connection()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
192:       rows = conn.execute("SELECT document_id, filename, created_at FROM documents WHERE session_id=? ORDER BY created_at DESC", (session_id,)).fetchall()
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
193:     conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
194:     return [dict(row) for row in rows]
```
Return: calculated result caller ko wapas bhejta hai.

File: data_ingestion/normalizer.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import re
```
Import: library/module ke functions yahan available karata hai.
```
 2: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
 3:
```
Blank line: code ko readable banane ke liye.
```
 4: def normalize_upc(raw_val, product_str=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
 5:     """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 6:     Standardize beverage UPCs to 13-digit strings with leading zeros preserved.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 7:     Recovers UPC from product description string if raw_val suffered Excel truncation.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 8:     """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 9:     # 1. Check if product string has trailing 11-14 digit UPC
```
Comment: developer ka note; Python ise execute nahi karta.
```
10:     if product_str and isinstance(product_str, str):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
11:         m = re.search(r'-\s*(\d{11,14})\s*$', product_str.strip())
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
12:         if m:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
13:             digits = m.group(1)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
14:             return digits.zfill(13) if len(digits) in [11, 12] else digits
```
Return: calculated result caller ko wapas bhejta hai.
```
15:
```
Blank line: code ko readable banane ke liye.
```
16:     if pd.isna(raw_val):
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
17:         return ""
```
Return: calculated result caller ko wapas bhejta hai.
```
18:
```
Blank line: code ko readable banane ke liye.
```
19:     s = str(raw_val).strip()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:     if not s or s.lower() in ['nan', 'null', 'none']:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:         return ""
```
Return: calculated result caller ko wapas bhejta hai.
```
22:
```
Blank line: code ko readable banane ke liye.
```
23:     # 2. Remove 'UPC=' or similar prefixes
```
Comment: developer ka note; Python ise execute nahi karta.
```
24:     s = s.replace('UPC=', '').replace('upc=', '').strip()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
25:
```
Blank line: code ko readable banane ke liye.
```
26:     # 3. Handle scientific notation (e.g. 6.19947E+11)
```
Comment: developer ka note; Python ise execute nahi karta.
```
27:     if 'e+' in s.lower() or 'e-' in s.lower():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
28:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
29:             s = f"{int(float(s))}"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:         except Exception:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
31:             pass
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
32:
```
Blank line: code ko readable banane ke liye.
```
33:     # 4. Remove trailing decimals (e.g. 18200967151.0)
```
Comment: developer ka note; Python ise execute nahi karta.
```
34:     if '.' in s:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
35:         parts = s.split('.')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:         if len(parts) == 2 and (parts[1] == '0' or parts[1] == ''):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
37:             s = parts[0]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
38:
```
Blank line: code ko readable banane ke liye.
```
39:     # 5. Extract only digits
```
Comment: developer ka note; Python ise execute nahi karta.
```
40:     digits = re.sub(r'\D', '', s)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
41:     if not digits:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
42:         return ""
```
Return: calculated result caller ko wapas bhejta hai.
```
43:
```
Blank line: code ko readable banane ke liye.
```
44:     # Standard 13-digit UPC formatting with leading zeros
```
Comment: developer ka note; Python ise execute nahi karta.
```
45:     if len(digits) in [11, 12]:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
46:         return digits.zfill(13)
```
Return: calculated result caller ko wapas bhejta hai.
```
47:     return digits
```
Return: calculated result caller ko wapas bhejta hai.
```
48:
```
Blank line: code ko readable banane ke liye.
```
49: def clean_text(val):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
50:     """Clean string values, handling nulls, multiple spaces, and special quotes."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
51:     if pd.isna(val):
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
52:         return ""
```
Return: calculated result caller ko wapas bhejta hai.
```
53:     s = str(val).strip()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
54:     if s.lower() in ['nan', 'null', 'none', '#n/a', 'na']:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
55:         return ""
```
Return: calculated result caller ko wapas bhejta hai.
```
56:     # normalize internal whitespace
```
Comment: developer ka note; Python ise execute nahi karta.
```
57:     s = re.sub(r'\s+', ' ', s)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
58:     return s
```
Return: calculated result caller ko wapas bhejta hai.
```
59:
```
Blank line: code ko readable banane ke liye.

File: data_ingestion/loader.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import os
```
Import: library/module ke functions yahan available karata hai.
```
 2: import re
```
Import: library/module ke functions yahan available karata hai.
```
 3: import csv
```
Import: library/module ke functions yahan available karata hai.
```
 4: import sqlite3
```
Import: library/module ke functions yahan available karata hai.
```
 5: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
 6: import logging
```
Import: library/module ke functions yahan available karata hai.
```
 7: from config import CSV_FILES, SQLITE_PATH
```
Import: library/module ke functions yahan available karata hai.
```
 8: from data_ingestion.normalizer import normalize_upc, clean_text
```
Import: library/module ke functions yahan available karata hai.
```
 9:
```
Blank line: code ko readable banane ke liye.
```
10: logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
11: logger = logging.getLogger("DataLoader")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
12:
```
Blank line: code ko readable banane ke liye.
```
13: def clean_column_name(col):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
14:     """Normalize CSV column names for safe SQL table column names."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
15:     col = col.strip()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
16:     col = re.sub(r'[\s\-/\(\)\.\$]+', '_', col)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
17:     col = re.sub(r'_+', '_', col).strip('_')
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
18:     if col and col[0].isdigit():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:         col = 'col_' + col
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:     return col or 'unnamed_col'
```
Return: calculated result caller ko wapas bhejta hai.
```
21:
```
Blank line: code ko readable banane ke liye.
```
22: def ingest_csv_to_sqlite(key, config_entry, progress_callback=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
23:     fname = config_entry["file"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
24:     delimiter = config_entry["delimiter"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
25:     table_name = config_entry["table"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
26:     upc_col = config_entry.get("upc_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
27:     alt_upc_col = config_entry.get("alt_upc_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
28:     brand_col = config_entry.get("brand_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
29:     supplier_col = config_entry.get("supplier_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:
```
Blank line: code ko readable banane ke liye.
```
31:     if not os.path.exists(fname):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
32:         logger.error(f"File not found: {fname}")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
33:         return 0
```
Return: calculated result caller ko wapas bhejta hai.
```
34:
```
Blank line: code ko readable banane ke liye.
```
35:     conn = sqlite3.connect(str(SQLITE_PATH))
```

SQLite: local offline file database; fast indexed queries ke liye.

```
36:     conn.execute("PRAGMA journal_mode = WAL")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
37:     conn.execute("PRAGMA synchronous = NORMAL")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
38:     conn.execute("PRAGMA cache_size = 100000")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
39:
```

Blank line: code ko readable banane ke liye.

```
40:     # Drop existing table if refreshing
```

Comment: developer ka note; Python ise execute nahi karta.

```
41:     conn.execute(f"DROP TABLE IF EXISTS `{table_name}`")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
42:     conn.commit()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
43:
```

Blank line: code ko readable banane ke liye.

```
44:     chunksize = 25000
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
45:     total_processed = 0
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
46:     row_offset = 1
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
47:
```

Blank line: code ko readable banane ke liye.

```
48:     logger.info(f"Starting ingestion of {fname} into SQLite table '{table_name}'...")
```

SQLite: local offline file database; fast indexed queries ke liye.

```
49:
```

Blank line: code ko readable banane ke liye.

```
50:     for chunk in pd.read_csv(fname, sep=delimiter, chunksize=chunksize, dtype=str, on_bad_lines='skip'):
```

Pandas: CSV/table data ko DataFrame mein process karta hai.

```
51:         chunk_len = len(chunk)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
52:         chunk['_source_file'] = os.path.basename(fname)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
53:         chunk['_original_row'] = list(range(row_offset, row_offset + chunk_len))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
54:         row_offset += chunk_len
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
55:
```

Blank line: code ko readable banane ke liye.

```
56:         # Extract normalized 13-digit UPC
```

Comment: developer ka note; Python ise execute nahi karta.

```
57:          prod_series = chunk['Product'] if 'Product' in chunk.columns else (chunk['ProductName'] if 'ProductName'
in chunk.columns else None)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
58:
```

Blank line: code ko readable banane ke liye.

```
59:         if upc_col and upc_col in chunk.columns:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
60:             if prod_series is not None:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
61:                 chunk['clean_upc'] = [normalize_upc(u, p) for u, p in zip(chunk[upc_col], prod_series)]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
62:             else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
63:                 chunk['clean_upc'] = [normalize_upc(u) for u in chunk[upc_col]]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
64:         else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
65:             chunk['clean_upc'] = ""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
66:
```

Blank line: code ko readable banane ke liye.

```
67:         # Normalize Alternate UPC if present
```

Comment: developer ka note; Python ise execute nahi karta.

```
68:         if alt_upc_col and alt_upc_col in chunk.columns:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
69:             chunk['clean_alt_upc'] = [normalize_upc(u) for u in chunk[alt_upc_col]]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
70:         else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
71:             chunk['clean_alt_upc'] = ""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
72:
```
Blank line: code ko readable banane ke liye.
```
73:         # Normalize standardized brand and supplier column
```
Comment: developer ka note; Python ise execute nahi karta.
```
74:         if brand_col and brand_col in chunk.columns:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
75:             chunk['norm_brand'] = chunk[brand_col].fillna("").astype(str).str.strip().str.upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
76:         else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
77:             chunk['norm_brand'] = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
78:
```
Blank line: code ko readable banane ke liye.
```
79:         if supplier_col and supplier_col in chunk.columns:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
80:             chunk['norm_supplier'] = chunk[supplier_col].fillna("").astype(str).str.strip().str.upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
81:         else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
82:             chunk['norm_supplier'] = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
83:
```
Blank line: code ko readable banane ke liye.
```
84:         # Clean all column names for SQL safety
```
Comment: developer ka note; Python ise execute nahi karta.
```
85:         chunk.columns = [clean_column_name(c) for c in chunk.columns]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
86:
```
Blank line: code ko readable banane ke liye.
```
87:         # Append to SQLite table
```
Comment: developer ka note; Python ise execute nahi karta.
```
88:         chunk.to_sql(table_name, conn, if_exists='append', index=False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
89:         total_processed += chunk_len
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
90:
```
Blank line: code ko readable banane ke liye.
```
91:         if progress_callback:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
92:             progress_callback(key, total_processed)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
93:         logger.info(f"  {table_name}: Ingested {total_processed:,} records...")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
94:
```
Blank line: code ko readable banane ke liye.
```
95:     # Create helpful indexes for lightning-fast queries
```
Comment: developer ka note; Python ise execute nahi karta.
```
96:     logger.info(f"Creating indexes for table '{table_name}'...")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
97:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
98:         available_columns = {row[1] for row in conn.execute(f"PRAGMA table_info(`{table_name}`)").fetchall()}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
99:         index_columns = {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
100:             'clean_upc': 'upc',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
101:             'clean_alt_upc': 'alt_upc',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
102:             'norm_brand': 'brand',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
103:             'norm_supplier': 'supplier',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
104:             'Id': 'id',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
105:             'id': 'id_value',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
106:             'SupplierId': 'supplier_id',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
107:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
108:         for column, suffix in index_columns.items():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
109:             if column in available_columns:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
110:                                 conn.execute(f"CREATE  INDEX  IF  NOT  EXISTS  idx_{table_name}_{suffix}  ON
`{table_name}`(`{column}`)")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
111:         conn.commit()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
112:     except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
113:         logger.warning(f"Index creation warning: {e}")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
114:
```
Blank line: code ko readable banane ke liye.
```
115:     conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
116:     logger.info(f"Finished '{table_name}': {total_processed:,} records ingested successfully.")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
117:     return total_processed
```
Return: calculated result caller ko wapas bhejta hai.
```
118:
```
Blank line: code ko readable banane ke liye.
```
119: def ingest_all_to_sqlite(progress_callback=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
120:     """Ingest all 7 datasets into SQLite local database."""
```
SQLite: local offline file database; fast indexed queries ke liye.
```
121:     total_all = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
122:     stats = {}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
123:     for key, config_entry in CSV_FILES.items():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
124:         count = ingest_csv_to_sqlite(key, config_entry, progress_callback)
```
SQLite: local offline file database; fast indexed queries ke liye.
```
125:         stats[key] = count
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:         total_all += count
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
127:     logger.info(f"ALL DATASETS INGESTED. Total records: {total_all:,}")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
128:     return stats
```
Return: calculated result caller ko wapas bhejta hai.
```
129:
```
Blank line: code ko readable banane ke liye.

File: database/db_manager.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import sqlite3
```
Import: library/module ke functions yahan available karata hai.
```
 2: import pymysql
```
Import: library/module ke functions yahan available karata hai.
```
 3: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
 4: import logging
```
Import: library/module ke functions yahan available karata hai.
```
 5: from datetime import datetime, timezone
```
Import: library/module ke functions yahan available karata hai.
```
 6: from config import SQLITE_PATH, MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DB, CSV_FILES
```
Import: library/module ke functions yahan available karata hai.
```
 7:
```
Blank line: code ko readable banane ke liye.
```
 8: logger = logging.getLogger("DBManager")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 9:
```
Blank line: code ko readable banane ke liye.
```
10: class DBManager:
```
Class: related data aur methods ka reusable blueprint.
```
11:     def __init__(self, use_mysql=False, mysql_password=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
12:         self.use_mysql = use_mysql
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
13:         self._active_backend = "mysql" if use_mysql else "sqlite"
```
SQLite: local offline file database; fast indexed queries ke liye.
```
14:         self.mysql_password = mysql_password if mysql_password is not None else MYSQL_PASSWORD
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
15:         self.sqlite_path = str(SQLITE_PATH)
```
SQLite: local offline file database; fast indexed queries ke liye.
```
16:         self.last_mysql_check = None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
17:         self.last_sqlite_check = None
```
SQLite: local offline file database; fast indexed queries ke liye.
```
18:
```
Blank line: code ko readable banane ke liye.
```
19:     def set_mysql_backend(self, enabled):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
20:         """Select MySQL as primary or return to the local SQLite fallback."""
```
Return: calculated result caller ko wapas bhejta hai.
```
21:         self.use_mysql = bool(enabled)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
22:         self._active_backend = "mysql" if self.use_mysql else "sqlite"
```
SQLite: local offline file database; fast indexed queries ke liye.
```
23:
```
Blank line: code ko readable banane ke liye.
```
24:     def backend_name(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
25:         """Return the backend currently selected for application queries."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
26:         return self._active_backend
```
Return: calculated result caller ko wapas bhejta hai.
```
27:
```
Blank line: code ko readable banane ke liye.
```
28:     def test_mysql_connection(self, password=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
29:         """Test if MySQL server on port 3307 is reachable and authenticates."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:         pwd = password if password is not None else self.mysql_password
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
31:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
32:             conn = pymysql.connect(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
33:                 host=MYSQL_HOST,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:                 port=MYSQL_PORT,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
35:                 user=MYSQL_USER,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:                 password=pwd,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
37:                 connect_timeout=3
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
38:             )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
39:             version = None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
40:             with conn.cursor() as cur:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
41:                 cur.execute("SELECT VERSION()")
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
42:                 version = cur.fetchone()[0]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
43:             conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
44:             self.last_mysql_check = datetime.now(timezone.utc).isoformat()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
45:             return True, f"Connected to MySQL {version} on {MYSQL_HOST}:{MYSQL_PORT}"
```
Return: calculated result caller ko wapas bhejta hai.
```
46:         except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
47:             self.last_mysql_check = datetime.now(timezone.utc).isoformat()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
48:             return False, str(e)
```
Return: calculated result caller ko wapas bhejta hai.
```
49:
```
Blank line: code ko readable banane ke liye.
```
50:     def get_mysql_connection(self, create_db=True):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
51:         """Obtain PyMySQL connection to MySQL 3307."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
52:         if create_db:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
53:             # First connect without DB to ensure DB exists
```
Comment: developer ka note; Python ise execute nahi karta.
```
54:             root_conn = pymysql.connect(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
55:                 host=MYSQL_HOST,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
56:                 port=MYSQL_PORT,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
57:                 user=MYSQL_USER,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
58:                 password=self.mysql_password,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
59:                 connect_timeout=5
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
60:             )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
61:             with root_conn.cursor() as cur:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 62:                 cur.execute(f"CREATE DATABASE IF NOT EXISTS `{MYSQL_DB}` CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
63:             root_conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
64:
```
Blank line: code ko readable banane ke liye.
```
65:         return pymysql.connect(
```
Return: calculated result caller ko wapas bhejta hai.
```
66:             host=MYSQL_HOST,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
67:             port=MYSQL_PORT,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
68:             user=MYSQL_USER,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
69:             password=self.mysql_password,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
70:             database=MYSQL_DB,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
71:             charset='utf8mb4',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            cursorclass=pymysql.cursors.DictCursor
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
    def get_sqlite_connection(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
        """Obtain high-performance local SQLite connection."""
```
SQLite: local offline file database; fast indexed queries ke liye.
```
        conn = sqlite3.connect(self.sqlite_path)
```
SQLite: local offline file database; fast indexed queries ke liye.
```
        conn.row_factory = sqlite3.Row
```
SQLite: local offline file database; fast indexed queries ke liye.
```
        conn.execute("PRAGMA journal_mode = WAL")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        conn.execute("PRAGMA synchronous = NORMAL")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        conn.execute("PRAGMA cache_size = 100000")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        conn.execute("PRAGMA temp_store = MEMORY")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        return conn
```
Return: calculated result caller ko wapas bhejta hai.
```

```
Blank line: code ko readable banane ke liye.
```
    def table_columns(self, table):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
        """Return column names for a configured table on the active backend."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        if self.use_mysql:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            rows = self.query(f"SHOW COLUMNS FROM `{table}`")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            return [row.get("Field") or row.get("field") for row in rows]
```
Return: calculated result caller ko wapas bhejta hai.
```
        rows = self.query(f"PRAGMA table_info(`{table}`)")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        return [row.get("name") for row in rows]
```
Return: calculated result caller ko wapas bhejta hai.
```

```
Blank line: code ko readable banane ke liye.
```
    def mysql_schema_status(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
        """Check that MySQL has the normalized tables this application queries."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        required = {entry["table"] for entry in CSV_FILES.values()}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
            conn = self.get_mysql_connection(create_db=False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
                with conn.cursor() as cur:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                    cur.execute("SHOW TABLES")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                    rows = cur.fetchall()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            finally:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            available = {next(iter(row.values())) for row in rows}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            missing = sorted(required - available)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            if missing:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                return False, f"MySQL is reachable but normalized beverage tables are missing: {',
'.join(missing)}"
```
Return: calculated result caller ko wapas bhejta hai.
```
            return True, "MySQL normalized beverage schema is ready"
```

Return: calculated result caller ko wapas bhejta hai.
```
109:         except Exception as exc:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
110:             return False, f"MySQL schema check failed: {exc}"
```
Return: calculated result caller ko wapas bhejta hai.
```
111:
```
Blank line: code ko readable banane ke liye.
```
112:     def database_status(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
113:         """Return safe connection state without exposing credentials."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
114:         sqlite_ok = False
```
SQLite: local offline file database; fast indexed queries ke liye.
```
115:         sqlite_message = "Unavailable"
```
SQLite: local offline file database; fast indexed queries ke liye.
```
116:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
117:             conn = self.get_sqlite_connection()
```
SQLite: local offline file database; fast indexed queries ke liye.
```
118:             conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
119:             sqlite_ok = True
```
SQLite: local offline file database; fast indexed queries ke liye.
```
120:             sqlite_message = "SQLite database is available"
```
SQLite: local offline file database; fast indexed queries ke liye.
```
121:             self.last_sqlite_check = datetime.now(timezone.utc).isoformat()
```
SQLite: local offline file database; fast indexed queries ke liye.
```
122:         except Exception as exc:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
123:             sqlite_message = str(exc)
```
SQLite: local offline file database; fast indexed queries ke liye.
```
124:
```
Blank line: code ko readable banane ke liye.
```
125:         mysql_ok, mysql_message = self.test_mysql_connection()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:             mysql_schema_ready, mysql_schema_message = self.mysql_schema_status() if mysql_ok else (False, "MySQL
schema was not checked because the server is unavailable")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
127:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
128:             "sqlite_connected": sqlite_ok,
```
SQLite: local offline file database; fast indexed queries ke liye.
```
129:             "sqlite_message": sqlite_message,
```
SQLite: local offline file database; fast indexed queries ke liye.
```
130:             "mysql_connected": mysql_ok,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
131:             "mysql_message": mysql_message,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
132:             "mysql_schema_ready": mysql_schema_ready,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
133:             "mysql_schema_message": mysql_schema_message,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
134:             "active_backend": self.backend_name(),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
135:             "database_name": MYSQL_DB if self.use_mysql else "beverage_data.sqlite3",
```
SQLite: local offline file database; fast indexed queries ke liye.
```
136:             "last_sqlite_check": self.last_sqlite_check,
```
SQLite: local offline file database; fast indexed queries ke liye.
```
137:             "last_mysql_check": self.last_mysql_check,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
138:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
139:
```
Blank line: code ko readable banane ke liye.
```
140:     def get_connection(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
141:         """Returns MySQL connection if configured and available, otherwise SQLite."""
```
SQLite: local offline file database; fast indexed queries ke liye.
```
142:         if self.use_mysql:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
143:             ok, _ = self.test_mysql_connection()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
144:             if ok:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
145:                 return self.get_mysql_connection()
```
Return: calculated result caller ko wapas bhejta hai.
```
146:             self.set_mysql_backend(False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
147:         return self.get_sqlite_connection()
```
Return: calculated result caller ko wapas bhejta hai.
```
148:
```
Blank line: code ko readable banane ke liye.
```
149:     def query(self, sql, params=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
150:         """Execute query and return list of dicts."""
```
Return: calculated result caller ko wapas bhejta hai.
```
151:         if self.use_mysql:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
152:             try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
153:                 if self.test_mysql_connection()[0]:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
154:                     conn = self.get_mysql_connection()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
155:                     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
156:                         with conn.cursor() as cur:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
157:                             cur.execute(sql.replace("?", "%s"), params or ())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
158:                             return cur.fetchall()
```
Return: calculated result caller ko wapas bhejta hai.
```
159:                     finally:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
160:                         conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
161:                 self.set_mysql_backend(False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
162:             except Exception as exc:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
163:                 logger.warning("MySQL query failed; using SQLite fallback: %s", exc)
```
SQLite: local offline file database; fast indexed queries ke liye.
```
164:                 self.set_mysql_backend(False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
165:
```
Blank line: code ko readable banane ke liye.
```
166:         conn = self.get_sqlite_connection()
```
SQLite: local offline file database; fast indexed queries ke liye.
```
167:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
168:             cur = conn.cursor()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
169:             cur.execute(sql, params or ())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
170:             rows = cur.fetchall()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
171:             return [dict(r) for r in rows]
```
Return: calculated result caller ko wapas bhejta hai.
```
172:         finally:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
173:             conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
174:
```
Blank line: code ko readable banane ke liye.
```
175:     def query_df(self, sql, params=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
176:         """Execute query and return Pandas DataFrame."""
```
Return: calculated result caller ko wapas bhejta hai.
```
177:         if self.use_mysql:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
178:             try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
179:                 if self.test_mysql_connection()[0]:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
180:                     conn = self.get_mysql_connection()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
181:                     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.

```
182:                         return pd.read_sql(sql.replace("?", "%s"), conn, params=params)
```
Return: calculated result caller ko wapas bhejta hai.
```
183:                     finally:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
184:                         conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
185:                 self.set_mysql_backend(False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
186:             except Exception as exc:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
187:                 logger.warning("MySQL dataframe query failed; using SQLite fallback: %s", exc)
```
SQLite: local offline file database; fast indexed queries ke liye.
```
188:                 self.set_mysql_backend(False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
189:
```
Blank line: code ko readable banane ke liye.
```
190:         conn = self.get_sqlite_connection()
```
SQLite: local offline file database; fast indexed queries ke liye.
```
191:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
192:             return pd.read_sql(sql, conn, params=params)
```
Return: calculated result caller ko wapas bhejta hai.
```
193:         finally:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
194:             conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
195:
```
Blank line: code ko readable banane ke liye.
```
196: # The application is MySQL-primary for the beverage task. Every query still
```
Comment: developer ka note; Python ise execute nahi karta.
```
197: # falls back to SQLite if MySQL becomes unavailable, so local development and
```
Comment: developer ka note; Python ise execute nahi karta.
```
198: # offline usage remain safe without silently making SQLite the intended source.
```
Comment: developer ka note; Python ise execute nahi karta.
```
199: db_manager = DBManager(use_mysql=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
200:
```
Blank line: code ko readable banane ke liye.

File: database/mysql_migrator.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import os
```
Import: library/module ke functions yahan available karata hai.
```
 2: import re
```
Import: library/module ke functions yahan available karata hai.
```
 3: import pymysql
```
Import: library/module ke functions yahan available karata hai.
```
 4: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
 5: import logging
```
Import: library/module ke functions yahan available karata hai.
```
 6: from config import CSV_FILES, MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DB
```
Import: library/module ke functions yahan available karata hai.
```
 7: from data_ingestion.normalizer import normalize_upc
```
Import: library/module ke functions yahan available karata hai.
```
 8: from data_ingestion.loader import clean_column_name
```
Import: library/module ke functions yahan available karata hai.
```
 9:
```
Blank line: code ko readable banane ke liye.
```
10: logger = logging.getLogger("MySQLMigrator")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
11:
```
Blank line: code ko readable banane ke liye.
```
12: def migrate_data_to_mysql(password=None, host=None, port=None, user=None, db_name=None, progress_callback=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
13:     """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
14:     Ingests all beverage CSV files directly into MySQL server on port 3307.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
15:     Enables viewing in DBeaver immediately.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
16:     """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
17:     host = host or MYSQL_HOST
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18:     port = port or MYSQL_PORT
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:     user = user or MYSQL_USER
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:     password = password if password is not None else MYSQL_PASSWORD
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:     db_name = db_name or MYSQL_DB
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
22:
```
Blank line: code ko readable banane ke liye.
```
23:     # 1. Connect and create database
```
Comment: developer ka note; Python ise execute nahi karta.
```
24:     try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
25:         conn = pymysql.connect(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
26:             host=host,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
27:             port=port,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
28:             user=user,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
29:             password=password,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:             connect_timeout=10
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
31:         )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
32:         with conn.cursor() as cur:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
33:                     cur.execute(f"CREATE DATABASE IF NOT EXISTS `{db_name}` CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:         conn.close()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
    except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
        return False, f"Failed to connect to MySQL on {host}:{port}: {e}"
```
Return: calculated result caller ko wapas bhejta hai.
```

```
Blank line: code ko readable banane ke liye.
```
    # 2. Connect to the created database
```
Comment: developer ka note; Python ise execute nahi karta.
```
    conn = pymysql.connect(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        host=host,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        port=port,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        user=user,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        password=password,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        database=db_name,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        charset='utf8mb4'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
    )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
    results = {}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
    try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
        for key, config_entry in CSV_FILES.items():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            fname = config_entry["file"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            delimiter = config_entry["delimiter"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            table_name = config_entry["table"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            upc_col = config_entry.get("upc_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            alt_upc_col = config_entry.get("alt_upc_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            brand_col = config_entry.get("brand_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            supplier_col = config_entry.get("supplier_col")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
            if not os.path.exists(fname):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                continue
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
            # Read sample to define columns
```
Comment: developer ka note; Python ise execute nahi karta.
```
            df_sample = pd.read_csv(fname, sep=delimiter, nrows=5, dtype=str)
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
            raw_cols = df_sample.columns.tolist()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            # Keep generated metadata names aligned with the normalizer used by both
```
Comment: developer ka note; Python ise execute nahi karta.
```
            # SQLite and the query engine. Leading underscores are stripped by
```
Comment: developer ka note; Python ise execute nahi karta.
```
            # clean_column_name(), which otherwise turns these values into blanks
```
Comment: developer ka note; Python ise execute nahi karta.
```
            # during reindexing and fails under MySQL strict mode.
```
Comment: developer ka note; Python ise execute nahi karta.
```
            sql_cols = [clean_column_name(c) for c in raw_cols] + ['source_file', 'original_row', 'clean_upc',
'clean_alt_upc', 'norm_brand', 'norm_supplier']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```

Blank line: code ko readable banane ke liye.

```
72:             # Deduplicate sql_cols preserving order
```

Comment: developer ka note; Python ise execute nahi karta.

```
73:             unique_cols = []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
74:             for col in sql_cols:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
75:                 if col not in unique_cols:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
76:                     unique_cols.append(col)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
77:
```

Blank line: code ko readable banane ke liye.

```
78:             # Drop old table
```

Comment: developer ka note; Python ise execute nahi karta.

```
79:             with conn.cursor() as cur:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
80:                 cur.execute(f"DROP TABLE IF EXISTS `{table_name}`")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
81:
```

Blank line: code ko readable banane ke liye.

```
82:                 # Build CREATE TABLE
```

Comment: developer ka note; Python ise execute nahi karta.

```
83:                 col_defs = []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
84:                 for c in unique_cols:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
85:                     if c == 'clean_upc':
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
86:                         col_defs.append(f"`{c}` VARCHAR(64)")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
87:                     elif c == 'clean_alt_upc':
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
88:                         # AlternateUPCCode can contain a set of UPCs such as
```

Comment: developer ka note; Python ise execute nahi karta.

```
89:                         # {081...,081...}; never truncate it in MySQL.
```

Comment: developer ka note; Python ise execute nahi karta.

```
90:                         col_defs.append(f"`{c}` TEXT")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
91:                     elif c in ['norm_brand', 'norm_supplier', 'BrandName', 'SupplierName']:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
92:                         col_defs.append(f"`{c}` VARCHAR(255)")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
93:                     elif c == 'original_row':
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
94:                         col_defs.append(f"`{c}` INT")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
95:                     elif c == 'source_file':
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
96:                         col_defs.append(f"`{c}` VARCHAR(128)")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
97:                     else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
98:                         col_defs.append(f"`{c}` TEXT")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
99:
```

Blank line: code ko readable banane ke liye.

```
100:                 create_sql = f"CREATE TABLE `{table_name}` (\n  " + ",\n  ".join(col_defs) + "\n) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4;"
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
101:                 cur.execute(create_sql)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
102:                 conn.commit()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
103:
```

Blank line: code ko readable banane ke liye.

```
104:             # Insert in chunks
```

Comment: developer ka note; Python ise execute nahi karta.

```
105:             chunksize = 10000
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
106:             total_rows = 0
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
107:             row_offset = 1
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
108:
```
Blank line: code ko readable banane ke liye.
```
109:             for chunk in pd.read_csv(fname, sep=delimiter, chunksize=chunksize, dtype=str, on_bad_lines='skip'):
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
110:                 chunk_len = len(chunk)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
111:                 chunk['source_file'] = os.path.basename(fname)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
112:                 chunk['original_row'] = list(range(row_offset, row_offset + chunk_len))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
113:                 row_offset += chunk_len
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
114:
```
Blank line: code ko readable banane ke liye.
```
115:                     prod_series = chunk['Product'] if 'Product' in chunk.columns else (chunk['ProductName'] if
'ProductName' in chunk.columns else None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
116:                 if upc_col and upc_col in chunk.columns:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
117:                     if prod_series is not None:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
118:                         chunk['clean_upc'] = [normalize_upc(u, p) for u, p in zip(chunk[upc_col], prod_series)]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
119:                     else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
120:                         chunk['clean_upc'] = [normalize_upc(u) for u in chunk[upc_col]]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
121:                 else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
122:                     chunk['clean_upc'] = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
123:
```
Blank line: code ko readable banane ke liye.
```
124:                 if alt_upc_col and alt_upc_col in chunk.columns:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
125:                     chunk['clean_alt_upc'] = [normalize_upc(u) for u in chunk[alt_upc_col]]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:                 else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
127:                     chunk['clean_alt_upc'] = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
128:
```
Blank line: code ko readable banane ke liye.
```
129:                 if brand_col and brand_col in chunk.columns:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
130:                     chunk['norm_brand'] = chunk[brand_col].fillna("").astype(str).str.strip().str.upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
131:                 else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
132:                     chunk['norm_brand'] = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
133:
```
Blank line: code ko readable banane ke liye.
```
134:                 if supplier_col and supplier_col in chunk.columns:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
135:                     chunk['norm_supplier'] = chunk[supplier_col].fillna("").astype(str).str.strip().str.upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
136:                 else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
137:                     chunk['norm_supplier'] = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
138:
```
Blank line: code ko readable banane ke liye.
```
139:                 chunk.columns = [clean_column_name(c) for c in chunk.columns]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
140:
```
Blank line: code ko readable banane ke liye.
```
141:                 # Keep only columns present in table definition
```
Comment: developer ka note; Python ise execute nahi karta.
```
142:                 insert_df = chunk.reindex(columns=unique_cols).fillna("")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
143:
```
Blank line: code ko readable banane ke liye.
```
144:                 # Bulk insert
```

Comment: developer ka note; Python ise execute nahi karta.

```
145:                 placeholders = ", ".join(["%s"] * len(unique_cols))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
146:                 col_names_sql = ", ".join([f"`{c}`" for c in unique_cols])
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
147:                 insert_sql = f"INSERT INTO `{table_name}` ({col_names_sql}) VALUES ({placeholders})"
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
148:
```

Blank line: code ko readable banane ke liye.

```
149:                 tuples = [tuple(x) for x in insert_df.to_numpy()]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
150:                 with conn.cursor() as cur:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
151:                     cur.executemany(insert_sql, tuples)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
152:                 conn.commit()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
153:
```

Blank line: code ko readable banane ke liye.

```
154:                 total_rows += chunk_len
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
155:                 if progress_callback:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
156:                     progress_callback(key, total_rows)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
157:
```

Blank line: code ko readable banane ke liye.

```
158:             # Create Indexes
```

Comment: developer ka note; Python ise execute nahi karta.

```
159:             with conn.cursor() as cur:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
160:                 try:
```

Error handling: failure ko handle karke app ko crash hone se bachata hai.

```
161:                     cur.execute(f"CREATE INDEX `idx_{table_name}_upc` ON `{table_name}`(clean_upc(32))")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
162:                     if 'clean_alt_upc' in unique_cols:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
163:                         cur.execute(f"CREATE INDEX `idx_{table_name}_alt_upc` ON `{table_name}`(clean_alt_upc(32))")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
164:                     cur.execute(f"CREATE INDEX `idx_{table_name}_brand` ON `{table_name}`(norm_brand(64))")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
165:                     if 'norm_supplier' in unique_cols:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
166:                         cur.execute(f"CREATE INDEX `idx_{table_name}_supplier` ON `{table_name}`(norm_supplier(64))")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
167:                     for id_column, suffix in [('Id', 'id'), ('id', 'id_value'), ('SupplierId', 'supplier_id')]:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
168:                         if id_column in unique_cols:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
169:                             cur.execute(f"CREATE INDEX `idx_{table_name}_{suffix}` ON `{table_name}`(`{id_column}`(64))")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
170:                     conn.commit()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
171:                 except Exception as ex:
```

Error handling: failure ko handle karke app ko crash hone se bachata hai.

```
172:                     logger.warning(f"MySQL index creation notice: {ex}")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
173:
```

Blank line: code ko readable banane ke liye.

```
174:             results[key] = total_rows
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
175:
```

Blank line: code ko readable banane ke liye.

```
176:         conn.close()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
177:             return True, f"Successfully migrated {sum(results.values()):,} records into MySQL on port {port} (Database: '{db_name}')"
```

Return: calculated result caller ko wapas bhejta hai.

```
178:
```

Blank line: code ko readable banane ke liye.

```
179:     except Exception as e:
```

Error handling: failure ko handle karke app ko crash hone se bachata hai.

```
180:         conn.close()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
181:         return False, f"Migration error: {e}"
```

Return: calculated result caller ko wapas bhejta hai.

```
182:
```

Blank line: code ko readable banane ke liye.

```
183: if __name__ == "__main__":
```

Entry-point guard: file direct run hone par startup code chalata hai.

```
184:     import sys
```

Import: library/module ke functions yahan available karata hai.

```
185:     pwd = sys.argv[1] if len(sys.argv) > 1 else MYSQL_PASSWORD
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
186:     print(f"Connecting to MySQL on 127.0.0.1:{MYSQL_PORT} with user root...")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
187:     ok, msg = migrate_data_to_mysql(password=pwd)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
188:     print("Result:", ok, msg)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
189:
```

Blank line: code ko readable banane ke liye.

File: engines/sql_engine.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
 2: import logging
```
Import: library/module ke functions yahan available karata hai.
```
 3: from collections import Counter
```
Import: library/module ke functions yahan available karata hai.
```
 4: from config import CSV_FILES
```
Import: library/module ke functions yahan available karata hai.
```
 5: from database.db_manager import db_manager
```
Import: library/module ke functions yahan available karata hai.
```
 6: from data_ingestion.normalizer import normalize_upc
```
Import: library/module ke functions yahan available karata hai.
```
 7:
```
Blank line: code ko readable banane ke liye.
```
 8: logger = logging.getLogger("SQLEngine")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 9:
```
Blank line: code ko readable banane ke liye.
```
10: class SQLEngine:
```
Class: related data aur methods ka reusable blueprint.
```
11:     def __init__(self, db=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
12:         self.db = db or db_manager
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
13:
```
Blank line: code ko readable banane ke liye.
```
14:     def lookup_upc(self, upc_query):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
15:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
16:         Exact search for a UPC across Master and Categorization tables.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
17:         Returns matching records with source files and original rows.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:         clean = normalize_upc(upc_query)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:         results = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:
```
Blank line: code ko readable banane ke liye.
```
22:         # 1. Search in AIMaster
```
Comment: developer ka note; Python ise execute nahi karta.
```
23:         master_table = CSV_FILES['ai_master']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
24:         m_rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
25:                 SELECT 'AIMaster' as source, source_file, original_row, clean_upc, clean_alt_upc, BrandName,
ExtendedBrandName, SupplierName, ProductName, Size, Price, Segment
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
26:             FROM `{master_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
27:             WHERE clean_upc = ? OR clean_alt_upc = ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
28:         """, (clean, clean))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
29:         results.extend(m_rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:
```
Blank line: code ko readable banane ke liye.
```
31:         # 2. Search in AllBrand if not in Master
```
Comment: developer ka note; Python ise execute nahi karta.
```
32:         allbrand_table = CSV_FILES['all_brands']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
33:         ab_rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:                 SELECT 'AllBrand' as source, source_file, original_row, clean_upc, clean_alt_upc, BrandName,
ExtendedBrandName, SupplierName, ProductName, Size, Price, Remarks
```

SQL: database se data read/write ya schema/index tayyar hota hai.
```
35:             FROM `{allbrand_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:             WHERE clean_upc = ? OR clean_alt_upc = ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
37:         """, (clean, clean))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
38:         results.extend(ab_rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
39:
```
Blank line: code ko readable banane ke liye.
```
40:         # 3. Search in Beer
```
Comment: developer ka note; Python ise execute nahi karta.
```
41:         beer_table = CSV_FILES['cat_beer']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
42:         b_rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
43:                 SELECT 'Beer' as source, source_file, original_row, clean_upc, Beer_Brands_Value as BrandName,
Beer_Category_Value as Category, Beer_Vendors_Value as SupplierName, Product, Status
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
44:             FROM `{beer_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
45:             WHERE clean_upc = ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
46:         """, (clean,))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
47:         results.extend(b_rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
48:
```
Blank line: code ko readable banane ke liye.
```
49:         # 4. Search in RTDs
```
Comment: developer ka note; Python ise execute nahi karta.
```
50:         rtd_table = CSV_FILES['cat_rtd']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
51:         r_rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
52:                     SELECT 'RTD' as source, source_file, original_row, clean_upc, Brands_Value as BrandName,
RTD_Category_Value as Category, RTD_Alcohol_Base_Value as AlcoholBase, Product, PD_Status
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
53:             FROM `{rtd_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
54:             WHERE clean_upc = ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
55:         """, (clean,))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
56:         results.extend(r_rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
57:
```
Blank line: code ko readable banane ke liye.
```
58:         # 5. Search in Spirits
```
Comment: developer ka note; Python ise execute nahi karta.
```
59:         spirits_table = CSV_FILES['cat_spirits']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
60:         s_rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
61:                 SELECT 'Spirits' as source, source_file, original_row, clean_upc, Spirits_Brands_Value as BrandName,
Spirits_Category_Value as Category, Spirits_Suppliers_Value as SupplierName, Product, PD_Status
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
62:             FROM `{spirits_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
63:             WHERE clean_upc = ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
64:         """, (clean,))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
65:         results.extend(s_rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
66:
```
Blank line: code ko readable banane ke liye.
```
67:         # 6. Search in Wine
```
Comment: developer ka note; Python ise execute nahi karta.
```
68:         wine_table = CSV_FILES['cat_wine']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
69:         w_rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
   70:                     SELECT 'Wine' as source, source_file, original_row, clean_upc, Brands_Value as BrandName,
Categories_Value as Category, Wine_Suppliers_Value as SupplierName, Product, PD_Status
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
  71:             FROM `{wine_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  72:             WHERE clean_upc = ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  73:         """, (clean,))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  74:         results.extend(w_rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  75:
```
Blank line: code ko readable banane ke liye.
```
  76:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
  77:             "query_upc": upc_query,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  78:             "normalized_upc": clean,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  79:             "total_matches": len(results),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  80:             "found_in_files": list(set([r['source_file'] for r in results])),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  81:             "records": results
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  82:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  83:
```
Blank line: code ko readable banane ke liye.
```
  84:     def search_brand(self, brand_query, limit=50):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
  85:         """Search products by brand name across master and categorization datasets."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  86:         term = f"%{brand_query.strip().upper()}%"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  87:         master_table = CSV_FILES['ai_master']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  88:         rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  89:              SELECT source_file, original_row, clean_upc, BrandName, ExtendedBrandName, SupplierName, ProductName,
Size, Price, Segment
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
  90:             FROM `{master_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  91:             WHERE norm_brand LIKE ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  92:             LIMIT {limit}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  93:         """, (term,))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  94:
```
Blank line: code ko readable banane ke liye.
```
  95:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
  96:             "query_brand": brand_query,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  97:             "total_matches": len(rows),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  98:             "records": rows
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  99:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 100:
```
Blank line: code ko readable banane ke liye.
```
 101:     def search_master_brands(self, query="", master_id=None, brand=None, upc=None, product=None, limit=100):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
 102:         """Search the Master Brand table using real columns and exact identifiers."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 103:         table = CSV_FILES["ai_master"]["table"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 104:         columns = [column for column in self.db.table_columns(table) if column]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 105:         column_lookup = {column.lower(): column for column in columns}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
106:         id_column = next((column_lookup[name] for name in (
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
107:             "id", "record_id", "masterbrandid", "master_brand_id", "master_id"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
108:         ) if name in column_lookup), None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
109:         brand_column = column_lookup.get("norm_brand") or column_lookup.get("brandname")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
110:         product_column = column_lookup.get("productname") or column_lookup.get("product")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
111:          clean_upc_column = next((column_lookup[name] for name in ("clean_upc", "upccode", "upc_13_digit", "upc 13
digit", "upc13digit") if name in column_lookup), None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
112:          clean_alt_column = next((column_lookup[name] for name in ("clean_alt_upc", "alternateupccode", "alternate
upc code", "alternateupccode") if name in column_lookup), None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
113:
```
Blank line: code ko readable banane ke liye.
```
114:         master_id = master_id or query if query and not (brand or upc or product) else master_id
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
115:         conditions = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
116:         params = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
117:         if master_id and id_column:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
118:             conditions.append(f"`{id_column}` = ?")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
119:             params.append(str(master_id).strip())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
120:         if upc and clean_upc_column:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
121:             normalized = normalize_upc(upc)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
122:             upc_conditions = [f"`{clean_upc_column}` = ?"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
123:             upc_params = [normalized]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
124:             if clean_alt_column:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
125:                 upc_conditions.append(f"`{clean_alt_column}` = ?")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:                 upc_params.append(normalized)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
127:             conditions.append("(" + " OR ".join(upc_conditions) + ")")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
128:             params.extend(upc_params)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
129:         if brand and brand_column:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
130:             conditions.append(f"LOWER(COALESCE(`{brand_column}`, '')) LIKE LOWER(?)")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
131:             params.append(f"%{brand.strip()}%")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
132:         if product and product_column:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
133:             conditions.append(f"LOWER(COALESCE(`{product_column}`, '')) LIKE LOWER(?)")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
134:             params.append(f"%{product.strip()}%")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
135:         if not conditions:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
136:             return {"columns": columns, "records": [], "total_matches": 0, "id_column": id_column}
```
Return: calculated result caller ko wapas bhejta hai.
```
137:
```
Blank line: code ko readable banane ke liye.
```
138:         safe_limit = max(1, min(int(limit), 500))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
139:         where_sql = f"FROM `{table}` WHERE {' AND '.join(conditions)}"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
140:         count_rows = self.db.query(f"SELECT COUNT(*) AS total_matches {where_sql}", tuple(params))
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
141:         rows = self.db.query(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
142:             f"SELECT * FROM `{table}` WHERE {' AND '.join(conditions)} LIMIT {safe_limit}",
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
143:             tuple(params),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
144:         )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
145:         result_columns = list(rows[0].keys()) if rows else columns
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
146:             total_matches = int((count_rows[0].get("total_matches") or 0)) if count_rows and "total_matches" in count_rows[0] else len(rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
147:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
148:             "columns": result_columns,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
149:             "records": rows,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
150:             "total_matches": total_matches,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
151:             "id_column": id_column,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
152:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
153:
```
Blank line: code ko readable banane ke liye.
```
154:     def lookup_identifier(self, identifier, limit=200):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
155:         """Return complete source rows for an exact ID, UPC, or alternate UPC."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
156:         requested = str(identifier or "").strip()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
157:         if not requested:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
158:             return {"normalized_identifier": "", "total_matches": 0,
```
Return: calculated result caller ko wapas bhejta hai.
```
159:                     "records": [], "related_records": [], "source_files": []}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
160:
```
Blank line: code ko readable banane ke liye.
```
161:         normalized = normalize_upc(requested) if requested.isdigit() and 10 <= len(requested) <= 14 else requested
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
162:         datasets = [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
163:             ("all_brands", "Id"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
164:             ("suppliers", "SupplierId"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
165:             ("ai_master", "id"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
166:             ("cat_beer", None),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
167:             ("cat_rtd", None),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
168:             ("cat_spirits", None),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
169:             ("cat_wine", None),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
170:         ]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
171:         records = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
172:         seen = set()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
173:         safe_limit = max(1, min(int(limit), 1000))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
174:         for table, id_column in datasets:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
175:             id_clause = f"CAST(`{id_column}` AS CHAR) = ? OR " if id_column else ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
176:             params = (requested, normalized, normalized) if id_column else (normalized, normalized)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
177:             rows = self.db.query(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
178:                         f"SELECT * FROM `{table}` WHERE ({id_clause}clean_upc = ? OR clean_alt_upc = ?) LIMIT {safe_limit}",
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
                params,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            )
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            for row in rows:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                key = (table, row.get("original_row"), row.get("clean_upc"), row.get("clean_alt_upc"))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                if key not in seen:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                    seen.add(key)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                    row.setdefault("_dataset", table)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                    records.append(row)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```

```

Blank line: code ko readable banane ke liye.

```
        related_upcs = {r.get("clean_upc") for r in records if r.get("clean_upc")}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
        related = []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
        for table, _ in datasets:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            for upc in related_upcs:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                rows = self.db.query(
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                    f"SELECT * FROM `{table}` WHERE clean_upc = ? OR clean_alt_upc = ? LIMIT {safe_limit}",
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
                    (upc, upc),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                )
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                for row in rows:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                    key = (table, row.get("original_row"), row.get("clean_upc"), row.get("clean_alt_upc"))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                    if key not in seen:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                        seen.add(key)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                        row.setdefault("_dataset", table)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                        related.append(row)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```

```

Blank line: code ko readable banane ke liye.

```
        source_files = sorted({r.get("source_file") for r in records + related if r.get("source_file")})
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
        return {
```

Return: calculated result caller ko wapas bhejta hai.

```
            "normalized_identifier": normalized,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            "total_matches": len(records),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            "records": records,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            "related_records": related,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            "source_files": source_files,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
        }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```

```

Blank line: code ko readable banane ke liye.

```
    def search_catalog(self, query, limit=100, offset=0):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
        """Search brand names and product descriptions across every beverage sheet."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
        term = f"%{query.strip()}%"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
        safe_limit = max(1, min(int(limit), 1000))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
216:         safe_offset = max(0, int(offset))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
217:         datasets = [
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
218:                 ("Master", CSV_FILES['ai_master']['table'], "norm_brand, BrandName, ProductName, Flavor, Flavor_1,
Size, SupplierName, Appellation, clean_upc, source_file, original_row", "ProductName"),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
219:                             ("Beer",  CSV_FILES['cat_beer']['table'],  "norm_brand,  Beer_Brands_Value,  Product,
Beer_Mezzo_Styles_Value,  Beer_Size_Groups_Value,  Beer_Vendors_Value,  Beer_DomVsImp_Value,  clean_upc,  source_file,
original_row
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
220:                 ("Wine", CSV_FILES['cat_wine']['table'], "norm_brand, Brands_Value, Product, Flavor_Groups_Value,
Flavors_Value, Size_Packages_Value, Wine_Suppliers_Value, DomVsImp_Value, clean_upc, source_file, original_row
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
221:              ("RTD", CSV_FILES['cat_rtd']['table'], "norm_brand, Brands_Value, Product, RTD_Size_Value, clean_upc,
source_file, original_row", "Product"),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
222:                       ("Spirits",  CSV_FILES['cat_spirits']['table'],  "norm_brand,  Spirits_Brands_Value,  Product,
Spirits_Flavors_Value, Spirits_Flavor_Scents_Value, Spirits_Sizes_Value, Spirits_Suppliers_Value, Spirits_DomVsImp_Va
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
223:         ]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
224:         results = []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
225:         for category, table, columns, product_column in datasets:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
226:             rows = self.db.query(
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
227:                 f"SELECT {columns} FROM `{table}` WHERE norm_brand LIKE ? OR LOWER(COALESCE({product_column}, ''))
LIKE LOWER(?) LIMIT {safe_limit} OFFSET {safe_offset}",
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
228:                 (term, term)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
229:             )
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
230:             for row in rows:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
231:                 if category == "Master":
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
232:                                 brand,  product,  flavour,  size  =  row.get("BrandName"),  row.get("ProductName"),  " /
".join(str(row.get(k) or "") for k in ("Flavor", "Flavor_1") if str(row.get(k) or "").strip()), row.get("Size")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
233:                     supplier, origin = row.get("SupplierName"), row.get("Appellation")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
234:                 elif category == "Beer":
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
235:                                 brand,  product,  flavour,  size  =  row.get("Beer_Brands_Value"),  row.get("Product"),
row.get("Beer_Mezzo_Styles_Value"), row.get("Beer_Size_Groups_Value")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
236:                     supplier, origin = row.get("Beer_Vendors_Value"), row.get("Beer_DomVsImp_Value")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
237:                 elif category == "Wine":
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
238:                                 brand,  product,  flavour,  size  =  row.get("Brands_Value"),  row.get("Product"),  " /
".join(str(row.get(k) or "") for k in ("Flavor_Groups_Value", "Flavors_Value") if str(row.get(k) or "").strip()), row.g
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
239:                     supplier, origin = row.get("Wine_Suppliers_Value"), row.get("DomVsImp_Value")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
240:                 elif category == "RTD":
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
241:                                 brand,  product,  flavour,  size  =  row.get("Brands_Value"),  row.get("Product"),  "",
row.get("RTD_Size_Value")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
242:                     supplier, origin = "", ""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
243:                 else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
244:                         brand, product, flavour, size = row.get("Spirits_Brands_Value"), row.get("Product"), " /
".join(str(row.get(k) or "") for k in ("Spirits_Flavors_Value", "Spirits_Flavor_Scents_Value") if str(row.get(k
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
245:                     supplier, origin = row.get("Spirits_Suppliers_Value"), row.get("Spirits_DomVsImp_Value")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
246:                 results.append({
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
247:                     "category": category, "brand": brand or "", "product_description": product or "",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
248:                     "flavour": flavour or "", "size": size or "", "supplier": supplier or "",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
249:                     "origin": origin or "", "upc": row.get("clean_upc") or "",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
250:                     "source_file": row.get("source_file"), "original_row": row.get("original_row")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
251:                 })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
252:                 if len(results) >= safe_limit:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
253:                     return {"query": query, "total_matches": len(results), "records": results}
```
Return: calculated result caller ko wapas bhejta hai.
```
254:         return {"query": query, "total_matches": len(results), "records": results}
```
Return: calculated result caller ko wapas bhejta hai.
```
255:
```
Blank line: code ko readable banane ke liye.
```
256:     def brand_profile(self, brand_query, limit=500):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
257:         """Return brand facts across the master and all categorization sheets."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
258:         term = f"%{brand_query.strip().upper()}%"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
259:         datasets = [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
260:             ("Master", CSV_FILES['ai_master']['table'],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
261:                     "norm_brand, BrandName, ProductCategory, Flavor, Flavor_1, Size, SupplierName, Appellation,
ProductName, clean_upc, source_file, original_row"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
262:             ("Beer", CSV_FILES['cat_beer']['table'],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
263:               "norm_brand, Beer_Brands_Value, Beer_Mezzo_Styles_Value, Beer_Size_Groups_Value, Beer_Vendors_Value,
Beer_DomVsImp_Value, Product, clean_upc, source_file, original_row"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
264:             ("Wine", CSV_FILES['cat_wine']['table'],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
265:                           "norm_brand, Brands_Value, Flavor_Groups_Value, Flavors_Value, Size_Packages_Value,
Wine_Suppliers_Value, DomVsImp_Value, Product, clean_upc, source_file, original_row"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
266:             ("RTD", CSV_FILES['cat_rtd']['table'],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
267:              "norm_brand, Brands_Value, RTD_Size_Value, Product, clean_upc, source_file, original_row"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
268:             ("Spirits", CSV_FILES['cat_spirits']['table'],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
269:                         "norm_brand, Spirits_Brands_Value, Spirits_Flavors_Value, Spirits_Flavor_Scents_Value,
Spirits_Sizes_Value, Spirits_Suppliers_Value, Spirits_DomVsImp_Value, Product, clean_upc, source_file, original_row"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
270:         ]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
271:
```
Blank line: code ko readable banane ke liye.
```
272:         records = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
273:         for category, table, columns in datasets:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
274:             rows = self.db.query(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
275:                 f"SELECT {columns} FROM `{table}` WHERE norm_brand LIKE ? LIMIT {int(limit)}",
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
276:                 (term,)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
277:             )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
278:             for row in rows:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
279:                 if category == "Master":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
280:                     flavour = " / ".join(str(row.get(k) or "") for k in ("Flavor", "Flavor_1") if str(row.get(k)
or "").strip())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
281:                     brand = row.get("BrandName")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
282:                     size = row.get("Size")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
283:                     supplier = row.get("SupplierName")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
284:                     origin = row.get("Appellation")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
285:                     product = row.get("ProductName")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
286:                 elif category == "Beer":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
287:                     flavour = row.get("Beer_Mezzo_Styles_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
288:                     brand = row.get("Beer_Brands_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
289:                     size = row.get("Beer_Size_Groups_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
290:                     supplier = row.get("Beer_Vendors_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
291:                     origin = row.get("Beer_DomVsImp_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
292:                     product = row.get("Product")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
293:                 elif category == "Wine":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
294:                     flavour = " / ".join(str(row.get(k) or "") for k in ("Flavor_Groups_Value", "Flavors_Value")
if str(row.get(k) or "").strip())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
295:                     brand = row.get("Brands_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
296:                     size = row.get("Size_Packages_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
297:                     supplier = row.get("Wine_Suppliers_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
298:                     origin = row.get("DomVsImp_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
299:                     product = row.get("Product")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
300:                 elif category == "RTD":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
301:                     flavour = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
302:                     brand = row.get("Brands_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
303:                     size = row.get("RTD_Size_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
304:                     supplier = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
305:                     origin = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
306:                     product = row.get("Product")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
307:                 else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
308:                         flavour = " / ".join(str(row.get(k) or "") for k in ("Spirits_Flavors_Value",
"Spirits_Flavor_Scents_Value") if str(row.get(k) or "").strip())
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
309:                     brand = row.get("Spirits_Brands_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
310:                     size = row.get("Spirits_Sizes_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
311:                     supplier = row.get("Spirits_Suppliers_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
312:                     origin = row.get("Spirits_DomVsImp_Value")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
313:                     product = row.get("Product")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
314:                 records.append({
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
315:                     "category": category, "brand": brand or brand_query, "flavour": flavour,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
316:                     "size": size, "supplier": supplier, "origin": origin,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
317:                     "product": product, "upc": row.get("clean_upc"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
318:                     "source_file": row.get("source_file"), "original_row": row.get("original_row")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
319:                 })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
320:
```
Blank line: code ko readable banane ke liye.
```
321:         def values(key):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
322:             return sorted({str(r[key]).strip() for r in records if str(r.get(key) or "").strip()})
```
Return: calculated result caller ko wapas bhejta hai.
```
323:
```
Blank line: code ko readable banane ke liye.
```
324:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
325:             "query_brand": brand_query,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
326:             "total_records": len(records),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
327:                     "unique_brand_count":  len({str(r.get("brand") or "").strip().upper() for r in records if str(r.get("brand") or "").strip()}),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
328:             "unique_flavour_count": len(values("flavour")),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
329:             "unique_size_count": len(values("size")),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
330:                 "unique_upc_count": len({str(r.get("upc") or "").strip() for r in records if str(r.get("upc") or "").strip()}),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
331:               "categories": [{"category": name, "count": count} for name, count in Counter(r["category"] for r in records).most_common()],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
332:             "flavours": values("flavour"), "sizes": values("size"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
333:             "suppliers": values("supplier"), "origins": values("origin"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
334:             "records": records,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
335:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
336:
```
Blank line: code ko readable banane ke liye.
```
337:     def search_supplier(self, supplier_query, limit=50):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
338:         """Search products and supplier details by supplier name."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
339:         term = f"%{supplier_query.strip().upper()}%"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
340:         master_table = CSV_FILES['ai_master']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
341:         rows = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
342:             SELECT source_file, original_row, clean_upc, BrandName, ExtendedBrandName, SupplierName, SupplierCode, ProductName, Price
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
343:             FROM `{master_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
344:             WHERE norm_supplier LIKE ?
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
345:             LIMIT {limit}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
346:         """, (term,))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
347:
```
Blank line: code ko readable banane ke liye.
```
348:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
349:             "query_supplier": supplier_query,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
350:             "total_matches": len(rows),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
351:             "records": rows
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
352:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
353:
```
Blank line: code ko readable banane ke liye.
```
354:     def count_master_brands(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
355:         """Return the distinct normalized brand count from the master catalog."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
356:         return self.count_brands()
```
Return: calculated result caller ko wapas bhejta hai.
```
357:
```
Blank line: code ko readable banane ke liye.
```
358:     def count_brands(self, category=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
359:         """Return distinct brand count for the master or requested category sheet."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
360:         table = CSV_FILES.get(category or 'ai_master', CSV_FILES['ai_master'])['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
361:         rows = self.db.query(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
362:             f"SELECT COUNT(DISTINCT norm_brand) AS brand_count FROM `{table}` "
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
363:             "WHERE TRIM(COALESCE(norm_brand, '')) != ''"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
364:         )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
365:         return int(rows[0].get('brand_count') or 0) if rows else 0
```
Return: calculated result caller ko wapas bhejta hai.
```
366:
```
Blank line: code ko readable banane ke liye.
```
367:     def count_by_category(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
368:         """Aggregates product counts by category across Master and Categorization tables."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
369:         master_table = CSV_FILES['ai_master']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
370:         m_counts = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
371:             SELECT ProductName as category, COUNT(*) as product_count
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
372:             FROM `{master_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
373:             GROUP BY ProductName
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
374:             ORDER BY product_count DESC
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
375:         """)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
376:
```
Blank line: code ko readable banane ke liye.
```
377:         beer_table = CSV_FILES['cat_beer']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
378:         b_counts = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
379:             SELECT Beer_Category_Value as category, COUNT(*) as product_count
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
380:             FROM `{beer_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
381:             GROUP BY Beer_Category_Value
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
382:             ORDER BY product_count DESC
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
383:         """)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
384:
```
Blank line: code ko readable banane ke liye.
```
385:         rtd_table = CSV_FILES['cat_rtd']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
386:         r_counts = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
387:             SELECT RTD_Category_Value as category, COUNT(*) as product_count
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
388:             FROM `{rtd_table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
            GROUP BY RTD_Category_Value
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            ORDER BY product_count DESC
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        """)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
        return {
```
Return: calculated result caller ko wapas bhejta hai.
```
            "master_categories": m_counts,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            "beer_categories": b_counts,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            "rtd_categories": r_counts
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
    def get_dataset_overview(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
        """Returns row counts and metrics for each active table."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        stats = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        for key, conf in CSV_FILES.items():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            tbl = conf['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
                row_count = self.db.query(f"SELECT COUNT(*) as c FROM `{tbl}`")[0]['c']
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
                upc_count = self.db.query(f"SELECT COUNT(DISTINCT clean_upc) as c FROM `{tbl}` WHERE clean_upc != ''")[0]['c']
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
            except Exception:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
                row_count = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                upc_count = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            stats.append({
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "dataset_key": key,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "file_name": conf['file'],
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "table_name": tbl,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "total_rows": row_count,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "unique_upcs": upc_count
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        return stats
```
Return: calculated result caller ko wapas bhejta hai.
```

```
Blank line: code ko readable banane ke liye.
```
    def get_dashboard_metrics(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
        """Return live category, UPC-quality, and Master Brand dashboard metrics."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        category_keys = ("beer", "wine", "rtd", "spirits")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        categories = {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            key: self._table_upc_metrics(CSV_FILES[f"cat_{key}"]["table"])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            for key in category_keys
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
426:                 master_columns = self.db.table_columns(CSV_FILES['ai_master']['table']) if hasattr(self.db, 'table_columns') else []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
427:         master_column_lookup = {column.lower(): column for column in master_columns if column}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
428:         master_id_column = (
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
429:             master_column_lookup.get('id')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
430:             or master_column_lookup.get('record_id')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
431:             or master_column_lookup.get('masterbrandid')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
432:             or master_column_lookup.get('master_brand_id')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
433:         )
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
434:          master_brand_column = master_column_lookup.get('norm_brand') or master_column_lookup.get('brandname') or master_column_lookup.get('extendedbrandname')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
435:             master_upc_column = next((master_column_lookup.get(name) for name in ('clean_upc', 'upccode', 'upc_13_digit', 'upc 13 digit', 'upc13digit') if master_column_lookup.get(name)), None)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
436:         id_expression = f"`{master_id_column}`" if master_id_column else "NULL"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
437:         brand_expression = f"`{master_brand_column}`" if master_brand_column else "NULL"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
438:         upc_expression = f"`{master_upc_column}`" if master_upc_column else "NULL"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
439:         master = self.db.query(f"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
440:             SELECT
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
441:                 COUNT(*) AS master_record_count,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
442:                 COUNT(DISTINCT NULLIF(TRIM(COALESCE({brand_expression}, '')), '')) AS unique_brands,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
443:                 COUNT(DISTINCT NULLIF(TRIM(COALESCE({id_expression}, '')), '')) AS unique_master_ids,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
444:                     SUM(CASE WHEN LOWER(TRIM(COALESCE({upc_expression}, ''))) NOT IN ('', 'null', 'none', 'nan', 'n/a', 'na', '0') THEN 1 ELSE 0 END) AS available_upcs,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
445:                   SUM(CASE WHEN LOWER(TRIM(COALESCE({upc_expression}, ''))) IN ('', 'null', 'none', 'nan', 'n/a', 'na', '0') THEN 1 ELSE 0 END) AS missing_upcs,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
446:                   COUNT(DISTINCT CASE WHEN LOWER(TRIM(COALESCE({upc_expression}, ''))) NOT IN ('', 'null', 'none', 'nan', 'n/a', 'na', '0') THEN TRIM({upc_expression}) END) AS distinct_upcs
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
447:             FROM `{CSV_FILES['ai_master']['table']}`
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
448:         """)[0]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
449:
```

Blank line: code ko readable banane ke liye.

```
450:         overall_total = sum(item["total_records"] for item in categories.values())
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
451:         overall_available = sum(item["available_upcs"] for item in categories.values())
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
452:         overall_missing = sum(item["missing_upcs"] for item in categories.values())
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
453:         return {
```

Return: calculated result caller ko wapas bhejta hai.

```
454:             "categories": categories,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
455:             "overall_upc": self._quality_summary(overall_total, overall_available, overall_missing),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
456:             "master_brand": {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
457:                 "total_records": int(master.get("master_record_count") or 0),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
458:                 "unique_brands": int(master.get("unique_brands") or 0),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
459:                 "unique_master_ids": int(master.get("unique_master_ids") or 0),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
460:                 "records_with_upc": int(master.get("available_upcs") or 0),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
461:                 "records_without_upc": int(master.get("missing_upcs") or 0),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
462:                 "distinct_upcs": int(master.get("distinct_upcs") or 0),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
463:             },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
464:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
465:
```
Blank line: code ko readable banane ke liye.
```
466:     @staticmethod
```
Decorator: neeche wale function ko Flask route ya extra behavior deta hai.
```
467:     def _quality_summary(total_records, available_upcs, missing_upcs, invalid_upcs=0, distinct_upcs=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
468:         total_records = int(total_records or 0)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
469:         available_upcs = int(available_upcs or 0)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
470:         missing_upcs = int(missing_upcs or 0)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
471:         invalid_upcs = int(invalid_upcs or 0)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
472:         distinct_upcs = int(available_upcs if distinct_upcs is None else distinct_upcs or 0)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
473:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
474:             "total_records": total_records,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
475:             "available_upcs": available_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
476:             "missing_upcs": missing_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
477:             "invalid_upcs": invalid_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
478:             "distinct_upcs": distinct_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
479:             "completion_percentage": round((available_upcs / total_records) * 100, 2) if total_records else 0.0,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
480:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
481:
```
Blank line: code ko readable banane ke liye.
```
482:     def _table_upc_metrics(self, table):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
483:             columns = {column.lower(): column for column in (self.db.table_columns(table) if hasattr(self.db,
'table_columns') else [])}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
484:         upc_column = next((columns.get(name) for name in ('clean_upc', 'upccode', 'upc_13_digit', 'upc 13 digit',
'upc13digit') if columns.get(name)), None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
485:         upc_expression = f"`{upc_column}`" if upc_column else "NULL"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
486:         row = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
487:             SELECT
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
488:                 COUNT(*) AS total_records,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
489:                     SUM(CASE WHEN LOWER(TRIM(COALESCE({upc_expression}, ''))) NOT IN ('', 'null', 'none', 'nan',
'n/a', 'na', '0') THEN 1 ELSE 0 END) AS available_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
490:                  SUM(CASE WHEN LOWER(TRIM(COALESCE({upc_expression}, ''))) IN ('', 'null', 'none', 'nan', 'n/a',
'na', '0') THEN 1 ELSE 0 END) AS missing_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
491:                  COUNT(DISTINCT CASE WHEN LOWER(TRIM(COALESCE({upc_expression}, ''))) NOT IN ('', 'null', 'none',
'nan', 'n/a', 'na', '0') THEN TRIM({upc_expression}) END) AS distinct_upcs
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
492:             FROM `{table}`
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
493:         """)[0]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
494:         return self._quality_summary(
```
Return: calculated result caller ko wapas bhejta hai.
```
495:             row.get("total_records"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
496:             row.get("available_upcs"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
497:             row.get("missing_upcs"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
498:             distinct_upcs=row.get("distinct_upcs"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
499:         )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
500:
```
Blank line: code ko readable banane ke liye.
```
501: sql_engine = SQLEngine()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

File: engines/vector_engine.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import os
```
Import: library/module ke functions yahan available karata hai.
```
 2: import re
```
Import: library/module ke functions yahan available karata hai.
```
 3: import math
```
Import: library/module ke functions yahan available karata hai.
```
 4: import pickle
```
Import: library/module ke functions yahan available karata hai.
```
 5: from collections import Counter
```
Import: library/module ke functions yahan available karata hai.
```
 6: from config import INDEX_DIR, CSV_FILES
```
Import: library/module ke functions yahan available karata hai.
```
 7: from database.db_manager import db_manager
```
Import: library/module ke functions yahan available karata hai.
```
 8:
```
Blank line: code ko readable banane ke liye.
```
 9: class LocalVectorIndex:
```
Class: related data aur methods ka reusable blueprint.
```
10:     """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
11:     Offline local persistent BM25 / TF-IDF Vector Index for semantic retrieval.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
12:     Zero external API requirements, zero internet, fast sub-second querying.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
13:     """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
14:     def __init__(self, index_name="beverage_vector_index"):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
15:         self.index_name = index_name
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
16:         self.index_file = INDEX_DIR / f"{index_name}.pkl"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
17:         self.documents = []  # List of metadata dicts
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18:         self.doc_tokens = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:         self.doc_lens = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:         self.avg_doc_len = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:         self.df = Counter()  # Document frequency
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
22:         self.k1 = 1.5
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
23:         self.b = 0.75
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
24:
```
Blank line: code ko readable banane ke liye.
```
25:     def tokenize(self, text):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
26:         if not text:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
27:             return []
```
Return: calculated result caller ko wapas bhejta hai.
```
28:         text = str(text).lower()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
29:         return re.findall(r'[a-z0-9]+', text)
```
Return: calculated result caller ko wapas bhejta hai.
```
30:
```
Blank line: code ko readable banane ke liye.
```
31:     def build_from_database(self, limit_per_table=10000):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
32:         """Build vector index from SQLite database records."""
```
SQLite: local offline file database; fast indexed queries ke liye.
```
33:         self.documents = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:         self.doc_tokens = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
35:         self.doc_lens = []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
36:         self.df = Counter()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
37:
```

Blank line: code ko readable banane ke liye.

```
38:         # Ingest representative sample across all tables for semantic coverage
```

Comment: developer ka note; Python ise execute nahi karta.

```
39:         tables = [
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
40:                 (CSV_FILES['ai_master']['table'], "BrandName, ExtendedBrandName, SupplierName, ProductName, Size, clean_upc, source_file, original_row"),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
41:             (CSV_FILES['cat_beer']['table'], "Beer_Brands_Value as BrandName, Beer_Category_Value as ProductName, Beer_Vendors_Value as SupplierName, Product as Size, clean_upc, source_file, original_row"),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
42:                 (CSV_FILES['cat_spirits']['table'], "Spirits_Brands_Value as BrandName, Spirits_Category_Value as ProductName, Spirits_Suppliers_Value as SupplierName, Product as Size, clean_upc, source_file, original_row"),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
43:                   (CSV_FILES['cat_wine']['table'], "Brands_Value as BrandName, Categories_Value as ProductName, Wine_Suppliers_Value as SupplierName, Product as Size, clean_upc, source_file, original_row"),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
44:         ]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
45:
```

Blank line: code ko readable banane ke liye.

```
46:         total_docs = 0
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
47:         for tbl, cols in tables:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
48:             try:
```

Error handling: failure ko handle karke app ko crash hone se bachata hai.

```
49:                         rows = db_manager.query(f"SELECT {cols} FROM `{tbl}` WHERE clean_upc != '' LIMIT {limit_per_table}")
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
50:                 for r in rows:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
51:                     brand = r.get('BrandName') or ''
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
52:                     supp = r.get('SupplierName') or ''
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
53:                     cat = r.get('ProductName') or ''
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
54:                     desc = r.get('Size') or ''
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
55:                     upc = r.get('clean_upc') or ''
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
56:                     doc_text = f"{brand} {supp} {cat} {desc} {upc}"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
57:
```

Blank line: code ko readable banane ke liye.

```
58:                     tokens = self.tokenize(doc_text)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
59:                     if not tokens:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
60:                         continue
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
61:
```

Blank line: code ko readable banane ke liye.

```
62:                     doc_dict = {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
63:                         "id": total_docs,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
64:                         "upc": upc,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
65:                         "brand": brand,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
66:                         "supplier": supp,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
67:                         "category": cat,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
68:                         "description": desc,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
69:                         "source_file": r.get('source_file'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
70:                         "original_row": r.get('original_row'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
71:                         "text": doc_text
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
72:                     }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
73:                     self.documents.append(doc_dict)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
74:                     self.doc_tokens.append(tokens)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
75:                     self.doc_lens.append(len(tokens))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
76:
```
Blank line: code ko readable banane ke liye.
```
77:                     # update df
```
Comment: developer ka note; Python ise execute nahi karta.
```
78:                     unique_tokens = set(tokens)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
79:                     for t in unique_tokens:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
80:                         self.df[t] += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
81:
```
Blank line: code ko readable banane ke liye.
```
82:                     total_docs += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
83:             except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
84:                 print(f"Index build error for {tbl}: {e}")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
85:
```
Blank line: code ko readable banane ke liye.
```
86:         if self.doc_lens:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
87:             self.avg_doc_len = sum(self.doc_lens) / len(self.doc_lens)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
88:
```
Blank line: code ko readable banane ke liye.
```
89:         self.save()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
90:         return len(self.documents)
```
Return: calculated result caller ko wapas bhejta hai.
```
91:
```
Blank line: code ko readable banane ke liye.
```
92:     def save(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
93:         with open(self.index_file, 'wb') as f:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
94:             pickle.dump({
```
Pickle: vector index object ko disk par save/load karta hai.
```
95:                 'documents': self.documents,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
96:                 'doc_tokens': self.doc_tokens,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
97:                 'doc_lens': self.doc_lens,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
98:                 'avg_doc_len': self.avg_doc_len,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
99:                 'df': self.df
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
100:             }, f)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
101:
```
Blank line: code ko readable banane ke liye.
```
102:     def load(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
103:         if not os.path.exists(self.index_file):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
104:             return False
```
Return: calculated result caller ko wapas bhejta hai.
```
105:         with open(self.index_file, 'rb') as f:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
106:             data = pickle.load(f)
```
Pickle: vector index object ko disk par save/load karta hai.

```
107:             self.documents = data['documents']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
108:             self.doc_tokens = data['doc_tokens']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
109:             self.doc_lens = data['doc_lens']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
110:             self.avg_doc_len = data['avg_doc_len']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
111:             self.df = data['df']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
112:         return True
```
Return: calculated result caller ko wapas bhejta hai.
```
113:
```
Blank line: code ko readable banane ke liye.
```
114:     def search(self, query, top_k=10, category_filter=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
115:         """Retrieve most semantically relevant documents using BM25 scoring."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
116:         if not self.documents:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
117:             if not self.load():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
118:                 return []
```
Return: calculated result caller ko wapas bhejta hai.
```
119:
```
Blank line: code ko readable banane ke liye.
```
120:         q_tokens = self.tokenize(query)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
121:         if not q_tokens:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
122:             return []
```
Return: calculated result caller ko wapas bhejta hai.
```
123:
```
Blank line: code ko readable banane ke liye.
```
124:         N = len(self.documents)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
125:         scores = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:
```
Blank line: code ko readable banane ke liye.
```
127:         for idx, tokens in enumerate(self.doc_tokens):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
128:             doc = self.documents[idx]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
129:             if category_filter and category_filter.lower() not in doc.get('category', '').lower():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
130:                 continue
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
131:
```
Blank line: code ko readable banane ke liye.
```
132:             doc_len = self.doc_lens[idx]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
133:             token_counts = Counter(tokens)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
134:             score = 0.0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
135:
```
Blank line: code ko readable banane ke liye.
```
136:             for q in q_tokens:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
137:                 if q in token_counts:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
138:                     tf = token_counts[q]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
139:                     df = self.df.get(q, 0)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
140:                     idf = math.log((N - df + 0.5) / (df + 0.5) + 1.0)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
141:                     numerator = tf * (self.k1 + 1)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
142:                     denominator = tf + self.k1 * (1 - self.b + self.b * (doc_len / self.avg_doc_len))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
143:                     score += idf * (numerator / denominator)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
144:
```
Blank line: code ko readable banane ke liye.
```
145:             if score > 0:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
146:                 scores.append((score, doc))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
147:
```
Blank line: code ko readable banane ke liye.
```
148:         scores.sort(key=lambda x: x[0], reverse=True)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
149:         return [doc for _, doc in scores[:top_k]]
```
Return: calculated result caller ko wapas bhejta hai.
```
150:
```
Blank line: code ko readable banane ke liye.
```
151: vector_engine = LocalVectorIndex()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

File: engines/rag_orchestrator.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import re
```
Import: library/module ke functions yahan available karata hai.
```
 2: import json
```
Import: library/module ke functions yahan available karata hai.
```
 3: import uuid
```
Import: library/module ke functions yahan available karata hai.
```
 4: import time
```
Import: library/module ke functions yahan available karata hai.
```
 5: import requests
```
Import: library/module ke functions yahan available karata hai.
```
 6: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
 7: from config import DB_DIR, OLLAMA_HOST, OLLAMA_MODEL, REPORTS_DIR
```
Import: library/module ke functions yahan available karata hai.
```
 8: from engines.sql_engine import sql_engine
```
Import: library/module ke functions yahan available karata hai.
```
 9: from engines.comparison_engine import comparison_engine
```
Import: library/module ke functions yahan available karata hai.
```
10: from engines.vector_engine import vector_engine
```
Import: library/module ke functions yahan available karata hai.
```
11: from data_ingestion.normalizer import normalize_upc
```
Import: library/module ke functions yahan available karata hai.
```
12: from document_rag import search as search_documents
```
Import: library/module ke functions yahan available karata hai.
```
13: from engines.conversation_store import ConversationStore
```
Import: library/module ke functions yahan available karata hai.
```
14:
```
Blank line: code ko readable banane ke liye.
```
15: class RAGOrchestrator:
```
Class: related data aur methods ka reusable blueprint.
```
16:     def __init__(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
17:         self.ollama_host = OLLAMA_HOST
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18:         self.ollama_model = OLLAMA_MODEL
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:         self._ollama_status = None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:         self._ollama_status_checked_at = 0.0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:         self.conversation_store = ConversationStore(DB_DIR / "conversation.sqlite3")
```
SQLite: local offline file database; fast indexed queries ke liye.
```
22:
```
Blank line: code ko readable banane ke liye.
```
23:     def is_ollama_available(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
24:         """Check if local Ollama server is running."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
25:         now = time.monotonic()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
26:         if self._ollama_status is not None and now - self._ollama_status_checked_at < 10:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
27:             return self._ollama_status
```
Return: calculated result caller ko wapas bhejta hai.
```
28:         try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
29:             r = requests.get(f"{self.ollama_host}/api/tags", timeout=1.0)
```
HTTP call: local Ollama API se connect karta hai.
```
30:             self._ollama_status = r.status_code == 200
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
31:         except Exception:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
32:             self._ollama_status = False
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
33:         self._ollama_status_checked_at = now
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:         return self._ollama_status
```
Return: calculated result caller ko wapas bhejta hai.
```
35:
```

Blank line: code ko readable banane ke liye.

```
36:     def query_ollama(self, prompt, system_prompt=None):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
37:         """Send prompt to local Ollama instance."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
38:         try:
```

Error handling: failure ko handle karke app ko crash hone se bachata hai.

```
39:             payload = {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
40:                 "model": self.ollama_model,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
41:                 "prompt": prompt,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
42:                 "stream": False,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
43:                   "system": system_prompt or "You are an expert beverage data assistant. Answer truthfully based
solely on retrieved context."
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
44:             }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
45:             r = requests.post(f"{self.ollama_host}/api/generate", json=payload, timeout=30)
```

HTTP call: local Ollama API se connect karta hai.

```
46:             if r.status_code == 200:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
47:                 return r.json().get('response', '')
```

Return: calculated result caller ko wapas bhejta hai.

```
48:         except Exception as e:
```

Error handling: failure ko handle karke app ko crash hone se bachata hai.

```
49:             pass
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
50:         return None
```

Return: calculated result caller ko wapas bhejta hai.

```
51:
```

Blank line: code ko readable banane ke liye.

```
52:     def route_and_execute(self, user_query, session_id='default-session'):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
53:         history = self.conversation_store.recent(session_id, limit=8)
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
54:         original_query = user_query.strip()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
55:         query = self._with_conversation_context(original_query, history)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
56:         result = self._route_and_execute(query, session_id=session_id)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
57:         self.conversation_store.append(session_id, "user", original_query)
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
58:         self.conversation_store.append(session_id, "assistant", result.get("answer", ""))
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
59:         result["conversation_context_used"] = query != original_query
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
60:         return result
```

Return: calculated result caller ko wapas bhejta hai.

```
61:
```

Blank line: code ko readable banane ke liye.

```
62:     def _route_and_execute(self, user_query, session_id='default-session'):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
63:         """
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
64:         Understands natural language questions in English and Hinglish,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
65:         executes relevant RAG / DB retrieval, and synthesizes accurate response.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
66:         """
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
67:         q = user_query.strip()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
68:         q_lower = q.lower()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
69:
```

Blank line: code ko readable banane ke liye.

```
70:         # Token for downloadable report
```

Comment: developer ka note; Python ise execute nahi karta.

```
71:         token = str(uuid.uuid4())[:8]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
  72:         report_records = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  73:         source_files = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  74:
```
Blank line: code ko readable banane ke liye.
```
  75:         # Keep deterministic definitions available even when they contain a
```
Comment: developer ka note; Python ise execute nahi karta.
```
  76:         # catalog term; other normal-chat messages should reach the LLM first.
```
Comment: developer ka note; Python ise execute nahi karta.
```
  77:         local_general = self._local_general_response(q)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  78:         is_definition = bool(re.search(r'\bwhat is\b|database kya hai|upc kya hai', q_lower))
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
  79:         if local_general and is_definition:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  80:             return self._format_response(local_general, [], [], token)
```
Return: calculated result caller ko wapas bhejta hai.
```
  81:
```
Blank line: code ko readable banane ke liye.
```
  82:         # Keep ordinary conversation away from the product vector index. The
```
Comment: developer ka note; Python ise execute nahi karta.
```
  83:         # index is useful for catalog questions, but it can produce false hits
```
Comment: developer ka note; Python ise execute nahi karta.
```
  84:         # for words such as "day", "joke", or "India" in product names.
```
Comment: developer ka note; Python ise execute nahi karta.
```
  85:         catalog_terms = [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  86:             'upc', 'barcode', 'product', 'brand', 'supplier', 'vendor',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  87:             'beer', 'wine', 'rtd', 'spirits', 'master', 'categorization',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  88:             'dataset', 'category', 'missing', 'compare', 'duplicate', 'audit', 'search', 'find'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  89:         ]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  90:         document_terms = ['document', 'pdf', 'file', 'upload', 'uploaded', 'image', 'sheet']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  91:         has_identifier_intent = bool(re.search(r'\b(?:id|identifier)\s*[:#-]?\s*[a-z0-9_-]+\b', q_lower))
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
  92:          has_catalog_intent = bool(re.search(r'\b\d{10,14}\b', q)) or has_identifier_intent or any(term in q_lower
for term in catalog_terms)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
  93:         has_document_intent = any(term in q_lower for term in document_terms)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  94:         if not has_catalog_intent and not has_document_intent:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  95:             if self.is_ollama_available():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  96:                 language = self._detect_response_language(q)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  97:                 general_system_prompt = (
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  98:                     "You are BevIQ, a friendly and capable general-purpose assistant. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
  99:                     "The user may ask about any topic, not only beverage data. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 100:                     "Reply in the same language and writing style as the user's latest message. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 101:                     "If the user writes in English, reply in English. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 102:                     "If the user writes in Hindi using Devanagari, reply in natural Hindi using Devanagari. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 103:                     "If the user writes in Hinglish/Roman Hindi, reply in natural Hinglish using Roman script. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 104:                     "If the user mixes English and Hindi, naturally match that mix. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 105:                     "Follow an explicit language request if the user asks for a different language. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 106:                     "Keep technical terms such as Python, Flask, API, SQL, and database in English. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 107:                     "Do not force beverage-related content into unrelated conversations. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 108:                     "Be helpful, conversational, accurate, and do not invent facts. "
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
109:                     f"Detected response style for this message: {language}."
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
110:                 )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
111:                 general = self.query_ollama(q, general_system_prompt)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
112:                 if general:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
113:                     return self._format_response(general, [], [], token)
```
Return: calculated result caller ko wapas bhejta hai.
```
114:
```
Blank line: code ko readable banane ke liye.
```
115:             local_general = self._local_general_response(q)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
116:             if local_general:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
117:                 return self._format_response(local_general, [], [], token)
```
Return: calculated result caller ko wapas bhejta hai.
```
118:
```
Blank line: code ko readable banane ke liye.
```
119:             detected_language = self._detect_response_language(q)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
120:             if detected_language == "Hindi":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
121:                 fallback = (
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
122:                     "### ? ??????? ??????\\n\\n"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
123:                     "??? ??????? ?????? ?? ?????? ??? ??? ?? ???? ???? "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
124:                     "???? ?????? ?? ??????? ???? ???? ?? ??? local Ollama ???? ????? "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
125:                     "BevIQ ?? database ?? uploaded-document ???? ???? ?????? ????"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
126:                 )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
127:             elif detected_language == "Hinglish":
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
128:                 fallback = (
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
129:                     "### ? Normal Chat\\n\\n"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
130:                     "Main normal questions aur casual conversation mein help kar sakta hoon. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
131:                     "Detailed open-ended answers ke liye local Ollama start karein. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
132:                     "BevIQ ke database aur uploaded-document questions bhi available hain."
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
133:                 )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
134:             else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
135:                 fallback = (
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
136:                     "### ? Normal Chat\\n\\n"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
137:                     "I can help with general questions and casual conversation. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
138:                     "Start local Ollama for detailed open-ended answers. "
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
139:                     "BevIQ database and uploaded-document questions are also available."
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
140:                 )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
141:             return self._format_response(fallback, [], [], token)
```
Return: calculated result caller ko wapas bhejta hai.
```
142:
```
Blank line: code ko readable banane ke liye.
```
143:         # Direct aggregates should never fall through to the slower vector index.
```
Comment: developer ka note; Python ise execute nahi karta.
```
144:         is_missing_intent = any(w in q_lower for w in ['missing', 'not found', 'absent', 'nahi hai', 'gap'])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
145:         is_master_brand_count = (
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
146:             bool(re.search(r'\b(how many|number of|count|kitne|kitni|gino)\b', q_lower))
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
147:             and 'brand' in q_lower
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
148:             and not is_missing_intent
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
149:         )
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
150:         if is_master_brand_count:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
151:                         category = next((f'cat_{name}' for name in ('beer', 'wine', 'rtd', 'spirits') if
re.search(rf'\b{name}\b', q_lower)), None)
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
152:             brand_count = sql_engine.count_brands(category) if category else sql_engine.count_master_brands()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
153:             label = category.replace('cat_', '').upper() if category else 'Master'
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
154:             if self._is_hinglish(q_lower):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
155:                 answer = f"### ? {label} Brand Summary\n\n{label} sheet mein **{brand_count:,} unique brands**
hain."
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
156:             else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
157:                 answer = f"### ? {label} Brand Summary\n\nThe {label.lower()} sheet contains **{brand_count:,}
unique brands**."
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
158:             return self._format_response(answer, [], [], token)
```

Return: calculated result caller ko wapas bhejta hai.

```
159:
```

Blank line: code ko readable banane ke liye.

```
160:         # Uploaded documents are scoped to the browser conversation. If a relevant
```

Comment: developer ka note; Python ise execute nahi karta.

```
161:         # chunk exists, answer from it before falling back to the beverage catalog.
```

Comment: developer ka note; Python ise execute nahi karta.

```
162:         document_hits = search_documents(q, session_id)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
163:         if document_hits:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
164:             context = "\n\n".join(
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
165:                 f"[{hit['filename']} - page/section {hit['page']}]\n{hit['content']}"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
166:                 for hit in document_hits
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
167:             )
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
168:             if self.is_ollama_available():
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
169:                     prompt = f"Use only this uploaded-document context to answer the question. Cite filename and
page/section.\n\nContext:\n{context}\n\nQuestion: {q}"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
170:                 answer = self.query_ollama(prompt, "You answer questions about user-uploaded documents. If the
answer is not in context, say so clearly.")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
171:             else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
172:                 answer = None
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
173:             if not answer:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
174:                 answer = "### ? Uploaded Document Answer\n\n" + "\n\n".join(
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
175:                     f"**{hit['filename']} ? page/section {hit['page']}**\n{hit['content'][:1200]}"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
176:                     for hit in document_hits
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
177:                 )
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
178:             return self._format_response(answer, document_hits,
```

Return: calculated result caller ko wapas bhejta hai.

```
179:                                          [hit['filename'] for hit in document_hits], token)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 180:
```
Blank line: code ko readable banane ke liye.
```
 181:         # 1. Check for UPC pattern (10 to 14 digits or 'upc' keyword)
```
Comment: developer ka note; Python ise execute nahi karta.
```
 182:         upc_match = re.search(r'\b(\d{10,14})\b', q)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 183:         is_upc_query = bool(upc_match) or 'upc' in q_lower or 'barcode' in q_lower
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 184:         identifier_match = re.search(r'\b(?:id|identifier)\s*[:#-]?\s*([a-z0-9_-]+)\b', q_lower)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 185:         is_master_brand_request = 'master brand' in q_lower or ('master' in q_lower and 'brand' in q_lower)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 186:           master_brand_phrase = re.search(r'(?:find|search|show)\s+(.+?)\s+with\s+(?:a\s+)?(?:upc|barcode)\b', q,
re.I)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 187:
```
Blank line: code ko readable banane ke liye.
```
 188:         # 2. Check for Comparison Intent
```
Comment: developer ka note; Python ise execute nahi karta.
```
 189:           is_compare = any(w in q_lower for w in ['compare', 'mismatch', 'diff', 'versus', 'vs', 'difference',
'farq'])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 190:         is_beer_rtd = ('beer' in q_lower and 'rtd' in q_lower) or 'hard seltzer' in q_lower
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 191:          is_supplier_brand = ('supplier' in q_lower and ('brand' in q_lower or 'sheet' in q_lower or 'master' in
q_lower))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 192:         is_missing = is_missing_intent
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 193:         is_duplicate = any(w in q_lower for w in ['duplicate', 'repeated', 'multiple', 'dohra'])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 194:           is_category_count = bool(re.search(r'\b(count|category count|gino|total count)\b', q_lower)) or 'total
products in each' in q_lower
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 195:         is_product_upc_gap = any(w in q_lower for w in [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 196:             'alternate upc', 'alternate barcode', 'same product', 'product match',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 197:             'upc missing', 'brand flavour size', 'brand flavor size',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 198:             'allmasterbrand', 'all master'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 199:         ])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 200:         # A supplier-only question should use the supplier search path below.
```
Comment: developer ka note; Python ise execute nahi karta.
```
 201:         # Brand profile is reserved for questions asking for product attributes.
```
Comment: developer ka note; Python ise execute nahi karta.
```
 202:         is_brand_profile = any(w in q_lower for w in [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 203:             'flavour', 'flavor', 'size', 'origin', 'country',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 204:             'details', 'profile'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 205:         ]) and not is_product_upc_gap
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 206:         is_supplier_search = any(w in q_lower for w in ['supplier', 'vendor']) and not is_compare
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 207:           is_brand_search = any(w in q_lower for w in ['brand', 'products belonging to', 'show products for',
'products of', 'search', 'find']) and not is_supplier_search
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 208:                                                                                 master_brand_query        =
re.sub(r'(?i)\b(show|find|search|master|brand|with|a|an|the|in|please|dikhao|batao|karo)\b', ' ', q).strip()
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 209:         master_brand_query = re.sub(r'\s+', ' ', master_brand_query).strip()
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 210:
```
Blank line: code ko readable banane ke liye.
```
 211:           category_words = [name for name in ('beer', 'wine', 'rtd', 'spirits') if re.search(rf'\b{name}\b',
q_lower)]
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 212:           if len(category_words) >= 1 and not is_compare and not is_upc_query and not is_brand_search and not
is_supplier_search and not is_product_upc_gap:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 213:             metrics = sql_engine.get_dashboard_metrics()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 214:             rows = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 215:             for name in category_words:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 216:                 item = metrics.get('categories', {}).get(name, {})
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 217:                         rows.append(f"- **{name.upper()}**: {int(item.get('total_records') or 0):,} records, {int(item.get('available_upcs') or 0):,} UPCs available, {int(item.get('missing_upcs') or 0):,} missing")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 218:             answer = "### ? Beverage Category Summary\n\n" + "\n".join(rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 219:             return self._format_response(answer, [], [], token)
```
Return: calculated result caller ko wapas bhejta hai.
```
 220:
```
Blank line: code ko readable banane ke liye.
```
 221:             if is_master_brand_request and not is_compare and not is_missing and not is_product_upc_gap and (identifier_match or upc_match or master_brand_query):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 222:             brand_query = (master_brand_phrase.group(1).strip() if master_brand_phrase else master_brand_query) or None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 223:             brand_query = re.sub(r'(?i)\s+in\s+master\s*$', '', brand_query or '').strip() or None
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
 224:             master_id = identifier_match.group(1) if identifier_match else None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 225:             res = sql_engine.search_master_brands(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 226:                 master_id=master_id,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 227:                 brand=brand_query,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 228:                 upc=upc_match.group(1) if upc_match else None,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 229:                 limit=200,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 230:             )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 231:             report_records = res.get('records', [])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 232:             source_files = ['AIMasterBrandData.csv'] if report_records else []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 233:             if report_records:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 234:                 details = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 235:                 for key, value in report_records[0].items():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 236:                     if value not in (None, ''):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 237:                         details.append(f"- **{key}**: {value}")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 238:                 answer_title = 'Master Brand Record' if len(report_records) == 1 else 'Master Brand Records'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 239:                 ans = (
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 240:                     f"### ? {answer_title}\n\n"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 241:                     f"Found **{len(report_records)} matching record(s)** in the Master Brand sheet.\n\n"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 242:                     + "\n".join(details[:40])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 243:                     + "\n\nAll details below are returned directly from the connected database."
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 244:                 )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 245:             else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 246:                 lookup_label = master_id or brand_query or (upc_match.group(1) if upc_match else q)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 247:                   ans = f"### ? Master Brand Record Not Found\n\nNo Master Brand record matched `{lookup_label}`. Try the exact ID, brand name, or UPC."
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
248:             if self._is_hinglish(q_lower):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
249:                     ans = ans.replace("Found **", "**Mila: **").replace(" matching record(s) in the Master Brand sheet.", " matching record(s) Master Brand sheet mein.").replace("No Master Brand record matched", "Master Bran
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
250:             return self._format_response(ans, report_records, source_files, token)
```
Return: calculated result caller ko wapas bhejta hai.
```
251:
```
Blank line: code ko readable banane ke liye.
```
252:         if identifier_match:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
253:             identifier = identifier_match.group(1)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
254:             res = sql_engine.lookup_identifier(identifier, limit=200)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
255:             report_records = res['records'] + res.get('related_records', [])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
256:             source_files = res.get('source_files', [])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
257:             if res['total_matches']:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
258:                 primary = report_records[0] if report_records else {}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
259:                                         product = primary.get('ProductName') or primary.get('Product') or primary.get('ProductDescription') or 'Not available'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
260:                                         brand = primary.get('BrandName') or primary.get('Brands_Value') or primary.get('Beer_Brands_Value') or primary.get('Spirits_Brands_Value') or 'Not available'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
261:                                     ans = f"### ? Complete ID Lookup: `{res['normalized_identifier']}`\n\nFound **{res['total_matches']} direct record(s)** and **{len(res.get('related_records', []))} related record(s)** across **{len(source
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
262:             else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
263:                     ans = f"### ? ID Not Found: `{res['normalized_identifier']}`\n\nNo exact matching ID, UPC, or alternate UPC was found in the configured beverage datasets."
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
264:             return self._format_response(ans, report_records, source_files, token)
```
Return: calculated result caller ko wapas bhejta hai.
```
265:
```
Blank line: code ko readable banane ke liye.
```
266:         if is_product_upc_gap:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
267:             from engines.upc_match_engine import upc_match_engine
```
Import: library/module ke functions yahan available karata hai.
```
268:                 category = next((f'cat_{name}' for name in ('beer', 'rtd', 'spirits', 'wine') if name in q_lower), None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
269:             report_records = upc_match_engine.audit(category, limit=None)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
270:             source_files = ['$ALLBrand_202610291524.csv', 'Categorization 2026 - Beer.csv', 'Categorization 2026 - RTDs.csv', 'Categorization 2026 - Spirits.csv', 'Categorization 2026 - Wine.csv']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
271:                 ans = f"### ? UPC-First Product Audit\n\nReviewed **{len(report_records)} category records** against AllBrand. The report includes exact UPC status, missing UPC details, product/flavour/size matches, and sugg
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
272:             return self._format_response(ans, report_records, source_files, token)
```
Return: calculated result caller ko wapas bhejta hai.
```
273:
```
Blank line: code ko readable banane ke liye.
```
274:         if is_brand_profile:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
275:             brand_guess = self._extract_brand_guess(q)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
276:             profile = sql_engine.brand_profile(brand_guess)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
277:             report_records = profile['records']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
278:             source_files = sorted({r.get('source_file') for r in report_records if r.get('source_file')})
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
279:             join = lambda values, fallback='Not available': ', '.join(values[:20]) if values else fallback
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
280:             if self._is_hinglish(q_lower):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
281:                 ans = f"""### ?? {profile['query_brand']} Brand Profile
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
282:
```

Blank line: code ko readable banane ke liye.

```
283: **Unique brand names**: {profile['unique_brand_count']:,} | **Total records**: {profile['total_records']:,}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
284:
```

Blank line: code ko readable banane ke liye.

```
285: - **Categories / sheets**: {', '.join(f"{c['category']} ({c['count']:,})" for c in profile['categories']) or 'Not
found'}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
286: - **Flavours ({profile['unique_flavour_count']:,} distinct)**: {join(profile['flavours'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
287: - **Sizes / packs ({profile['unique_size_count']:,} distinct)**: {join(profile['sizes'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
288: - **Suppliers**: {join(profile['suppliers'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
289: - **Origin / domestic-import**: {join(profile['origins'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
290: - **Distinct UPCs**: {profile['unique_upc_count']:,}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
291:
```

Blank line: code ko readable banane ke liye.

```
292: Neeche matching product records aur UPC details diye gaye hain."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
293:             else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
294:                 ans = f"""### ?? {profile['query_brand']} Brand Profile
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
295:
```

Blank line: code ko readable banane ke liye.

```
296: **Unique brand names**: {profile['unique_brand_count']:,} | **Total records**: {profile['total_records']:,}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
297:
```

Blank line: code ko readable banane ke liye.

```
298: - **Categories / sheets**: {', '.join(f"{c['category']} ({c['count']:,})" for c in profile['categories']) or 'Not
found'}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
299: - **Flavours ({profile['unique_flavour_count']:,} distinct)**: {join(profile['flavours'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
300: - **Sizes / packs ({profile['unique_size_count']:,} distinct)**: {join(profile['sizes'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
301: - **Suppliers**: {join(profile['suppliers'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
302: - **Origin / domestic-import**: {join(profile['origins'])}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
303: - **Distinct UPCs**: {profile['unique_upc_count']:,}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
304:
```

Blank line: code ko readable banane ke liye.

```
305: Matching product records and UPC details are shown below."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
306:             return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
307:
```

Blank line: code ko readable banane ke liye.

```
308:         # Handle Query Execution by Intent:
```

Comment: developer ka note; Python ise execute nahi karta.

```
309:
```

Blank line: code ko readable banane ke liye.

```
310:         # A. Beer vs RTD Comparison
```

Comment: developer ka note; Python ise execute nahi karta.

```
311:         if is_compare and is_beer_rtd:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
312:             res = comparison_engine.compare_beer_vs_rtd(limit=300)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
313:             report_records = res['records']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
314:             source_files = ["Categorization 2026 - Beer.csv", "Categorization 2026 - RTDs.csv"]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
315:
```

Blank line: code ko readable banane ke liye.

```
316:             summary = res['summary']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
317:             ans = f"""### ? Beer vs RTD Categorization Analysis
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
318:
```

Blank line: code ko readable banane ke liye.

```
319: - **Total Overlapping Products**: 50,376 UPCs exist in both Beer and RTDs datasets.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
320: - **Sample Analyzed**: {summary['overlapping_sample']} records
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
321: - **Verified 'NON RTD' in RTD Sheet**: {summary['verified_non_rtd']} records (Correctly mapped)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
322: - **Requires Verification**: {summary['requires_verification']} records (Flagged as FAB or Malt Beverage)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
323: - **Direct Mismatches**: {summary['mismatches']} records
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
324:
```

Blank line: code ko readable banane ke liye.

```
325: Most beers appear in the RTD dataset with category value `NON RTD`. Products flagged with `Requires Verification`
belong to flavored malt beverages (FMBs) or hard seltzers where category boundaries cross between standard
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
326:             return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
327:
```

Blank line: code ko readable banane ke liye.

```
328:         # B. AllBrand vs Supplier Comparison
```

Comment: developer ka note; Python ise execute nahi karta.

```
329:         if is_compare and is_supplier_brand:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
330:             res = comparison_engine.compare_allbrand_vs_supplier(limit=300)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
331:             report_records = res['records']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
332:             source_files = ["$ALLBrand_202610291524.csv", "$Supplier_202610291521.csv"]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
333:             summary = res['summary']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
334:
```

Blank line: code ko readable banane ke liye.

```
335:             ans = f"""### ? AllBrand vs Supplier Master Sheet Comparison
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
336:
```

Blank line: code ko readable banane ke liye.

```
337: - **Total Supplier Codes in Brand Master**: 15,325 unique codes
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
338: - **Matched with Supplier Sheet**: 15,324 codes (99.99% integrity match!)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
339: - **Missing Supplier Codes**: Only 1 code in Master is missing from official Supplier sheet
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
340: - **Records Evaluated in Sample**: {summary['records_analyzed']}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
341: - **Exact Supplier Name Matches**: {summary['exact_matches']}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
342: - **Name Mismatches / Variations**: {summary['mismatches']}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
343: - **Unmatched Suppliers in Sample**: {summary['missing_in_supplier_sheet']}"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
344:             return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
345:
```

Blank line: code ko readable banane ke liye.

```
346:         # C. Master vs Categorization Comparison (Beer / RTD / Spirits / Wine)
```

Comment: developer ka note; Python ise execute nahi karta.

```
347:             if (is_compare or is_missing) and ('master' in q_lower or 'beer' in q_lower or 'rtd' in q_lower or
'spirit' in q_lower or 'wine' in q_lower or 'sheet' in q_lower):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
348:             # Check if multiple categories or all categories requested
```

Comment: developer ka note; Python ise execute nahi karta.

```
349:             cat_mentions = [c for c in ['beer', 'wine', 'rtd', 'spirit'] if c in q_lower]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
350:                 if len(cat_mentions) >= 2 or any(w in q_lower for w in ['all', 'sab', 'charo', 'overall',
'together']):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
351:                 res = comparison_engine.compare_all_missing_in_master(limit_per_category=150)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
352:                 report_records = res['records']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
353:                             source_files = ["Categorization 2026 - Beer.csv", "Categorization 2026 - Wine.csv", "Categorization 2026 - RTDs.csv", "Categorization 2026 - Spirits.csv", "AIMasterBrandData.csv"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
354:                 sm = res['summary']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
355:
```
Blank line: code ko readable banane ke liye.
```
356:                 cat_rows_md = "\n".join([
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
357:                                         f"|  **{c['category']}**  |  `{c['source_file']}`  |  {c['total_upcs']:,}  | {c['matched_in_master']:,} | **{c['missing_from_master']:,}** | `{c['pct_missing']}` |"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
358:                     for c in sm['categories']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
359:                 ])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
360:
```
Blank line: code ko readable banane ke liye.
```
361:                 ans = f"""### ? Audit: Missing UPCs in Master Brands (Beer, Wine, RTD & Spirits)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
362:
```
Blank line: code ko readable banane ke liye.
```
363: Yahan sabhi 4 categories (Beer, Wine, RTD, Spirits) ka Master Brands (`AIMasterBrandData.csv`) ke sath exact missing UPC comparison report di gayi hai:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
364:
```
Blank line: code ko readable banane ke liye.
```
365: | Category | Source File | Total UPCs | In Master Brands | Missing in Master | % Missing |
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
366: | :--- | :--- | :---: | :---: | :---: | :---: |
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
367: {cat_rows_md}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
368:     |     **TOTAL**     |     *Across     All     4     Files*     |     **{sm['total_categorization_upcs']:,}**     | **{sm['total_matched_in_master']:,}** | **{sm['total_missing_from_master']:,}** | **`{sm['overall_missing_pct']}%`** |
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
369:
```
Blank line: code ko readable banane ke liye.
```
370: #### ? Key Takeaways (Important Insights):
```
Comment: developer ka note; Python ise execute nahi karta.
```
371:  1.  **Total  {sm['total_missing_from_master']:,}  unique  UPCs**  categorization  sheets  me  available  hain  jo **AIMasterBrandData me missing hain**.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
372: 2. **Beer** me sabse zyada gap hai (**65.5% missing** - 33,047 UPCs).
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
373: 3. **RTD** me **60,020 UPCs** master catalog me nahi hain.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
374: 4. **Wine** me **15,380 UPCs** aur **Spirits** me **12,089 UPCs** missing hain.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
375:
```
Blank line: code ko readable banane ke liye.
```
376: *Neeche di gayi table me sample records dekhein aur poora CSV export download karne ke liye **Download CSV Report** button par click karein.*"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
377:                 return self._format_response(ans, report_records, source_files, token)
```
Return: calculated result caller ko wapas bhejta hai.
```
378:
```
Blank line: code ko readable banane ke liye.
```
379:             # Single category comparison
```
Comment: developer ka note; Python ise execute nahi karta.
```
380:             cat_target = 'cat_beer'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
381:             cat_name = 'Beer'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
382:             if 'wine' in q_lower:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
383:                 cat_target = 'cat_wine'; cat_name = 'Wine'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
384:             elif 'spirit' in q_lower:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
385:                 cat_target = 'cat_spirits'; cat_name = 'Spirits'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
386:             elif 'rtd' in q_lower:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
387:                 cat_target = 'cat_rtd'; cat_name = 'RTDs'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
388:
```
Blank line: code ko readable banane ke liye.
```
389:             res = comparison_engine.compare_master_vs_categorization(cat_target, limit=300)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
390:             report_records = res['records']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
391:             source_files = ["AIMasterBrandData.csv", f"Categorization 2026 - {cat_name}.csv"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
392:             summary = res['summary']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
393:
```
Blank line: code ko readable banane ke liye.
```
394:             if self._is_hinglish(q_lower):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
395:                     ans = f"""### ? Master Data vs {cat_name} Categorization Comparison
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
396:
```
Blank line: code ko readable banane ke liye.
```
397: - **Master mein total records**: {summary['master_total_records']:,}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
398: - **{cat_name} sheet mein total records**: {summary['categorization_total_records']:,}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
399: - **{cat_name} ke unique UPCs**: {summary['categorization_unique_upcs']:,}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
400: - **Dono sheets mein matching UPCs**: {summary['matching_upcs_in_both']:,} ({summary['match_rate_pct']}%)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
401: - **Master mein missing UPCs**: {summary['missing_from_master_total']:,}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
402:
```
Blank line: code ko readable banane ke liye.
```
403: Sample review mein:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
404: - **Exact matches**: {summary['exact_matches_in_sample']}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
405: - **Partial brand matches**: {summary['partial_matches_in_sample']}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
406: - **Brand mismatch**: {summary['mismatches_in_sample']}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
407: - **Missing records**: {summary['missing_in_sample']}"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
408:             else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
409:                     ans = f"""### ? Master Data vs {cat_name} Categorization Comparison
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
410:
```
Blank line: code ko readable banane ke liye.
```
411: - **Master Total Records**: {summary['master_total_records']:,}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
412: - **{cat_name} Total Records**: {summary['categorization_total_records']:,}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
413: - **Unique UPCs in {cat_name}**: {summary['categorization_unique_upcs']:,}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
414: - **Matching UPCs (Found in Both)**: {summary['matching_upcs_in_both']:,} ({summary['match_rate_pct']}%)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
415: - **Missing from Master Catalog**: {summary['missing_from_master_total']:,} UPCs present in {cat_name} do not exist in AIMasterBrandData.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
416:
```
Blank line: code ko readable banane ke liye.
```
417: In the inspected sample of {summary['sample_analyzed']} records:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
418: - **Exact Matches**: {summary['exact_matches_in_sample']}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
419: - **Partial Brand Name Matches**: {summary['partial_matches_in_sample']}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
420: - **Brand Mismatches**: {summary['mismatches_in_sample']}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
421: - **Missing Records**: {summary['missing_in_sample']}"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
422:             return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
423:
```

Blank line: code ko readable banane ke liye.

```
424:         # D. Duplicate UPC Audit
```

Comment: developer ka note; Python ise execute nahi karta.

```
425:         if is_duplicate:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
426:             res = comparison_engine.find_duplicate_upcs_across_all(limit=100)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
427:             report_records = res['records']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
428:                 source_files = ["AIMasterBrandData.csv", "Categorization 2026 - Beer.csv", "Categorization 2026 - RTDs.csv", "Categorization 2026 - Spirits.csv", "Categorization 2026 - Wine.csv"]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
429:             ans = f"""### ?? Duplicate UPCs Audit Across Datasets
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
430:
```

Blank line: code ko readable banane ke liye.

```
431: - Evaluated cross-file presence across Master, Beer, RTDs, Spirits, and Wine.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
432: - Identified **{len(report_records)} primary duplicate groups** where identical UPCs appear across 2 or more separate files.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
433: - Common occurrences include beer items tracked concurrently in RTD sheets and master items shared with categorization sheets."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
434:             return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
435:
```

Blank line: code ko readable banane ke liye.

```
436:         # E. Exact UPC Lookup
```

Comment: developer ka note; Python ise execute nahi karta.

```
437:         if is_upc_query and upc_match:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
438:             upc_val = upc_match.group(1)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
439:             res = sql_engine.lookup_upc(upc_val)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
440:             report_records = res['records']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
441:             source_files = res['found_in_files']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
442:
```

Blank line: code ko readable banane ke liye.

```
443:             if res['total_matches'] > 0:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
444:                 row_refs = [str(r.get('original_row', r.get('_original_row', 'N/A'))) for r in report_records]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
445:                 ans = f"""### ? Product Lookup: UPC `{res['normalized_upc']}`
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
446:
```

Blank line: code ko readable banane ke liye.

```
447: Found **{res['total_matches']} matching record(s)** across **{len(source_files)} source file(s)**:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
448: - **Source Files**: {', '.join(source_files)}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
449: - **Original Row Reference(s)**: {', '.join(row_refs)}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
450: - **Primary Brand**: {report_records[0].get('BrandName', 'N/A')}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
451: - **Product Type / Desc**: {report_records[0].get('ProductName') or report_records[0].get('Product') or 'N/A'}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
452: - **Supplier**: {report_records[0].get('SupplierName', 'N/A')}"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
453:             else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
454:                 ans = f"""### ? Product Lookup: UPC `{res['normalized_upc']}`
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
455:
```

Blank line: code ko readable banane ke liye.

```
456: The requested UPC was **not found** in any of the 7 beverage datasets.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
457: - Checked: Master Brand Data, AllBrand Master, Beer, RTDs, Spirits, and Wine.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
458: - Recommendation: Verify the digits or check whether it is listed under an alternate UPC code."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
459:             return self._format_response(ans, report_records, source_files, token)
```
Return: calculated result caller ko wapas bhejta hai.
```
460:
```
Blank line: code ko readable banane ke liye.
```
461:         # F. Category Product Counts
```
Comment: developer ka note; Python ise execute nahi karta.
```
462:         if is_category_count:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
463:             res = sql_engine.count_by_category()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
464:             master_cats = res['master_categories']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
465:             beer_cats = res['beer_categories']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
466:             report_records = master_cats + beer_cats
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
467:             source_files = ["AIMasterBrandData.csv", "Categorization 2026 - Beer.csv"]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
468:
```
Blank line: code ko readable banane ke liye.
```
469:             ans = f"""### ? Beverage Product Counts by Category
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
470:
```
Blank line: code ko readable banane ke liye.
```
471: #### AIMaster Brand Catalog:
```
Comment: developer ka note; Python ise execute nahi karta.
```
472:  """  +  "\n".join([f"-  **{r['category']}  or  'Uncategorized'}**:  {r['product_count']:,}  products"  for  r  in
master_cats[:8]]) + f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
473:
```
Blank line: code ko readable banane ke liye.
```
474: #### Beer Categorization:
```
Comment: developer ka note; Python ise execute nahi karta.
```
475:  """  +  "\n".join([f"-  **{r['category']}  or  'Uncategorized'}**:  {r['product_count']:,}  products"  for  r  in
beer_cats[:6]])
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
476:             return self._format_response(ans, report_records, source_files, token)
```
Return: calculated result caller ko wapas bhejta hai.
```
477:
```
Blank line: code ko readable banane ke liye.
```
478:         # G. Brand Search
```
Comment: developer ka note; Python ise execute nahi karta.
```
479:          if is_brand_search or any(b in q_lower for b in ['michelob', 'modelo', 'bud light', 'corona', 'twisted
tea', 'titos', 'la marca']):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
480:             # Extract brand name guess
```
Comment: developer ka note; Python ise execute nahi karta.
```
     481:                                                                                                  brand_guess      =
re.sub(r'(?i)(show|find|search|all|products?|product|description|belonging|to|for|brand|karo|dikhao)', '', q).strip()
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
482:             if not brand_guess:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
483:                 brand_guess = q
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
484:             res = sql_engine.search_catalog(brand_guess, limit=100)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
485:             report_records = res['records']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
486:             source_files = sorted({r.get('source_file') for r in report_records if r.get('source_file')})
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
487:
```
Blank line: code ko readable banane ke liye.
```
488:             if res['total_matches'] > 0:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
489:                 sample = report_records[0]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
490:                 if self._is_hinglish(q_lower):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
491:                     ans = f"### ? Catalog Search: '{res['query']}'\n\nAll sheets mein **{res['total_matches']}
matching record(s)** mile.\n\n**Sample product description**: {sample.get('product_description') or 'Not avai
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
492:                 else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 493:                         ans = f"### ? Catalog Search: '{res['query']}'\n\nFound **{res['total_matches']} matching
record(s)** across the Master, Beer, Wine, RTD and Spirits sheets.\n\n**Sample product description**: {sample.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 494:                 return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
 495:
```

Blank line: code ko readable banane ke liye.

```
 496:             if res['total_matches'] > 0:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 497:                 ans = f"""### ?? Brand Search: '{res['query_brand']}'
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 498:
```

Blank line: code ko readable banane ke liye.

```
 499: Found **{res['total_matches']} matching record(s)** in AIMasterBrandData.csv:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 500: - **Brand Name**: {report_records[0].get('BrandName')}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 501: - **Supplier**: {report_records[0].get('SupplierName')}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 502: - **Sample Sizes**: {', '.join(set([r.get('Size') for r in report_records if r.get('Size')])) or 'Various'}"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 503:                 return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
 504:             else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 505:                 pass
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 506:
```

Blank line: code ko readable banane ke liye.

```
 507:         # H. Supplier Search
```

Comment: developer ka note; Python ise execute nahi karta.

```
 508:         if is_supplier_search:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 509:             supp_guess = re.sub(r'(?i)(find|show|products|with|supplier|vendor|dikhao)', '', q).strip()
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
 510:             res = sql_engine.search_supplier(supp_guess, limit=100)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 511:             report_records = res['records']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 512:             source_files = ["AIMasterBrandData.csv"]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 513:             if res['total_matches'] > 0:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 514:                 ans = f"""### ? Supplier Search: '{res['query_supplier']}'
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 515:
```

Blank line: code ko readable banane ke liye.

```
 516: Found **{res['total_matches']} matching record(s)** associated with supplier '{res['query_supplier']}':
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 517: - **Supplier Name**: {report_records[0].get('SupplierName')}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 518: - **Supplier Code**: {report_records[0].get('SupplierCode', 'N/A')}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 519: - **Associated Brands**: {', '.join(set([r.get('BrandName') for r in report_records if r.get('BrandName')])
)[:200]}"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 520:                 return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
 521:
```

Blank line: code ko readable banane ke liye.

```
 522:         # I. Fallback to Semantic Vector Search + Local LLM
```

Comment: developer ka note; Python ise execute nahi karta.

```
 523:         docs = vector_engine.search(q, top_k=6)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 524:         report_records = docs
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 525:         source_files = list(set([d.get('source_file') for d in docs if d.get('source_file')]))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 526:
```

Blank line: code ko readable banane ke liye.

```
 527:         if docs:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 528:             context_text = "\n".join([f"[{d['source_file']} Line {d['original_row']}] UPC: {d['upc']} | Brand:
{d['brand']} | Supplier: {d['supplier']} | Category: {d['category']} | Desc: {d['description']}" for d in doc
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
529:
```

Blank line: code ko readable banane ke liye.

```
530:             # If Ollama is running locally, use it to generate conversational synthesis
```

Comment: developer ka note; Python ise execute nahi karta.

```
531:             if self.is_ollama_available():
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
532:                     prompt = f"""Based on the following retrieved beverage records, provide a direct, helpful,
accurate answer to the user's question.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
533:
```

Blank line: code ko readable banane ke liye.

```
534: Retrieved Records:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
535: {context_text}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
536:
```

Blank line: code ko readable banane ke liye.

```
537: User Question: {q}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
538:
```

Blank line: code ko readable banane ke liye.

```
539: Answer with source file references and exact details:"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
540:                 ollama_ans = self.query_ollama(prompt)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
541:                 if ollama_ans:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
542:                     return self._format_response(ollama_ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
543:
```

Blank line: code ko readable banane ke liye.

```
544:             # Deterministic Semantic Answer
```

Comment: developer ka note; Python ise execute nahi karta.

```
545:             ans = f"""### ? Semantic Search Results for: *"{q}"*
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
546:
```

Blank line: code ko readable banane ke liye.

```
547: Retrieved **{len(docs)} highly relevant beverage records** from local vector storage:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
548: """ + "\n".join([f"- **{d['brand']}** (`UPC: {d['upc']}`): {d['description']} *(Source: `{d['source_file']}`, Row:
{d['original_row']})*" for d in docs[:5]])
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
549:         else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
550:             if self.is_ollama_available():
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
551:                     general = self.query_ollama(q, "You are a helpful general chat assistant with beverage-data
expertise. Be concise, honest, and answer in the user's language. Do not invent database facts unless they are i
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
552:                 if general:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
553:                     return self._format_response(general, [], [], token)
```

Return: calculated result caller ko wapas bhejta hai.

```
554:             local_general = self._local_general_response(q)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
555:             if local_general:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
556:                 return self._format_response(local_general, [], [], token)
```

Return: calculated result caller ko wapas bhejta hai.

```
557:             ans = f"""### ?? No Direct Records Found
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
558:
```

Blank line: code ko readable banane ke liye.

```
559: No matching beverage records were retrieved for query: *"{q}"*.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
560: - Try searching with a 13-digit UPC, exact brand name (e.g. *Michelob Ultra*, *Modelo*, *Titos*), or comparison
keywords (e.g. *Compare Master with Beer*)."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
561:
```

Blank line: code ko readable banane ke liye.

```
562:         return self._format_response(ans, report_records, source_files, token)
```

Return: calculated result caller ko wapas bhejta hai.

```
563:
```

Blank line: code ko readable banane ke liye.

```
564:     def _with_conversation_context(self, query, history):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
565:         """Resolve short pronoun-based follow-ups without changing explicit searches."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
566:         if not history:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
567:             return query
```

Return: calculated result caller ko wapas bhejta hai.

```
568:         lowered = query.lower()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
569:         contextual = len(query.split()) <= 8 and bool(re.search(
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
570:             r'\b(what about|and|also|it|that|this|same|iska|uska|iske|uske|aur)\b', lowered
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
571:         ))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
572:         if not contextual:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
573:             return query
```

Return: calculated result caller ko wapas bhejta hai.

```
574:         previous_user = next((turn["content"] for turn in reversed(history) if turn["role"] == "user"), "")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
575:         if not previous_user:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
576:             return query
```

Return: calculated result caller ko wapas bhejta hai.

```
577:         return query + "\nPrevious user request for context: " + previous_user
```

Return: calculated result caller ko wapas bhejta hai.

```
578:
```

Blank line: code ko readable banane ke liye.

```
579:     def _local_general_response(self, query):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
580:         """Keep basic normal chat useful even when Ollama is not running."""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
581:         text = query.lower().strip()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
582:         if re.search(r'\bwhat is (a )?database\b', text) or 'database kya hai' in text:
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
583:             if self._detect_response_language(text) == 'Hinglish':
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
584:                 return "### ?? Database\n\nDatabase ek organized collection hota hai jahan records ko securely store, search aur update kiya ja sakta hai. BevIQ mein beverage products, UPCs, brands aur suppliers database
```

Return: calculated result caller ko wapas bhejta hai.

```
585:             return "### ?? Database\n\nA database is an organized collection of information that can be stored, searched, and updated reliably. In BevIQ, beverage products, UPCs, brands, and suppliers are read from the c
```

Return: calculated result caller ko wapas bhejta hai.

```
586:         if re.search(r'\bwhat is (an )?upc\b', text) or 'upc kya hai' in text:
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
587:             if self._detect_response_language(text) == 'Hinglish':
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
588:                 return "### ?? UPC\n\nUPC (Universal Product Code) product ka barcode identifier hota hai. BevIQ mein UPC se exact product, brand, category aur Master Brand record lookup kiya ja sakta hai."
```

Return: calculated result caller ko wapas bhejta hai.

```
589:             return "### ?? UPC\n\nUPC (Universal Product Code) is a barcode identifier for a product. BevIQ uses it to find exact product, brand, category, and Master Brand records."
```

Return: calculated result caller ko wapas bhejta hai.

```
590:         if not self._is_hinglish(text):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
591:             if re.search(r'^(hi|hello|hey|hii)\b', text):
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.

```
592:                 return "### ? Hello!\n\nI?m the BevIQ Assistant. You can chat normally or ask about beverage data, UPCs, comparisons, and uploaded documents."
```

Return: calculated result caller ko wapas bhejta hai.

```
593:             if 'how are you' in text:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
594:                 return "### ? I?m ready!\n\nYou can chat casually or ask me about a product, UPC, PDF, image, or dataset."
```

Return: calculated result caller ko wapas bhejta hai.

```
595:             if 'capital of india' in text:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
596:                 return "### ?? Answer\n\nThe capital of India is **New Delhi**."
```

Return: calculated result caller ko wapas bhejta hai.

```
597:             if re.search(r'\b(joke|funny|make me laugh)\b', text) and re.search(r'\b(tell|give|share|about|hear|make)\b', text):
```

Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
598:                 return "### ? Joke\n\nWhy did the data analyst drink coffee? To clear the **cache**!"
```
Return: calculated result caller ko wapas bhejta hai.
```
599:             if any(word in text for word in ['thank you', 'thanks']):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
600:                 return "### ? You?re welcome!\n\nGlad to help. Ask me anything whenever you like."
```
Return: calculated result caller ko wapas bhejta hai.
```
601:             if any(word in text for word in ['what can you do', 'help', 'features']):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
602:                 return "### ? BevIQ Help\n\n- Normal chat in English or Hinglish\n- UPC, brand, supplier, and category search\n- Beer, Wine, RTD, and Spirits comparisons\n- Missing/product-match UPC audits\n- Questions a
```
Return: calculated result caller ko wapas bhejta hai.
```
603:         if re.search(r'^(hi|hello|hey|hii|namaste|??????)\b', text):
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
604:                 return "### ? Hello!\n\nMain BevIQ Assistant hoon. Aap normal chat, beverage data, UPC lookup, comparisons, ya uploaded documents ke baare mein pooch sakte ho."
```
Return: calculated result caller ko wapas bhejta hai.
```
605:         if any(word in text for word in ['how are you', 'kaise ho', 'kya haal']):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
606:             return "### ? Main ready hoon!\n\nAap casual chat kar sakte ho ya koi product, UPC, PDF, image, ya dataset question pooch sakte ho."
```
Return: calculated result caller ko wapas bhejta hai.
```
607:         if 'capital of india' in text or 'india ki capital' in text:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
608:             return "### ?? Answer\n\nIndia ki capital **New Delhi** hai."
```
Return: calculated result caller ko wapas bhejta hai.
```
609:         if (
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
610:             re.search(r'\b(joke|funny|hansi|hasao)\b', text)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
611:             and re.search(r'\b(tell|give|share|about|suna|sunao|batao|make)\b', text)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
612:         ):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
613:             return "### ? Joke\n\nData analyst ne coffee kyun pi? Kyunki usse apna **cache** clear karna tha!"
```
Return: calculated result caller ko wapas bhejta hai.
```
614:         if any(word in text for word in ['thank you', 'thanks', 'dhanyavaad', 'shukriya']):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
615:             return "### ? Welcome\n\nKhushi hui! Jab chahein pooch sakte ho."
```
Return: calculated result caller ko wapas bhejta hai.
```
616:         if any(word in text for word in ['what can you do', 'help', 'kya kar sakte', 'features', 'madad']):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
617:             return "### ? BevIQ Help\n\n- Normal chat in English/Hinglish\n- UPC, brand, supplier aur category search\n- Beer, Wine, RTD aur Spirits comparison\n- Missing/product-match UPC audit\n- PDF, image, DOCX, CSV
```
Return: calculated result caller ko wapas bhejta hai.
```
618:         return None
```
Return: calculated result caller ko wapas bhejta hai.
```
619:
```
Blank line: code ko readable banane ke liye.
```
620:     def _detect_response_language(self, text):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
621:         """Detect the reply style from the latest user message."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
622:         if re.search(r'[\\u0900-\\u097f]', text):
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
623:             return "Hindi"
```
Return: calculated result caller ko wapas bhejta hai.
```
624:
```
Blank line: code ko readable banane ke liye.
```
625:         hindi_markers = [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
626:             'kya', 'hai', 'hain', 'hoon', 'karna', 'karo', 'kaise', 'kyun',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
627:             'mujhe', 'mera', 'mere', 'aap', 'apka', 'aapka', 'batao',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
628:             'dikhao', 'chahiye', 'nahi', 'nahin', 'mein', 'mujhko',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
629:             'kyunki', 'acha', 'achha', 'haan'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
630:         ]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
631:         if any(re.search(rf'\\b{re.escape(marker)}\\b', text.lower()) for marker in hindi_markers):
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
632:             return "Hinglish"
```

Return: calculated result caller ko wapas bhejta hai.
```
633:         return "English"
```
Return: calculated result caller ko wapas bhejta hai.
```
634:
```
Blank line: code ko readable banane ke liye.
```
635:     def _is_hinglish(self, text):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
636:         """Backward-compatible helper used by existing deterministic responses."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
637:         return self._detect_response_language(text) in ("Hindi", "Hinglish")
```
Return: calculated result caller ko wapas bhejta hai.
```
638:
```
Blank line: code ko readable banane ke liye.
```
639:     def _extract_brand_guess(self, query):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
640:         """Remove profile question words while keeping a multi-word brand name."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
641:         cleaned = re.sub(r'[^a-zA-Z0-9&.\- ]+', ' ', query)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
642:         stop_words = {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
643:             'mujhe', 'batao', 'dikhao', 'show', 'find', 'tell', 'me', 'about',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
644:             'how', 'many', 'what', 'which', 'are', 'is', 'the', 'this', 'brand',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
645:             'ke', 'ka', 'ki', 'hain', 'hai', 'kya', 'mein', 'me', 'present',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
646:             'kitne', 'kitni', 'flavour', 'flavours', 'flavor', 'flavors',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
647:             'size', 'sizes', 'supplier', 'suppliers', 'origin', 'origins',
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
648:             'country', 'details', 'profile', 'and', 'or', 'aur', 'please', 'products'
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
649:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
650:         words = [word for word in cleaned.split() if word.lower() not in stop_words]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
651:         return ' '.join(words).strip() or query.strip()
```
Return: calculated result caller ko wapas bhejta hai.
```
652:
```
Blank line: code ko readable banane ke liye.
```
653:     def _format_response(self, markdown_text, records, source_files, token):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
654:         # Save records for CSV download
```
Comment: developer ka note; Python ise execute nahi karta.
```
655:         if records:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
656:             try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
657:                 df = pd.DataFrame(records)
```
Pandas: CSV/table data ko DataFrame mein process karta hai.
```
658:                 csv_path = REPORTS_DIR / f"report_{token}.csv"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
659:                 df.to_csv(csv_path, index=False)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
660:             except Exception as e:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
661:                 pass
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
662:
```
Blank line: code ko readable banane ke liye.
```
663:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
664:             "answer": markdown_text,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
665:             "source_files": source_files,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
666:             "record_count": len(records),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
667:             "records": records[:100],  # send first 100 to UI table
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
668:             "download_token": token if records else None,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
669:             "read_only": True
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
670:             }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
671:
```

Blank line: code ko readable banane ke liye.

```
672: rag_orchestrator = RAGOrchestrator()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

File: engines/comparison_engine.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: import pandas as pd
```
Import: library/module ke functions yahan available karata hai.
```
 2: import json
```
Import: library/module ke functions yahan available karata hai.
```
 3: import logging
```
Import: library/module ke functions yahan available karata hai.
```
 4: from config import CSV_FILES, REPORTS_DIR
```
Import: library/module ke functions yahan available karata hai.
```
 5: from database.db_manager import db_manager
```
Import: library/module ke functions yahan available karata hai.
```
 6:
```
Blank line: code ko readable banane ke liye.
```
 7: logger = logging.getLogger("ComparisonEngine")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 8:
```
Blank line: code ko readable banane ke liye.
```
 9: class ComparisonEngine:
```
Class: related data aur methods ka reusable blueprint.
```
10:     def __init__(self, db=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
11:         self.db = db or db_manager
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
12:
```
Blank line: code ko readable banane ke liye.
```
13:     def compare_master_vs_categorization(self, cat_key='cat_beer', limit=500, search=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
14:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
15:         Compare AIMasterBrandData vs a Categorization Sheet (Beer, RTDs, Spirits, or Wine).
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
16:         Classifies records into Exact Match, Partial Match, Mismatch, Missing in Master, Duplicate.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
17:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
18:         master_table = CSV_FILES['ai_master']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:         cat_table = CSV_FILES.get(cat_key, {}).get('table', 'cat_beer')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
20:         cat_file = CSV_FILES.get(cat_key, {}).get('file', cat_key)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:
```
Blank line: code ko readable banane ke liye.
```
22:         # 1. Total counts
```
Comment: developer ka note; Python ise execute nahi karta.
```
23:         master_count = self.db.query(f"SELECT COUNT(*) as c FROM `{master_table}`")[0]['c']
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
24:         cat_count = self.db.query(f"SELECT COUNT(*) as c FROM `{cat_table}`")[0]['c']
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
25:
```
Blank line: code ko readable banane ke liye.
```
26:         # 2. Overlap query (matching UPCs)
```
Comment: developer ka note; Python ise execute nahi karta.
```
27:         search_clause = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
28:         search_params = ()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
29:         if search:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
30:             search_clause = " AND (LOWER(COALESCE(m.BrandName,'')) LIKE ? OR LOWER(COALESCE(c.norm_brand,'')) LIKE
? OR LOWER(COALESCE(c.Product,'')) LIKE ? OR c.clean_upc LIKE ?)"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
31:             needle = f"%{search.lower()}%"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
32:             search_params = (needle, needle, needle, f"%{search}%")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
33:         match_sql = f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:         SELECT
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
35:             c.clean_upc AS upc,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:             m.source_file AS master_file,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
37:             m.original_row AS master_row,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
38:             m.BrandName AS master_brand,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
39:             m.SupplierName AS master_supplier,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
40:             m.ProductName AS master_product_type,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
41:             c.source_file AS cat_file,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
42:             c.original_row AS cat_row,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
43:             c.norm_brand AS cat_brand,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
44:             COALESCE(c.Product, c.clean_upc) AS cat_product_desc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
45:         FROM `{cat_table}` c
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
46:         JOIN `{master_table}` m ON c.clean_upc = m.clean_upc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
47:         WHERE c.clean_upc != ''{search_clause}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
48:         LIMIT {limit}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
49:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
50:         matches = self.db.query(match_sql, search_params)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
51:
```
Blank line: code ko readable banane ke liye.
```
52:         # Classify each match
```
Comment: developer ka note; Python ise execute nahi karta.
```
53:         records = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
54:         exact_matches = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
55:         partial_matches = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
56:         brand_mismatches = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
57:
```
Blank line: code ko readable banane ke liye.
```
58:         for r in matches:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
59:             m_brand = (r.get('master_brand') or '').strip().upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
60:             c_brand = (r.get('cat_brand') or '').strip().upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
61:
```
Blank line: code ko readable banane ke liye.
```
62:             if m_brand == c_brand:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
63:                 status = "Exact Match"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
64:                 reason = "UPC and Brand Name align perfectly"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
65:                 exact_matches += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
66:             elif m_brand in c_brand or c_brand in m_brand:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
67:                 status = "Partial Match"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
68:                 reason = f"Brand variation: Master='{r.get('master_brand')}' vs Sheet='{r.get('cat_brand')}'"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
69:                 partial_matches += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
70:             else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
71:                 status = "Mismatch"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
                reason = f"Brand discrepancy: Master='{r.get('master_brand')}' vs Sheet='{r.get('cat_brand')}'"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                brand_mismatches += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
            records.append({
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "upc": r.get('upc'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "status": status,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "reason": reason,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "master_brand": r.get('master_brand'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "cat_brand": r.get('cat_brand'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "master_file": r.get('master_file'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "master_row": r.get('master_row'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "cat_file": r.get('cat_file'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "cat_row": r.get('cat_row'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                "product_desc": r.get('cat_product_desc')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```

```
Blank line: code ko readable banane ke liye.
```
        # 3. Missing in Master (present in Cat Sheet but missing from Master)
```
Comment: developer ka note; Python ise execute nahi karta.
```
        missing_search_clause = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        missing_search_params = ()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        if search:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            missing_search_clause = " AND (LOWER(COALESCE(c.norm_brand,'')) LIKE ? OR LOWER(COALESCE(c.Product,'')) LIKE ? OR c.clean_upc LIKE ?)"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            needle = f"%{search.lower()}%"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            missing_search_params = (needle, needle, f"%{search}%")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        missing_in_master_sql = f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        SELECT
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            c.clean_upc AS upc,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            c.source_file AS cat_file,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            c.original_row AS cat_row,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            c.norm_brand AS cat_brand,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
            COALESCE(c.Product, c.clean_upc) AS cat_product_desc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        FROM `{cat_table}` c
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        LEFT JOIN `{master_table}` m ON c.clean_upc = m.clean_upc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        WHERE m.clean_upc IS NULL AND c.clean_upc != ''{missing_search_clause}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        LIMIT {min(limit, 200)}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        missing_rows = self.db.query(missing_in_master_sql, missing_search_params)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
        for r in missing_rows:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
109:             records.append({
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
110:                 "upc": r.get('upc'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
111:                 "status": "Missing Record",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
112:                 "reason": f"UPC exists in {cat_file} but is completely missing from Master",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
113:                 "master_brand": "MISSING",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
114:                 "cat_brand": r.get('cat_brand'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
115:                 "master_file": "AIMasterBrandData.csv",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
116:                 "master_row": "N/A",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
117:                 "cat_file": r.get('cat_file'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
118:                 "cat_row": r.get('cat_row'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
119:                 "product_desc": r.get('cat_product_desc')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
120:             })
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
121:
```

Blank line: code ko readable banane ke liye.

```
122:         # 4. Aggregated stats
```

Comment: developer ka note; Python ise execute nahi karta.

```
123:         total_matched_count = self.db.query(f"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
124:             SELECT COUNT(DISTINCT c.clean_upc) as c
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
125:             FROM `{cat_table}` c
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
126:             JOIN `{master_table}` m ON c.clean_upc = m.clean_upc
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
127:             WHERE c.clean_upc != ''
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
128:         """)[0]['c']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
129:
```

Blank line: code ko readable banane ke liye.

```
130:         total_cat_upcs = self.db.query(f"SELECT COUNT(DISTINCT clean_upc) as c FROM `{cat_table}` WHERE clean_upc
!= ''")[0]['c']
```

SQL: database se data read/write ya schema/index tayyar hota hai.

```
131:         missing_from_master_total = total_cat_upcs - total_matched_count
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
132:
```

Blank line: code ko readable banane ke liye.

```
133:         category_labels = {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
134:             "cat_beer": "Beer",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
135:             "cat_rtd": "RTD",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
136:             "cat_spirits": "Spirits",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
137:             "cat_wine": "Wine",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
138:         }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
139:         return {
```

Return: calculated result caller ko wapas bhejta hai.

```
140:             "comparison_type": f"Master vs {category_labels.get(cat_key, cat_key)}",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
141:             "summary": {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
142:                 "master_total_records": master_count,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
143:                 "categorization_total_records": cat_count,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
144:                 "categorization_unique_upcs": total_cat_upcs,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
145:                 "matching_upcs_in_both": total_matched_count,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
146:                 "missing_from_master_total": missing_from_master_total,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
147:                 "match_rate_pct": round((total_matched_count / total_cat_upcs * 100), 2) if total_cat_upcs else 0,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
148:                 "sample_analyzed": len(records),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
149:                 "exact_matches_in_sample": exact_matches,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
150:                 "partial_matches_in_sample": partial_matches,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
151:                 "mismatches_in_sample": brand_mismatches,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
152:                 "missing_in_sample": len(missing_rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
153:             },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
154:             "records": records
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
155:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
156:
```
Blank line: code ko readable banane ke liye.
```
157:     def compare_allbrand_vs_supplier(self, limit=500):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
158:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
159:         Compare AllBrand / AIMaster with Supplier master sheet ($Supplier_202610291521.csv).
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
160:         Checks for supplier name mismatches, missing suppliers, and inactive/orphan codes.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
161:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
162:         master_table = CSV_FILES['ai_master']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
163:         supplier_table = CSV_FILES['suppliers']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
164:
```
Blank line: code ko readable banane ke liye.
```
165:         # Join on SupplierCode
```
Comment: developer ka note; Python ise execute nahi karta.
```
166:         sql = f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
167:         SELECT
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
168:             m.SupplierCode AS supplier_code,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
169:             m.SupplierName AS master_supplier_name,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
170:             m.BrandName AS brand_name,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
171:             m.source_file AS master_file,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
172:             m.original_row AS master_row,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
173:             s.SupplierName AS official_supplier_name,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
174:             s.RecordStatusId AS supplier_status,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
175:             s.source_file AS supplier_file,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
176:             s.original_row AS supplier_row
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
177:         FROM `{master_table}` m
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
178:         LEFT JOIN `{supplier_table}` s ON m.SupplierCode = s.SupplierCode
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
179:         WHERE m.SupplierCode IS NOT NULL AND m.SupplierCode != ''
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
180:         LIMIT {limit}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
181:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
182:         rows = self.db.query(sql)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
183:
```
Blank line: code ko readable banane ke liye.
```
184:         records = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
185:         exact = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
186:         mismatch = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
187:         missing = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
188:
```
Blank line: code ko readable banane ke liye.
```
189:         for r in rows:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
190:             m_supp = (r.get('master_supplier_name') or '').strip().upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
191:             o_supp = (r.get('official_supplier_name') or '').strip().upper()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
192:
```
Blank line: code ko readable banane ke liye.
```
193:             if not o_supp:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
194:                 status = "Missing Record"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
195:                 reason = f"SupplierCode {r.get('supplier_code')} in Brand sheet not found in Supplier master"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
196:                 missing += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
197:             elif m_supp == o_supp:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
198:                 status = "Exact Match"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
199:                 reason = "Supplier name matches official master"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
200:                 exact += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
201:             else:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
202:                 status = "Mismatch"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
203:                 reason = f"Supplier name discrepancy: Brand='{r.get('master_supplier_name')}' vs
Master='{r.get('official_supplier_name')}'"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
204:                 mismatch += 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
205:
```
Blank line: code ko readable banane ke liye.
```
206:             records.append({
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
207:                 "supplier_code": r.get('supplier_code'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
208:                 "status": status,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
209:                 "reason": reason,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
210:                 "brand_name": r.get('brand_name'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
211:                 "brand_supplier_name": r.get('master_supplier_name'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
212:                 "official_supplier_name": r.get('official_supplier_name') or "MISSING",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
213:                 "master_file": r.get('master_file'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
214:                 "master_row": r.get('master_row'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
215:                 "supplier_file": r.get('supplier_file') or "$Supplier_202610291521.csv",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
216:                 "supplier_row": r.get('supplier_row') or "N/A"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
217:             })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
218:
```

Blank line: code ko readable banane ke liye.

```
219:         return {
```

Return: calculated result caller ko wapas bhejta hai.

```
220:             "comparison_type": "AllBrand vs Supplier Sheet",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
221:             "summary": {
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
222:                 "records_analyzed": len(records),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
223:                 "exact_matches": exact,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
224:                 "mismatches": mismatch,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
225:                 "missing_in_supplier_sheet": missing
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
226:             },
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
227:             "records": records
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
228:         }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
229:
```

Blank line: code ko readable banane ke liye.

```
230:     def compare_beer_vs_rtd(self, limit=500):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
231:         """
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
232:         Compare Beer vs RTD categorization.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
233:         Flags products that cross both categories and checks for classification discrepancies.
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
234:         """
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
235:         beer_table = CSV_FILES['cat_beer']['table']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
236:         rtd_table = CSV_FILES['cat_rtd']['table']
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
237:
```

Blank line: code ko readable banane ke liye.

```
238:         sql = f"""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
239:         SELECT
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
240:             b.clean_upc AS upc,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
241:             b.Beer_Brands_Value AS beer_brand,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
242:             b.Beer_Category_Value AS beer_category,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
243:             b.Beer_Segments_Value AS beer_segment,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
244:             b.source_file AS beer_file,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
245:             b.original_row AS beer_row,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
246:             r.Brands_Value AS rtd_brand,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
247:             r.RTD_Category_Value AS rtd_category,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
248:             r.RTD_Alcohol_Base_Value AS rtd_alcohol_base,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
249:             r.RTD_Segment_Value AS rtd_segment,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
250:             r.source_file AS rtd_file,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
251:             r.original_row AS rtd_row
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
252:         FROM `{beer_table}` b
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
253:         JOIN `{rtd_table}` r ON b.clean_upc = r.clean_upc
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
254:         WHERE b.clean_upc != ''
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
255:         LIMIT {limit}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
256:         """
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
257:         rows = self.db.query(sql)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
258:
```

Blank line: code ko readable banane ke liye.

```
259:         records = []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
260:         for r in rows:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
261:             rtd_cat = (r.get('rtd_category') or '').strip().upper()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
262:             rtd_base = (r.get('rtd_alcohol_base') or '').strip().upper()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
263:
```

Blank line: code ko readable banane ke liye.

```
264:             if 'NON RTD' in rtd_cat or 'NON RTD' in rtd_base:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
265:                 status = "Exact Match"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
266:                 reason = "Correctly categorized as Beer and flagged as NON RTD in RTD dataset"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
267:             elif 'FAB' in rtd_cat or 'SELTZER' in rtd_base or 'MALT' in rtd_base:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
268:                 status = "Requires Verification"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
269:                 reason = f"Dual Classification: Listed as '{r.get('beer_category')}' in Beer but
'{r.get('rtd_category')}' in RTD"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
270:             else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
271:                 status = "Mismatch"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
272:                 reason = f"Category conflict: Beer='{r.get('beer_category')}' vs RTD='{r.get('rtd_category')}'"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
273:
```

Blank line: code ko readable banane ke liye.

```
274:             records.append({
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
275:                 "upc": r.get('upc'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
276:                 "status": status,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
277:                 "reason": reason,
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
278:                 "beer_brand": r.get('beer_brand'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
279:                 "beer_category": r.get('beer_category'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
280:                 "beer_segment": r.get('beer_segment'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
281:                 "rtd_brand": r.get('rtd_brand'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
282:                 "rtd_category": r.get('rtd_category'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
283:                 "rtd_base": r.get('rtd_alcohol_base'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
284:                 "beer_file": r.get('beer_file'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
285:                 "beer_row": r.get('beer_row'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
286:                 "rtd_file": r.get('rtd_file'),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
287:                 "rtd_row": r.get('rtd_row')
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
288:             })
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
289:
```

Blank line: code ko readable banane ke liye.

```
290:         return {
```

Return: calculated result caller ko wapas bhejta hai.

```
291:             "comparison_type": "Beer vs RTD Categorization Validation",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
292:             "summary": {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
293:                 "overlapping_sample": len(records),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
294:                 "verified_non_rtd": sum(1 for x in records if x['status'] == 'Exact Match'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
295:                 "requires_verification": sum(1 for x in records if x['status'] == 'Requires Verification'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
296:                 "mismatches": sum(1 for x in records if x['status'] == 'Mismatch')
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
297:             },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
298:             "records": records
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
299:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
300:
```
Blank line: code ko readable banane ke liye.
```
301:     def find_duplicate_upcs_across_all(self, limit=100):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
302:         """Find UPCs appearing in multiple distinct datasets."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
303:         # Query distinct UPC count across tables
```
Comment: developer ka note; Python ise execute nahi karta.
```
304:         union_sql = f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
305:             SELECT clean_upc, source_file, original_row, norm_brand FROM `{CSV_FILES['ai_master']['table']}` WHERE clean_upc != ''
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
306:         UNION ALL
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
307:             SELECT clean_upc, source_file, original_row, norm_brand FROM `{CSV_FILES['cat_beer']['table']}` WHERE clean_upc != ''
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
308:         UNION ALL
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
309:             SELECT clean_upc, source_file, original_row, norm_brand FROM `{CSV_FILES['cat_rtd']['table']}` WHERE clean_upc != ''
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
310:         UNION ALL
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
311:             SELECT clean_upc, source_file, original_row, norm_brand FROM `{CSV_FILES['cat_spirits']['table']}` WHERE clean_upc != ''
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
312:         UNION ALL
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
313:             SELECT clean_upc, source_file, original_row, norm_brand FROM `{CSV_FILES['cat_wine']['table']}` WHERE clean_upc != ''
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
314:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
315:
```
Blank line: code ko readable banane ke liye.
```
316:         dup_query = f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
317:         WITH all_records AS ({union_sql})
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
318:             SELECT clean_upc, COUNT(DISTINCT source_file) AS file_count, GROUP_CONCAT(DISTINCT source_file) AS files, GROUP_CONCAT(DISTINCT norm_brand) AS brands, COUNT(*) as total_occurrences
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
319:         FROM all_records
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
320:         GROUP BY clean_upc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
321:         HAVING file_count > 1
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
322:         ORDER BY total_occurrences DESC
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
323:         LIMIT {limit}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
324:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
325:         dups = self.db.query(dup_query)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
326:
```
Blank line: code ko readable banane ke liye.
```
327:         results = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
328:         for d in dups:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
329:             results.append({
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
330:                 "upc": d.get('clean_upc'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
331:                 "status": "Duplicate Record",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
332:                 "file_count": d.get('file_count'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
333:                 "files_present": d.get('files'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
334:                 "brands": d.get('brands'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
335:                 "total_occurrences": d.get('total_occurrences'),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
336:                 "reason": f"UPC appears across {d.get('file_count')} different files"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
337:             })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
338:
```
Blank line: code ko readable banane ke liye.
```
339:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
340:             "comparison_type": "Cross-File Duplicate UPC Audit",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
341:             "summary": {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
342:                 "duplicates_found_in_sample": len(results)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
343:             },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
344:             "records": results
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
345:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
346:
```
Blank line: code ko readable banane ke liye.
```
347:     def compare_all_missing_in_master(self, limit_per_category=200, search=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
348:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
349:         Compare all 4 categorization sheets (Beer, Wine, RTD, Spirits) against AIMasterBrandData.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
350:         Identifies all UPCs that exist in categorization sheets but are missing from Master Brands.
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
351:         """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
352:         categories = [
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
353:             ("Beer", CSV_FILES['cat_beer']['table'], "Beer_Brands_Value", "Product", "Categorization 2026 -
Beer.csv"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
354:             ("Wine", CSV_FILES['cat_wine']['table'], "Brands_Value", "Product", "Categorization 2026 - Wine.csv"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
355:             ("RTD", CSV_FILES['cat_rtd']['table'], "Brands_Value", "Product", "Categorization 2026 - RTDs.csv"),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
356:             ("Spirits", CSV_FILES['cat_spirits']['table'], "Spirits_Brands_Value", "Product", "Categorization 2026
- Spirits.csv")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
357:         ]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
358:
```
Blank line: code ko readable banane ke liye.
```
359:         master_table = CSV_FILES['ai_master']['table']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
360:         category_summaries = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
361:         all_missing_records = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
362:         total_unique_cat_upcs = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
363:         total_matched_upcs = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
364:         total_missing_upcs = 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
365:
```
Blank line: code ko readable banane ke liye.
```
366:         for cat_name, tbl, brand_col, prod_col, fname in categories:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
367:             total_cat = self.db.query(f"SELECT COUNT(DISTINCT clean_upc) as c FROM `{tbl}` WHERE clean_upc !=
''")[0]['c']
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
368:             matched_count = self.db.query(f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
369:                 SELECT COUNT(DISTINCT c.clean_upc) as c
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
370:                 FROM `{tbl}` c
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
371:                 JOIN `{master_table}` m ON c.clean_upc = m.clean_upc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
372:                 WHERE c.clean_upc != ''
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
373:             """)[0]['c']
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
374:             missing_count = total_cat - matched_count
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
375:             pct_missing = round((missing_count / total_cat * 100), 1) if total_cat else 0
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
376:
```
Blank line: code ko readable banane ke liye.
```
377:             total_unique_cat_upcs += total_cat
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
378:             total_matched_upcs += matched_count
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
379:             total_missing_upcs += missing_count
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
380:
```
Blank line: code ko readable banane ke liye.
```
381:             category_summaries.append({
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
382:                 "category": cat_name,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
383:                 "total_upcs": total_cat,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
384:                 "matched_in_master": matched_count,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
385:                 "missing_from_master": missing_count,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
386:                 "pct_missing": f"{pct_missing}%",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
387:                 "source_file": fname
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
388:             })
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
389:
```
Blank line: code ko readable banane ke liye.
```
390:             # Retrieve sample missing records
```
Comment: developer ka note; Python ise execute nahi karta.
```
391:             search_clause = ""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
392:             search_params = ()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
393:             if search:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
394:                 search_clause = f" AND (LOWER(COALESCE(c.{brand_col},'')) LIKE ? OR
LOWER(COALESCE(c.{prod_col},'')) LIKE ? OR c.clean_upc LIKE ?)"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
395:                 needle = f"%{search.lower()}%"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
396:                 search_params = (needle, needle, f"%{search}%")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
397:             missing_sql = f"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
398:             SELECT
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
399:                 '{cat_name}' AS category,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
400:                 c.clean_upc AS upc,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
401:                 COALESCE(c.{brand_col}, 'N/A') AS brand,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
402:                 COALESCE(c.{prod_col}, 'N/A') AS product,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
403:                 c.source_file AS source_file,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
404:                 c.original_row AS original_row,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
405:                 'Missing Record' AS status,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
406:                 'UPC exists in {fname} but is missing from Master Brands' AS reason
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
407:             FROM `{tbl}` c
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
408:             LEFT JOIN `{master_table}` m ON c.clean_upc = m.clean_upc
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
409:             WHERE m.clean_upc IS NULL AND c.clean_upc != ''{search_clause}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
410:             LIMIT {limit_per_category}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
411:             """
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
412:             rows = self.db.query(missing_sql, search_params)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
413:             all_missing_records.extend(rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
414:
```
Blank line: code ko readable banane ke liye.
```
415:         return {
```
Return: calculated result caller ko wapas bhejta hai.
```
416:             "comparison_type": "Missing UPC Audit: Beer, Wine, RTD & Spirits vs Master",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
417:             "summary": {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
418:                 "categories": category_summaries,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
419:                 "total_categorization_upcs": total_unique_cat_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
420:                 "total_matched_in_master": total_matched_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
421:                 "total_missing_from_master": total_missing_upcs,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
422:                         "overall_missing_pct": round((total_missing_upcs / total_unique_cat_upcs * 100), 1) if
total_unique_cat_upcs else 0,
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
423:                 "sample_missing_records": len(all_missing_records)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
424:             },
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
425:             "records": all_missing_records
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
426:         }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
427:
```
Blank line: code ko readable banane ke liye.
```
428: comparison_engine = ComparisonEngine()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

File: engines/upc_match_engine.py

Pehle source line padho, phir uski explanation. Blank lines bhi include hain taaki original numbering follow ho.

```
 1: """Read-only, UPC-first product matching for category audits."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
 2:
```
Blank line: code ko readable banane ke liye.
```
 3: import re
```
Import: library/module ke functions yahan available karata hai.
```
 4:
```
Blank line: code ko readable banane ke liye.
```
 5: from config import CSV_FILES
```
Import: library/module ke functions yahan available karata hai.
```
 6: from database.db_manager import db_manager
```
Import: library/module ke functions yahan available karata hai.
```
 7:
```
Blank line: code ko readable banane ke liye.
```
 8: try:
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
 9:     from rapidfuzz.fuzz import token_set_ratio
```
Import: library/module ke functions yahan available karata hai.
```
10: except ImportError:  # pragma: no cover
```
Error handling: failure ko handle karke app ko crash hone se bachata hai.
```
11:     from difflib import SequenceMatcher
```
Import: library/module ke functions yahan available karata hai.
```
12:
```
Blank line: code ko readable banane ke liye.
```
13:     def token_set_ratio(a, b):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
14:         return int(SequenceMatcher(None, a, b).ratio() * 100)
```
Return: calculated result caller ko wapas bhejta hai.
```
15:
```
Blank line: code ko readable banane ke liye.
```
16:
```
Blank line: code ko readable banane ke liye.
```
17: class UPCMatchEngine:
```
Class: related data aur methods ka reusable blueprint.
```
18:     """Compare category rows with AllBrand without changing any source data."""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
19:
```
Blank line: code ko readable banane ke liye.
```
20:     fields = {
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
21:             "cat_beer": {"brand": "Beer_Brands_Value", "size": "Beer_Size_Groups_Value", "product": "Product",
"category": "Beer_Category_Value", "flavour": "Beer_Mezzo_Styles_Value"},
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
22:             "cat_rtd": {"brand": "Brands_Value", "size": "RTD_Size_Value", "product": "Product", "category":
"RTD_Category_Value", "flavour": "Flavor_Groups_Value"},
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
23:           "cat_spirits": {"brand": "Spirits_Brands_Value", "size": "Spirits_Sizes_Value", "product": "Product",
"category": "Spirits_Category_Value", "flavour": "Spirits_Flavors_Value"},
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
24:          "cat_wine": {"brand": "Brands_Value", "size": "Size_Packages_Value", "product": "Product", "category":
"Categories_Value", "flavour": "Flavors_Value"},
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
25:     }
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
                        26:                                                                 size_pattern            =
re.compile(r"(\d+(?:\.\d+)?)\s*(oz|ounce|ounces|ml|lt|l|liter|litre|gal|gallon|pack|pk|ct|count)\b", re.IGNORECASE)
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
27:
```
Blank line: code ko readable banane ke liye.
```
28:     def _norm(self, value):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
29:         return re.sub(r"[^a-z0-9]+", " ", str(value or "").lower()).strip()
```
Return: calculated result caller ko wapas bhejta hai.
```
30:
```
Blank line: code ko readable banane ke liye.
```
31:     def _size_signature(self, value):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
32:         signature = set()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
33:         for number, unit in self.size_pattern.findall(str(value or "")):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
34:             amount = float(number)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
35:             unit = unit.lower()
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
36:             if unit in {"l", "lt", "liter", "litre"}:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
37:                 amount, unit = amount * 1000, "ml"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
38:             elif unit in {"ounce", "ounces"}:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
39:                 unit = "oz"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
40:             elif unit in {"pk", "ct", "count"}:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
41:                 unit = "pack"
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
42:             signature.add((round(amount, 3), unit))
```
Regex/text cleaning: UPC ya words se pattern extract karta hai.
```
43:         return signature
```
Return: calculated result caller ko wapas bhejta hai.
```
44:
```
Blank line: code ko readable banane ke liye.
```
45:     def _similarity(self, left, right):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
46:         left, right = self._norm(left), self._norm(right)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
47:         return int(token_set_ratio(left, right)) if left and right else 0
```
Return: calculated result caller ko wapas bhejta hai.
```
48:
```
Blank line: code ko readable banane ke liye.
```
49:     def _size_status(self, source_size, master_size):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
50:         source, master = self._size_signature(source_size), self._size_signature(master_size)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
51:         if source and master and source.isdisjoint(master):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
52:             return "conflict", 0
```
Return: calculated result caller ko wapas bhejta hai.
```
53:         if not source or not master:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
54:             return "unknown", 60
```
Return: calculated result caller ko wapas bhejta hai.
```
55:         return "compatible", self._similarity(source_size, master_size)
```
Return: calculated result caller ko wapas bhejta hai.
```
56:
```
Blank line: code ko readable banane ke liye.
```
57:     def _category_rows(self, table, limit):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
58:         if limit is None:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
59:             return db_manager.query(f"SELECT * FROM `{table}` ORDER BY original_row")
```
Return: calculated result caller ko wapas bhejta hai.
```
60:         return db_manager.query(f"SELECT * FROM `{table}` ORDER BY original_row LIMIT ?", (int(limit),))
```
Return: calculated result caller ko wapas bhejta hai.
```
61:
```
Blank line: code ko readable banane ke liye.
```
62:     def _load_master_index(self):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
63:         rows = db_manager.query(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
64:                 """SELECT source_file, original_row, clean_upc, clean_alt_upc, BrandName, ProductName, Size,
SupplierName
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
65:                FROM all_brands WHERE (clean_upc != '' OR clean_alt_upc != '')"""
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
66:         )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
67:         index = {}
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
68:         for row in rows:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
69:             key = self._norm(row.get("BrandName"))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
70:             if key:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
71:                 index.setdefault(key, []).append(row)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
72:         return index
```
Return: calculated result caller ko wapas bhejta hai.
```
73:
```
Blank line: code ko readable banane ke liye.
```
74:     def _master_candidates(self, brand, cache=None, master_index=None):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
75:         cache_key = self._norm(brand)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
76:         if cache is not None and cache_key in cache:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
77:             return cache[cache_key]
```
Return: calculated result caller ko wapas bhejta hai.
```
78:         if master_index is not None:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
79:             candidates = []
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
80:             for master_brand, rows in master_index.items():
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
81:                 if cache_key in master_brand or master_brand in cache_key:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
82:                     candidates.extend(rows)
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
83:                     if len(candidates) >= 120:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
84:                         break
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
85:             candidates = candidates[:120]
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
86:             if cache is not None:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
87:                 cache[cache_key] = candidates
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
88:             return candidates
```
Return: calculated result caller ko wapas bhejta hai.
```
89:         candidates = db_manager.query(
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
90:                 """SELECT source_file, original_row, clean_upc, clean_alt_upc, BrandName, ProductName, Size,
SupplierName
```
SQL: database se data read/write ya schema/index tayyar hota hai.
```
91:                 FROM all_brands WHERE norm_brand LIKE ? AND (clean_upc != '' OR clean_alt_upc != '') LIMIT 120""",
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
92:             (f"%{self._norm(brand)}%",),
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
93:         )
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
94:         if cache is not None:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
95:             cache[cache_key] = candidates
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
96:         return candidates
```
Return: calculated result caller ko wapas bhejta hai.
```
97:
```
Blank line: code ko readable banane ke liye.
```
98:     def _best_candidate(self, row, spec, candidates):
```
Function: reusable kaam ka block; call hone par body run hoti hai.
```
99:         product = row.get(spec["product"], "")
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
100:         if not self._norm(product):
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
101:             return None
```
Return: calculated result caller ko wapas bhejta hai.
```
102:             source_product = " ".join(self._norm(row.get(spec[key], "")) for key in ("product", "flavour",
"category"))
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
103:         best = None
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
104:         for candidate in candidates:
```
Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.
```
105:                         product_score = self._similarity(source_product, f"{candidate.get('ProductName', '')}
{candidate.get('BrandName', '')}")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
106:             size_status, size_score = self._size_status(row.get(spec["size"]), candidate.get("Size"))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
107:             brand_score = self._similarity(row.get(spec["brand"]), candidate.get("BrandName"))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
108:             if size_status == "conflict" or product_score < 75 or brand_score < 70:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
109:                 continue
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
110:             # Equivalent normalized sizes (for example 0.75 LT and 750 ML)
```

Comment: developer ka note; Python ise execute nahi karta.

```
111:             # should receive full size confidence even when their raw text differs.
```

Comment: developer ka note; Python ise execute nahi karta.

```
112:                         if size_status == "compatible" and self._size_signature(row.get(spec["size"])) ==
self._size_signature(candidate.get("Size")):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
113:                 size_score = 100
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
114:             score = round((product_score * 0.55) + (max(size_score, 70) * 0.30) + (brand_score * 0.15))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
115:             if best is None or score > best["score"]:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
116:                 best = {"score": int(score), "description_score": int(product_score), "candidate": candidate}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
117:         return best if best and best["score"] >= 80 else None
```

Return: calculated result caller ko wapas bhejta hai.

```
118:
```

Blank line: code ko readable banane ke liye.

```
119:         def _record(self, category, row, spec, candidate=None, status=None, confidence=0, remarks="",
description_score=0):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
120:         candidate = candidate or {}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
121:         source_description = " ".join(str(row.get(spec[key]) or "").strip() for key in ("product", "flavour",
"category") if str(row.get(spec[key]) or "").strip())
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
122:         master_description = str(candidate.get("ProductName") or "").strip()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
123:         return {
```

Return: calculated result caller ko wapas bhejta hai.

```
124:             "category": category.replace("cat_", "").upper(), "status": status or "Missing in AllBrand - No
Product Match",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
125:             "current_upc": str(row.get("clean_upc") or ""), "suggested_upc": str(candidate.get("clean_upc") or
""), "alternate_upc": str(candidate.get("clean_alt_upc") or ""), "confidence": int(confidence),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
126:                 "source_file": row.get("source_file"), "source_row": row.get("original_row"), "source_brand":
row.get(spec["brand"]), "source_product": row.get(spec["product"]), "source_flavour": row.get(spec["flavour"]), "s
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
127:             "master_brand": candidate.get("BrandName", ""), "master_product": candidate.get("ProductName", ""),
"master_size": candidate.get("Size", ""), "master_row": candidate.get("original_row", ""),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
128:                         "source_description": source_description, "master_description": master_description,
"description_match_score": int(description_score),
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
129:             "remarks": remarks or "Review only; no master product or UPC was changed.",
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
130:         }
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
131:
```

Blank line: code ko readable banane ke liye.

```
132:     def audit(self, category=None, limit=100):
```

Function: reusable kaam ka block; call hone par body run hoti hai.

```
133:         categories = [category] if category in self.fields else list(self.fields)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
134:         output = []
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
135:         candidate_cache = {}
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
136:         master_index = self._load_master_index()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
137:         for cat_key in categories:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
138:             spec = self.fields[cat_key]
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 139:             for row in self._category_rows(CSV_FILES[cat_key]["table"], limit):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 140:                 current_upc = str(row.get("clean_upc") or "")
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 141:                 if not self._norm(row.get(spec["brand"], "")):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 142:                     output.append(self._record(cat_key, row, spec, status="Manual Review - Insufficient Product
Match"))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 143:                     continue
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 144:                 candidates = self._master_candidates(row.get(spec["brand"], ""), candidate_cache, master_index)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 145:                 exact = next((c for c in candidates if current_upc and current_upc in {str(c.get("clean_upc") or
""), str(c.get("clean_alt_upc") or "")}), None)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 146:                 if exact:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 147:                     verified = self._record(cat_key, row, spec, exact, "Verified UPC in AllBrand", 100, "Exact UPC
or known alternate UPC found in AllBrand.", 100)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 148:                     verified["suggested_upc"] = ""
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 149:                     output.append(verified)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 150:                     continue
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 151:                 best = self._best_candidate(row, spec, candidates)
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 152:                 if best:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 153:                     status = "Product Match - UPC Missing" if not current_upc else "Product Match - UPC Different"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 154:                         output.append(self._record(cat_key, row, spec, best["candidate"], status, best["score"],
"Brand, product/flavour, and size/pack are compatible; review suggested UPC before manual update.", best["descr
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 155:                     continue
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 156:                 if not self._norm(row.get(spec["product"], "")):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 157:                     status = "Manual Review - Insufficient Product Match"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 158:                     elif any(self._size_status(row.get(spec["size"]), c.get("Size"))[0] == "conflict" for c in
candidates):
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 159:                     status = "Manual Review - Size/Pack Conflict"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 160:                 else:
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 161:                     status = "Missing in AllBrand - No Product Match"
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 162:                 output.append(self._record(cat_key, row, spec, status=status))
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

```
 163:         return output
```

Return: calculated result caller ko wapas bhejta hai.

```
 164:
```

Blank line: code ko readable banane ke liye.

```
 165:
```

Blank line: code ko readable banane ke liye.

```
 166: upc_match_engine = UPCMatchEngine()
```

Data flow, configuration, calculation, ya object state update; exact intent surrounding function se samjho.

4. Tools, frontend aur tests ka purpose

tools/inspect_data.py delimiter, columns aur row count inspect karta hai. deep_data_analysis.py UPC duplicates/scientific notation/overlap nikalta hai. beer_rtd_check.py cross-category overlap dekhta hai. supplier_check.py supplier code integrity check karta hai. full_profile.py dataset profiling JSON banata hai. missing_upc_matcher.py aur v2/v3/v4 conservative fuzzy product matching aur reports ke iterations hain. generate_project_pdf scripts documentation PDF banate hain. tests/ endpoint, RAG routing, UPC matching, cached Ollama status aur read-only behavior verify karte hain.

templates/index.html browser UI hai: chat form FormData bhejta hai, comparison buttons API call karte hain, returned records ko table/status badge/CSV link mein render karta hai. Frontend backend ki business logic nahi karta; backend result ka presentation karta hai.

5. Complete function and class catalog

Yeh quick revision index hai. Project ke Python files se saare classes aur functions nikale gaye hain. Detailed source line explanation upar file sections mein hai.

app.py

Line 22: index
Purpose / Why: Root browser page serve karta hai. render_template() index.html ko config values ke saath HTML response banata hai.
Line 31: api_chat
Purpose / Why: Main chat endpoint. Form/JSON message aur optional file leta hai, document_rag.ingest() se file store karta hai, phir rag_orchestrator.route_and_execute() call karke JSON answer deta hai.
Line 58: api_documents
Purpose / Why: Current browser session ke uploaded documents list karta hai; chat UI ko document history dene ke liye.
Line 62: api_upc_audit
Purpose / Why: Read-only UPC audit chalata hai, records ko CSV report mein save karta hai aur download token return karta hai.
Line 82: api_compare
Purpose / Why: Comparison type ke basis par comparison_engine ka correct method select karta hai; result ko CSV export token ke saath JSON banata hai.
Line 121: api_stats
Purpose / Why: Har dataset ka row count aur unique UPC count nikal kar dashboard metrics return karta hai.
Line 135: api_dashboard_overview
Purpose / Why: Api dashboard overview: reusable operation.
Line 154: api_dashboard_metrics
Purpose / Why: Return live category, UPC quality, and Master Brand dashboard metrics.
Line 165: api_master_brand_search
Purpose / Why: Api master brand search: reusable operation.
Line 186: api_master_brand_export
Purpose / Why: Api master brand export: reusable operation.
Line 199: api_dashboard_categories
Purpose / Why: Api dashboard categories: reusable operation.
Line 206: api_dashboard_missing_upc
Purpose / Why: Api dashboard missing upc: reusable operation.
Line 218: api_dashboard_export
Purpose / Why: Api dashboard export: reusable operation.
Line 235: api_db_status
Purpose / Why: MySQL connectivity aur Ollama availability check karke UI status badges ke liye data return karta hai.
Line 248: api_configure_db
Purpose / Why: User ke MySQL password se connection test karta hai; optional migration start karta hai.
Line 270: api_rebuild_index
Purpose / Why: SQLite records se persistent local vector/BM25 index dobara build karta hai.
Line 278: api_export_csv
Purpose / Why: Generated report token validate karke CSV file browser ko download karata hai.

run.py

Line 8: setup_environment
Purpose / Why: Startup bootstrap: SQLite missing ho to ingestion chalata hai, vector index missing ho to build karta hai, phir Flask server ready karta hai.

document_rag.py

Line 25: _connection
Purpose / Why: Document metadata aur chunks ke liye SQLite connection/table/index create karta hai.
Line 49: _clean
Purpose / Why: clean: reusable operation.
Line 53: _extract
Purpose / Why: PDF/DOCX/TXT/CSV/XLSX/image extension ke hisaab se text extract karta hai.
Line 90: validate_upload
Purpose / Why: Validate upload name, extension, and size before writing it to disk.
Line 107: _chunks
Purpose / Why: Long text ko overlapping chunks mein todta hai taaki retrieval aur LLM context manageable rahe.
Line 119: ingest
Purpose / Why: Uploaded file validate/save, text extraction/chunking, hash generation aur document chunks database insert karta hai.
Line 156: _tokens
Purpose / Why: tokens: reusable operation.

Line 160: search
Purpose / Why: Session ke chunks ko query-token overlap score se rank karke top relevant document context return karta hai.
Line 190: list_documents
Purpose / Why: List documents: reusable operation.

data_ingestion/normalizer.py

Line 4: normalize_upc
Purpose / Why: Scientific notation, UPC= prefix, decimal aur missing leading zero clean karke UPC ko string rakhta hai.
Line 49: clean_text
Purpose / Why: Null values aur extra whitespace ko clean karta hai.

data_ingestion/loader.py

Line 13: clean_column_name
Purpose / Why: Normalize CSV column names for safe SQL table column names.
Line 22: ingest_csv_to_sqlite
Purpose / Why: CSV ko chunks mein read, normalize, source row metadata add aur indexed SQLite table mein append karta hai.
Line 119: ingest_all_to_sqlite
Purpose / Why: Config registry ke saare seven CSV datasets sequentially ingest karta hai.

database/db_manager.py

Line 10: DBManager
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 11: DBManager -> __init__
Purpose / Why: init : reusable operation.
Line 19: DBManager -> set_mysql_backend
Purpose / Why: Select MySQL as primary or return to the local SQLite fallback.
Line 24: DBManager -> backend_name
Purpose / Why: Return the backend currently selected for application queries.
Line 28: DBManager -> test_mysql_connection
Purpose / Why: Test if MySQL server on port 3307 is reachable and authenticates.
Line 50: DBManager -> get_mysql_connection
Purpose / Why: Obtain PyMySQL connection to MySQL 3307.
Line 75: DBManager -> get_sqlite_connection
Purpose / Why: SQLite WAL, cache aur performance pragmas ke saath local connection banata hai.
Line 85: DBManager -> table_columns
Purpose / Why: Return column names for a configured table on the active backend.
Line 93: DBManager -> mysql_schema_status
Purpose / Why: Check that MySQL has the normalized tables this application queries.
Line 112: DBManager -> database_status
Purpose / Why: Return safe connection state without exposing credentials.
Line 140: DBManager -> get_connection
Purpose / Why: MySQL available ho to MySQL, warna SQLite fallback choose karta hai.
Line 149: DBManager -> query
Purpose / Why: SQL execute karke rows ko dictionaries ki list mein return karta hai; engines ka common database interface.
Line 175: DBManager -> query_df
Purpose / Why: SQL result ko Pandas DataFrame mein return karta hai.

database/mysql_migrator.py

Line 12: migrate_data_to_mysql
Purpose / Why: Ingests all beverage CSV files directly into MySQL server on port 3307.

engines/sql_engine.py

Line 10: SQLEngine
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 11: SQLEngine -> __init__
Purpose / Why: init : reusable operation.
Line 14: SQLEngine -> lookup_upc
Purpose / Why: Normalized UPC ko AIMaster, AllBrand, Beer, RTD, Spirits aur Wine tables mein exact/alternate match karta hai.
Line 84: SQLEngine -> search_brand
Purpose / Why: Search products by brand name across master and categorization datasets.
Line 101: SQLEngine -> search_master_brands
Purpose / Why: Search the Master Brand table using real columns and exact identifiers.
Line 154: SQLEngine -> lookup_identifier
Purpose / Why: Return complete source rows for an exact ID, UPC, or alternate UPC.
Line 212: SQLEngine -> search_catalog
Purpose / Why: Brand/product text ko sabhi catalog sheets mein search karke common result shape banata hai.

Line 256: SQLEngine -> brand_profile
Purpose / Why: Ek brand ke flavours, sizes, suppliers, origins, categories aur UPC summary banata hai.
Line 321: SQLEngine -> brand_profile -> values
Purpose / Why: Values: reusable operation.
Line 337: SQLEngine -> search_supplier
Purpose / Why: Supplier name ke basis par AIMaster products search karta hai.
Line 354: SQLEngine -> count_master_brands
Purpose / Why: Return the distinct normalized brand count from the master catalog.
Line 358: SQLEngine -> count_brands
Purpose / Why: Requested table mein distinct normalized brands ka exact SQL count deta hai.
Line 367: SQLEngine -> count_by_category
Purpose / Why: Master, Beer aur RTD categories ka GROUP BY aggregation deta hai.
Line 399: SQLEngine -> get_dataset_overview
Purpose / Why: Har configured table ka total row aur unique UPC metric return karta hai.
Line 419: SQLEngine -> get_dashboard_metrics
Purpose / Why: Return live category, UPC-quality, and Master Brand dashboard metrics.
Line 467: SQLEngine -> _quality_summary
Purpose / Why: quality summary: reusable operation.
Line 482: SQLEngine -> _table_upc_metrics
Purpose / Why: table upc metrics: reusable operation.

engines/vector_engine.py

Line 9: LocalVectorIndex
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 14: LocalVectorIndex -> __init__
Purpose / Why: init : reusable operation.
Line 25: LocalVectorIndex -> tokenize
Purpose / Why: Search text ko lowercase alphanumeric tokens mein convert karta hai.
Line 31: LocalVectorIndex -> build_from_database
Purpose / Why: Selected database rows ka BM25 document index, metadata, document frequency aur lengths build karta hai.
Line 92: LocalVectorIndex -> save
Purpose / Why: Index state ko pickle file mein persist karta hai.
Line 102: LocalVectorIndex -> load
Purpose / Why: Saved pickle index ko memory mein load karta hai.
Line 114: LocalVectorIndex -> search
Purpose / Why: BM25-style TF/IDF score se top relevant beverage records retrieve karta hai.

engines/rag_orchestrator.py

Line 15: RAGOrchestrator
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 16: RAGOrchestrator -> __init__
Purpose / Why: init : reusable operation.
Line 23: RAGOrchestrator -> is_ollama_available
Purpose / Why: Local Ollama /api/tags endpoint ko short cache ke saath check karta hai.
Line 36: RAGOrchestrator -> query_ollama
Purpose / Why: Prompt, model aur system instruction ko local Ollama /api/generate endpoint par bhejta hai.
Line 52: RAGOrchestrator -> route_and_execute
Purpose / Why: User intent classify karke normal chat, SQL, document RAG, comparison, UPC audit, brand search ya vector retrieval ka correct route select karta hai.
Line 62: RAGOrchestrator -> _route_and_execute
Purpose / Why: Understands natural language questions in English and Hinglish,
Line 564: RAGOrchestrator -> _with_conversation_context
Purpose / Why: Resolve short pronoun-based follow-ups without changing explicit searches.
Line 579: RAGOrchestrator -> _local_general_response
Purpose / Why: Ollama unavailable hone par greetings, help, joke aur basic Hinglish/English replies deta hai.
Line 620: RAGOrchestrator -> _detect_response_language
Purpose / Why: Detect the reply style from the latest user message.
Line 635: RAGOrchestrator -> _is_hinglish
Purpose / Why: Lightweight language markers se Hinglish response choose karta hai.
Line 639: RAGOrchestrator -> _extract_brand_guess
Purpose / Why: Question words hata kar actual multi-word brand name nikalta hai.
Line 653: RAGOrchestrator -> _format_response
Purpose / Why: Answer, records, sources, CSV token aur read-only flag ko frontend-friendly response shape mein pack karta hai.

engines/comparison_engine.py

Line 9: ComparisonEngine
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 10: ComparisonEngine -> __init__
Purpose / Why: init : reusable operation.
Line 13: ComparisonEngine -> compare_master_vs_categorization
Purpose / Why: UPC join karke Exact, Partial, Mismatch aur Missing statuses classify karta hai.
Line 157: ComparisonEngine -> compare_allbrand_vs_supplier
Purpose / Why: SupplierCode join karke official supplier name integrity check karta hai.
Line 230: ComparisonEngine -> compare_beer_vs_rtd
Purpose / Why: Overlapping Beer/RTD UPCs ko NON RTD, FAB/seltzer verification aur mismatch rules se validate karta hai.
Line 301: ComparisonEngine -> find_duplicate_upcs_across_all
Purpose / Why: Multiple source files mein same UPC groups find karta hai.
Line 347: ComparisonEngine -> compare_all_missing_in_master
Purpose / Why: Beer, Wine, RTD aur Spirits ke UPCs ko Master se compare karke missing samples aur summary banata hai.

engines/upc_match_engine.py

Line 17: UPCMatchEngine
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 28: UPCMatchEngine -> _norm
Purpose / Why: norm: reusable operation.
Line 31: UPCMatchEngine -> _size_signature
Purpose / Why: size signature: reusable operation.
Line 45: UPCMatchEngine -> _similarity
Purpose / Why: similarity: reusable operation.
Line 49: UPCMatchEngine -> _size_status
Purpose / Why: size status: reusable operation.
Line 57: UPCMatchEngine -> _category_rows
Purpose / Why: category rows: reusable operation.
Line 62: UPCMatchEngine -> _load_master_index
Purpose / Why: load master index: reusable operation.
Line 74: UPCMatchEngine -> _master_candidates
Purpose / Why: master candidates: reusable operation.
Line 98: UPCMatchEngine -> _best_candidate
Purpose / Why: best candidate: reusable operation.
Line 119: UPCMatchEngine -> _record
Purpose / Why: record: reusable operation.
Line 132: UPCMatchEngine -> audit
Purpose / Why: AllBrand candidates ke against exact UPC, alternate UPC, product/brand/size similarity check karta hai; source data modify nahi karta.

tools\beer_rtd_check.py

Line 4: clean_upc
Purpose / Why: Clean upc: reusable operation.

tools\cross_compare_test.py

Line 5: clean_upc
Purpose / Why: Clean upc: reusable operation.

tools\deep_data_analysis.py

Line 6: normalize_upc
Purpose / Why: Normalize upc: reusable operation.
Line 35: analyze_upcs
Purpose / Why: Analyze upcs: reusable operation.

tools\fix_pdf_v2.py

Line 41: safe_latin1
Purpose / Why: Safe latin1: reusable operation.

tools\full_profile.py

Line 19: clean_upc
Purpose / Why: Clean upc: reusable operation.
Line 39: analyze_dataset
Purpose / Why: Analyze dataset: reusable operation.

tools\generate_function_explanation_pdf.py

Line 85: catalog
Purpose / Why: Catalog: reusable operation.

Line 89: catalog -> visit
Purpose / Why: Visit: reusable operation.
Line 104: Guide
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 105: Guide -> header
Purpose / Why: Header: reusable operation.
Line 112: Guide -> footer
Purpose / Why: Footer: reusable operation.
Line 118: Guide -> cover_title
Purpose / Why: Cover title: reusable operation.
Line 124: Guide -> heading
Purpose / Why: Heading: reusable operation.
Line 130: Guide -> para
Purpose / Why: Para: reusable operation.
Line 136: Guide -> item
Purpose / Why: Item: reusable operation.
Line 146: explanation_for
Purpose / Why: Explanation for: reusable operation.
Line 157: script_summary
Purpose / Why: Script summary: reusable operation.
Line 176: build
Purpose / Why: Build: reusable operation.

tools\generate_project_pdf.py

Line 34: PDF
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 35: PDF -> __init__
Purpose / Why: init : reusable operation.
Line 43: PDF -> header
Purpose / Why: Header: reusable operation.
Line 58: PDF -> footer
Purpose / Why: Footer: reusable operation.
Line 70: PDF -> h1
Purpose / Why: H1: reusable operation.
Line 79: PDF -> h2
Purpose / Why: H2: reusable operation.
Line 89: PDF -> h3
Purpose / Why: H3: reusable operation.
Line 96: PDF -> body
Purpose / Why: Body: reusable operation.
Line 104: PDF -> bullet
Purpose / Why: Bullet: reusable operation.
Line 112: PDF -> numbered
Purpose / Why: Numbered: reusable operation.
Line 120: PDF -> info_box
Purpose / Why: Info box: reusable operation.
Line 132: PDF -> table_row
Purpose / Why: Table row: reusable operation.
Line 152: PDF -> code_block
Purpose / Why: Code block: reusable operation.
Line 165: PDF -> spacer
Purpose / Why: Spacer: reusable operation.
Line 168: PDF -> callout
Purpose / Why: Callout: reusable operation.
Line 185: build_pdf
Purpose / Why: Build pdf: reusable operation.

tools\generate_project_pdf_v2.py

Line 34: PDF
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 35: PDF -> __init__
Purpose / Why: init : reusable operation.
Line 43: PDF -> header
Purpose / Why: Header: reusable operation.

Line 58: PDF -> footer
Purpose / Why: Footer: reusable operation.
Line 70: PDF -> h1
Purpose / Why: H1: reusable operation.
Line 79: PDF -> h2
Purpose / Why: H2: reusable operation.
Line 89: PDF -> h3
Purpose / Why: H3: reusable operation.
Line 96: PDF -> body
Purpose / Why: Body: reusable operation.
Line 104: PDF -> bullet
Purpose / Why: Bullet: reusable operation.
Line 112: PDF -> numbered
Purpose / Why: Numbered: reusable operation.
Line 120: PDF -> info_box
Purpose / Why: Info box: reusable operation.
Line 132: PDF -> table_row
Purpose / Why: Table row: reusable operation.
Line 152: PDF -> code_block
Purpose / Why: Code block: reusable operation.
Line 165: PDF -> spacer
Purpose / Why: Spacer: reusable operation.
Line 168: PDF -> callout
Purpose / Why: Callout: reusable operation.
Line 185: build_pdf
Purpose / Why: Build pdf: reusable operation.

tools\generate_project_pdf_v3.py

Line 34: PDF
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 35: PDF -> __init__
Purpose / Why: init : reusable operation.
Line 43: PDF -> header
Purpose / Why: Header: reusable operation.
Line 58: PDF -> footer
Purpose / Why: Footer: reusable operation.
Line 70: PDF -> h1
Purpose / Why: H1: reusable operation.
Line 79: PDF -> h2
Purpose / Why: H2: reusable operation.
Line 89: PDF -> h3
Purpose / Why: H3: reusable operation.
Line 96: PDF -> body
Purpose / Why: Body: reusable operation.
Line 104: PDF -> bullet
Purpose / Why: Bullet: reusable operation.
Line 112: PDF -> numbered
Purpose / Why: Numbered: reusable operation.
Line 120: PDF -> info_box
Purpose / Why: Info box: reusable operation.
Line 132: PDF -> table_row
Purpose / Why: Table row: reusable operation.
Line 152: PDF -> code_block
Purpose / Why: Code block: reusable operation.
Line 165: PDF -> spacer
Purpose / Why: Spacer: reusable operation.
Line 168: PDF -> callout
Purpose / Why: Callout: reusable operation.
Line 185: build_pdf
Purpose / Why: Build pdf: reusable operation.

tools\inspect_data.py

Line 7: inspect_csv
Purpose / Why: Inspect csv: reusable operation.

tools\missing_upc_matcher.py

Line 52: clean_upc
Purpose / Why: Convert any representation of a UPC to a zero-padded 13-digit string.
Line 97: extract_upc_from_product_text
Purpose / Why: Pull the 13-digit UPC embedded at the end of cat-sheet Product strings.
Line 110: parse_alternate_upc_set
Purpose / Why: Parse AllBrand AlternateUPCCode field which looks like:
Line 134: find_csv
Purpose / Why: Find csv: reusable operation.
Line 142: load_allbrand
Purpose / Why: Load allbrand: reusable operation.
Line 153: load_allbrand -> all_upcs
Purpose / Why: All upcs: reusable operation.
Line 166: load_cat_sheet
Purpose / Why: Load cat sheet: reusable operation.
Line 172: load_cat_sheet -> get_upc
Purpose / Why: Get upc: reusable operation.
Line 191: build_master_upc_index
Purpose / Why: Returns:
Line 216: build_product_key
Purpose / Why: Combine fields into a normalised comparison key.
Line 224: build_master_product_keys
Purpose / Why: Build master product keys: reusable operation.
Line 238: cat_row_product_key
Purpose / Why: Extract brand, size, etc. from cat-sheet row.
Line 243: cat_row_product_key -> pick
Purpose / Why: Pick: reusable operation.
Line 271: fuzzy_match_to_master
Purpose / Why: Find top-N master rows by fuzzy token-sort ratio against cat_key.
Line 295: classify_match
Purpose / Why: Classify match: reusable operation.
Line 310: analyse_sheet
Purpose / Why: Analyse sheet: reusable operation.
Line 466: _style_sheet
Purpose / Why: style sheet: reusable operation.
Line 500: _status_fill_color
Purpose / Why: status fill color: reusable operation.
Line 511: write_excel_report
Purpose / Why: Write excel report: reusable operation.
Line 590: main
Purpose / Why: Main: reusable operation.

tools\missing_upc_matcher_v2.py

Line 53: clean_upc
Purpose / Why: Clean upc: reusable operation.
Line 73: extract_upc_from_product
Purpose / Why: Extract upc from product: reusable operation.
Line 80: parse_alt_upcs
Purpose / Why: Parse alt upcs: reusable operation.
Line 89: norm_str
Purpose / Why: Norm str: reusable operation.
Line 95: brand_token
Purpose / Why: First meaningful word of brand for pre-filtering.
Line 106: find_csv
Purpose / Why: Find csv: reusable operation.
Line 114: load_allbrand
Purpose / Why: Load allbrand: reusable operation.
Line 126: load_cat_sheet
Purpose / Why: Load cat sheet: reusable operation.
Line 131: load_cat_sheet -> get_upc
Purpose / Why: Get upc: reusable operation.
Line 146: build_indexes
Purpose / Why: Returns upc_idx, alt_idx, brand_token_idx, master_keys_list, master_keys_series
Line 192: cat_product_key
Purpose / Why: Cat product key: reusable operation.

Line 193: cat_product_key -> pick
Purpose / Why: Pick: reusable operation.
Line 224: classify
Purpose / Why: Classify: reusable operation.
Line 231: analyse_sheet
Purpose / Why: Analyse sheet: reusable operation.
Line 436: write_excel
Purpose / Why: Write excel: reusable operation.
Line 529: main
Purpose / Why: Main: reusable operation.

tools\missing_upc_matcher_v3.py

Line 46: clean_upc
Purpose / Why: Clean upc: reusable operation.
Line 65: extract_upc_from_product
Purpose / Why: Extract upc from product: reusable operation.
Line 71: parse_alt_upcs
Purpose / Why: Parse alt upcs: reusable operation.
Line 79: ns
Purpose / Why: Normalise string.
Line 85: brand_token
Purpose / Why: Brand token: reusable operation.
Line 89: find_csv
Purpose / Why: Find csv: reusable operation.
Line 100: load_master
Purpose / Why: Load master: reusable operation.
Line 111: build_master_indexes
Purpose / Why: Build master indexes: reusable operation.
Line 144: _get_upc_from_row
Purpose / Why: get upc from row: reusable operation.
Line 148: _brand_from_cat_row
Purpose / Why: brand from cat row: reusable operation.
Line 156: _size_from_cat_row
Purpose / Why: size from cat row: reusable operation.
Line 164: _cat_key
Purpose / Why: cat key: reusable operation.
Line 183: load_cat
Purpose / Why: Load cat: reusable operation.
Line 205: classify_score
Purpose / Why: Classify score: reusable operation.
Line 211: _master_fields
Purpose / Why: master fields: reusable operation.
Line 230: analyse_sheet
Purpose / Why: Analyse sheet: reusable operation.
Line 383: write_excel
Purpose / Why: Write excel: reusable operation.
Line 472: main
Purpose / Why: Main: reusable operation.

tools\missing_upc_matcher_v4.py

Line 44: clean_upc
Purpose / Why: Clean upc: reusable operation.
Line 63: extract_upc_from_product
Purpose / Why: Extract upc from product: reusable operation.
Line 69: parse_alt_upcs
Purpose / Why: Returns list of clean UPCs from {upc1,upc2,...} notation.
Line 78: ns
Purpose / Why: Ns: reusable operation.
Line 83: brand_token
Purpose / Why: Brand token: reusable operation.
Line 87: find_csv
Purpose / Why: Find csv: reusable operation.
Line 94: classify_score
Purpose / Why: Classify score: reusable operation.

Line 111: load_master
Purpose / Why: Load master: reusable operation.
Line 130: build_brand_index
Purpose / Why: Brand token -> list of (master_idx, product_key_str)
Line 146: _get_upc
Purpose / Why: get upc: reusable operation.
Line 150: _brand
Purpose / Why: brand: reusable operation.
Line 158: _size
Purpose / Why: size: reusable operation.
Line 166: _cat_key
Purpose / Why: cat key: reusable operation.
Line 184: load_cat
Purpose / Why: Load cat: reusable operation.
Line 198: master_fields
Purpose / Why: Master fields: reusable operation.
Line 217: analyse_sheet
Purpose / Why: Analyse sheet: reusable operation.
Line 374: write_excel
Purpose / Why: Write excel: reusable operation.
Line 461: main
Purpose / Why: Main: reusable operation.

tools\optimize_csv.py

Line 12: optimize_csv_file
Purpose / Why: Optimize csv file: reusable operation.
Line 90: main
Purpose / Why: Main: reusable operation.

tests\test_all_missing.py

Line 4: main
Purpose / Why: Main: reusable operation.

tests\test_app_endpoints.py

Line 11: test_all_endpoints
Purpose / Why: Test all endpoints: reusable operation.
Line 78: test_upc_audit_returns_stable_csv_export_without_writing
Purpose / Why: Test upc audit returns stable csv export without writing: reusable operation.
Line 107: test_index_exposes_upc_first_review_action
Purpose / Why: Test index exposes upc first review action: reusable operation.
Line 117: test_comparison_uses_human_category_label
Purpose / Why: Test comparison uses human category label: reusable operation.
Line 123: test_comparison_search_is_sent_to_server_query
Purpose / Why: Test comparison search is sent to server query: reusable operation.
Line 126: test_comparison_search_is_sent_to_server_query -> FakeComparison
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 127: test_comparison_search_is_sent_to_server_query -> FakeComparison -> compare_all_missing_in_master
Purpose / Why: Compare all missing in master: reusable operation.
Line 139: test_upc_audit_full_review_passes_unbounded_limit
Purpose / Why: Test upc audit full review passes unbounded limit: reusable operation.
Line 142: test_upc_audit_full_review_passes_unbounded_limit -> FakeEngine
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 143: test_upc_audit_full_review_passes_unbounded_limit -> FakeEngine -> audit
Purpose / Why: Audit: reusable operation.

tests\test_chat_routing.py

Line 4: test_master_brand_id_question_uses_schema_aware_search
Purpose / Why: Test master brand id question uses schema aware search: reusable operation.
Line 5: test_master_brand_id_question_uses_schema_aware_search -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 6: test_master_brand_id_question_uses_schema_aware_search -> FakeSQL -> search_master_brands
Purpose / Why: Search master brands: reusable operation.
Line 22: test_brand_and_upc_question_uses_combined_master_search
Purpose / Why: Test brand and upc question uses combined master search: reusable operation.
Line 23: test_brand_and_upc_question_uses_combined_master_search -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.

Line 24: test_brand_and_upc_question_uses_combined_master_search -> FakeSQL -> search_master_brands
Purpose / Why: Search master brands: reusable operation.
Line 36: test_general_database_question_gets_an_answer_without_catalog_lookup
Purpose / Why: Test general database question gets an answer without catalog lookup: reusable operation.
Line 37: test_general_database_question_gets_an_answer_without_catalog_lookup -> ExplodingSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 38: test_general_database_question_gets_an_answer_without_catalog_lookup -> ExplodingSQL -> __getattr__
Purpose / Why: getattr : reusable operation.
Line 47: test_how_are_you_gets_a_response
Purpose / Why: Test how are you gets a response: reusable operation.
Line 53: test_general_chat_prefers_ollama_over_predefined_fallback
Purpose / Why: Test general chat prefers ollama over predefined fallback: reusable operation.
Line 66: test_master_brand_brand_only_question_uses_master_search
Purpose / Why: Test master brand brand only question uses master search: reusable operation.
Line 67: test_master_brand_brand_only_question_uses_master_search -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 68: test_master_brand_brand_only_question_uses_master_search -> FakeSQL -> search_master_brands
Purpose / Why: Search master brands: reusable operation.
Line 77: test_missing_master_audit_is_not_captured_by_brand_count_handler
Purpose / Why: Test missing master audit is not captured by brand count handler: reusable operation.
Line 78: test_missing_master_audit_is_not_captured_by_brand_count_handler -> FakeComparison
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 79: test_missing_master_audit_is_not_captured_by_brand_count_handler -> FakeComparison -> compare_all_missing_in_master
Purpose / Why: Compare all missing in master: reusable operation.
Line 91: test_missing_master_audit_is_not_captured_by_brand_count_handler -> ExplodingSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 92: test_missing_master_audit_is_not_captured_by_brand_count_handler -> ExplodingSQL -> count_brands
Purpose / Why: Count brands: reusable operation.

tests\test_conversation_context.py

Line 9: test_conversation_store_keeps_recent_bounded_turns
Purpose / Why: Test conversation store keeps recent bounded turns: reusable operation.
Line 22: test_conversation_store_is_session_scoped
Purpose / Why: Test conversation store is session scoped: reusable operation.
Line 29: test_follow_up_context_is_added_only_for_contextual_short_questions
Purpose / Why: Test follow up context is added only for contextual short questions: reusable operation.

tests\test_dashboard_endpoints.py

Line 11: test_dashboard_overview_returns_live_kpis
Purpose / Why: Test dashboard overview returns live kpis: reusable operation.
Line 12: test_dashboard_overview_returns_live_kpis -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 13: test_dashboard_overview_returns_live_kpis -> FakeSQL -> get_dataset_overview
Purpose / Why: Get dataset overview: reusable operation.
Line 16: test_dashboard_overview_returns_live_kpis -> FakeDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 17: test_dashboard_overview_returns_live_kpis -> FakeDB -> backend_name
Purpose / Why: Backend name: reusable operation.
Line 32: test_dashboard_missing_upc_is_bounded
Purpose / Why: Test dashboard missing upc is bounded: reusable operation.
Line 33: test_dashboard_missing_upc_is_bounded -> FakeComparison
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 34: test_dashboard_missing_upc_is_bounded -> FakeComparison -> compare_all_missing_in_master
Purpose / Why: Compare all missing in master: reusable operation.
Line 46: test_dashboard_export_rejects_unknown_dataset
Purpose / Why: Test dashboard export rejects unknown dataset: reusable operation.
Line 52: test_dashboard_export_returns_csv
Purpose / Why: Test dashboard export returns csv: reusable operation.
Line 53: test_dashboard_export_returns_csv -> FakeDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 54: test_dashboard_export_returns_csv -> FakeDB -> query
Purpose / Why: Query: reusable operation.

tests\test_dashboard_metrics.py

Line 4: MetricsDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 5: MetricsDB -> query
Purpose / Why: Query: reusable operation.
Line 27: test_dashboard_metrics_returns_live_category_and_master_brand_quality
Purpose / Why: Test dashboard metrics returns live category and master brand quality: reusable operation.
Line 40: test_dashboard_metrics_handles_master_schema_without_normalized_brand_or_id
Purpose / Why: Test dashboard metrics handles master schema without normalized brand or id: reusable operation.
Line 41: test_dashboard_metrics_handles_master_schema_without_normalized_brand_or_id -> LegacyMetricsDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 42: test_dashboard_metrics_handles_master_schema_without_normalized_brand_or_id -> LegacyMetricsDB -> table_columns
Purpose / Why: Table columns: reusable operation.
Line 45: test_dashboard_metrics_handles_master_schema_without_normalized_brand_or_id -> LegacyMetricsDB -> query
Purpose / Why: Query: reusable operation.

tests\test_db_backend.py

Line 11: test_db_manager_reports_sqlite_until_mysql_is_enabled
Purpose / Why: Test db manager reports sqlite until mysql is enabled: reusable operation.
Line 20: test_configure_db_switches_to_mysql_after_success
Purpose / Why: Test configure db switches to mysql after success: reusable operation.
Line 21: test_configure_db_switches_to_mysql_after_success -> FakeManager
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 25: test_configure_db_switches_to_mysql_after_success -> FakeManager -> test_mysql_connection
Purpose / Why: Test mysql connection: reusable operation.
Line 28: test_configure_db_switches_to_mysql_after_success -> FakeManager -> set_mysql_backend
Purpose / Why: Set mysql backend: reusable operation.
Line 31: test_configure_db_switches_to_mysql_after_success -> FakeManager -> backend_name
Purpose / Why: Backend name: reusable operation.
Line 44: test_configure_db_keeps_sqlite_after_failed_mysql_connection
Purpose / Why: Test configure db keeps sqlite after failed mysql connection: reusable operation.
Line 45: test_configure_db_keeps_sqlite_after_failed_mysql_connection -> FakeManager
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 49: test_configure_db_keeps_sqlite_after_failed_mysql_connection -> FakeManager -> test_mysql_connection
Purpose / Why: Test mysql connection: reusable operation.
Line 52: test_configure_db_keeps_sqlite_after_failed_mysql_connection -> FakeManager -> set_mysql_backend
Purpose / Why: Set mysql backend: reusable operation.
Line 55: test_configure_db_keeps_sqlite_after_failed_mysql_connection -> FakeManager -> backend_name
Purpose / Why: Backend name: reusable operation.
Line 67: test_query_falls_back_to_sqlite_when_mysql_query_fails
Purpose / Why: Test query falls back to sqlite when mysql query fails: reusable operation.
Line 70: test_query_falls_back_to_sqlite_when_mysql_query_fails -> Cursor
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 71: test_query_falls_back_to_sqlite_when_mysql_query_fails -> Cursor -> execute
Purpose / Why: Execute: reusable operation.
Line 74: test_query_falls_back_to_sqlite_when_mysql_query_fails -> Cursor -> fetchall
Purpose / Why: Fetchall: reusable operation.
Line 77: test_query_falls_back_to_sqlite_when_mysql_query_fails -> Connection
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 78: test_query_falls_back_to_sqlite_when_mysql_query_fails -> Connection -> cursor
Purpose / Why: Cursor: reusable operation.
Line 81: test_query_falls_back_to_sqlite_when_mysql_query_fails -> Connection -> close
Purpose / Why: Close: reusable operation.
Line 92: test_database_status_reports_safe_sqlite_and_mysql_fields
Purpose / Why: Test database status reports safe sqlite and mysql fields: reusable operation.
Line 105: test_mysql_schema_readiness_is_explicit
Purpose / Why: Test mysql schema readiness is explicit: reusable operation.

tests\test_document_rag.py

Line 13: storage
Purpose / Why: Storage: reusable operation.
Line 17: test_validate_upload_rejects_unsupported_extension
Purpose / Why: Test validate upload rejects unsupported extension: reusable operation.
Line 22: test_validate_upload_rejects_large_upload
Purpose / Why: Test validate upload rejects large upload: reusable operation.

Line 27: test_ingest_searches_text_without_loading_all_chunks
Purpose / Why: Test ingest searches text without loading all chunks: reusable operation.
Line 39: test_doc_upload_fails_with_clear_message
Purpose / Why: Test doc upload fails with clear message: reusable operation.
Line 47: test_search_falls_back_for_chunks_created_before_fts_index
Purpose / Why: Test search falls back for chunks created before fts index: reusable operation.

tests\test_frontend_contract.py

Line 7: test_frontend_contains_dashboard_and_backend_status_hooks
Purpose / Why: Test frontend contains dashboard and backend status hooks: reusable operation.
Line 14: test_frontend_contains_upload_preview_and_clear_loading_states
Purpose / Why: Test frontend contains upload preview and clear loading states: reusable operation.
Line 21: test_frontend_contains_live_analytics_and_master_brand_search_hooks
Purpose / Why: Test frontend contains live analytics and master brand search hooks: reusable operation.
Line 31: test_frontend_does_not_present_stale_totals_or_database_credentials
Purpose / Why: Test frontend does not present stale totals or database credentials: reusable operation.
Line 39: test_frontend_uses_pie_chart_and_clear_upc_copy
Purpose / Why: Test frontend uses pie chart and clear upc copy: reusable operation.
Line 47: test_frontend_separates_master_and_category_upc_pie_segments
Purpose / Why: Test frontend separates master and category upc pie segments: reusable operation.
Line 57: test_frontend_explains_powerbi_metric_definitions_and_refresh_context
Purpose / Why: Test frontend explains powerbi metric definitions and refresh context: reusable operation.
Line 65: test_frontend_uses_beverage_dashboard_title_and_explains_backend
Purpose / Why: Test frontend uses beverage dashboard title and explains backend: reusable operation.
Line 70: test_frontend_exposes_persistent_light_dark_theme_toggle
Purpose / Why: Test frontend exposes persistent light dark theme toggle: reusable operation.

tests\test_master_brand_endpoints.py

Line 4: test_dashboard_metrics_endpoint_returns_live_payload
Purpose / Why: Test dashboard metrics endpoint returns live payload: reusable operation.
Line 17: test_master_brand_search_accepts_alphanumeric_id
Purpose / Why: Test master brand search accepts alphanumeric id: reusable operation.
Line 20: test_master_brand_search_accepts_alphanumeric_id -> fake_search_master_brands
Purpose / Why: Fake search master brands: reusable operation.
Line 32: test_master_brand_search_rejects_empty_filters
Purpose / Why: Test master brand search rejects empty filters: reusable operation.
Line 39: test_master_brand_export_returns_validation_error_without_500
Purpose / Why: Test master brand export returns validation error without 500: reusable operation.

tests\test_master_brand_search.py

Line 4: MasterDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 5: MasterDB -> table_columns
Purpose / Why: Table columns: reusable operation.
Line 9: MasterDB -> query
Purpose / Why: Query: reusable operation.
Line 25: test_master_brand_search_returns_exact_alphanumeric_id_and_all_fields
Purpose / Why: Test master brand search returns exact alphanumeric id and all fields: reusable operation.
Line 33: test_master_brand_search_supports_record_id_and_reports_full_match_count
Purpose / Why: Test master brand search supports record id and reports full match count: reusable operation.
Line 34: test_master_brand_search_supports_record_id_and_reports_full_match_count -> LegacyDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 35: test_master_brand_search_supports_record_id_and_reports_full_match_count -> LegacyDB -> table_columns
Purpose / Why: Table columns: reusable operation.
Line 38: test_master_brand_search_supports_record_id_and_reports_full_match_count -> LegacyDB -> query
Purpose / Why: Query: reusable operation.
Line 54: test_master_brand_metrics_falls_back_to_source_upc_column
Purpose / Why: Test master brand metrics falls back to source upc column: reusable operation.
Line 55: test_master_brand_metrics_falls_back_to_source_upc_column -> SourceUPCDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 56: test_master_brand_metrics_falls_back_to_source_upc_column -> SourceUPCDB -> table_columns
Purpose / Why: Table columns: reusable operation.
Line 61: test_master_brand_metrics_falls_back_to_source_upc_column -> SourceUPCDB -> query
Purpose / Why: Query: reusable operation.

tests\test_mysql_conn.py

Line 3: main
Purpose / Why: Main: reusable operation.

tests\test_optimize_csv.py

Line 6: test_optimize_csv_preserves_source_and_deduplicates_to_separate_file
Purpose / Why: Test optimize csv preserves source and deduplicates to separate file: reusable operation.
Line 34: test_optimize_csv_rejects_unknown_requested_column
Purpose / Why: Test optimize csv rejects unknown requested column: reusable operation.

tests\test_queries.py

Line 8: main
Purpose / Why: Main: reusable operation.

tests\test_rag_audit.py

Line 4: test_ollama_availability_check_is_cached
Purpose / Why: Test ollama availability check is cached: reusable operation.
Line 7: test_ollama_availability_check_is_cached -> FakeResponse
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 10: test_ollama_availability_check_is_cached -> fake_get
Purpose / Why: Fake get: reusable operation.
Line 22: test_master_brand_count_uses_direct_sql_count
Purpose / Why: Test master brand count uses direct sql count: reusable operation.
Line 26: test_master_brand_count_uses_direct_sql_count -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 27: test_master_brand_count_uses_direct_sql_count -> FakeSQL -> count_master_brands
Purpose / Why: Count master brands: reusable operation.
Line 31: test_master_brand_count_uses_direct_sql_count -> FakeVector
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 32: test_master_brand_count_uses_direct_sql_count -> FakeVector -> search
Purpose / Why: Search: reusable operation.
Line 45: test_hinglish_master_brand_count_gets_hinglish_answer
Purpose / Why: Test hinglish master brand count gets hinglish answer: reusable operation.
Line 46: test_hinglish_master_brand_count_gets_hinglish_answer -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 47: test_hinglish_master_brand_count_gets_hinglish_answer -> FakeSQL -> count_master_brands
Purpose / Why: Count master brands: reusable operation.
Line 57: test_short_english_brand_count_question_gets_master_count
Purpose / Why: Test short english brand count question gets master count: reusable operation.
Line 58: test_short_english_brand_count_question_gets_master_count -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 59: test_short_english_brand_count_question_gets_master_count -> FakeSQL -> count_master_brands
Purpose / Why: Count master brands: reusable operation.
Line 69: test_category_brand_count_uses_requested_sheet
Purpose / Why: Test category brand count uses requested sheet: reusable operation.
Line 70: test_category_brand_count_uses_requested_sheet -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 71: test_category_brand_count_uses_requested_sheet -> FakeSQL -> count_brands
Purpose / Why: Count brands: reusable operation.
Line 83: test_flexible_joke_request_returns_a_joke
Purpose / Why: Test flexible joke request returns a joke: reusable operation.
Line 90: test_brand_profile_query_returns_sheet_details_in_hinglish
Purpose / Why: Test brand profile query returns sheet details in hinglish: reusable operation.
Line 91: test_brand_profile_query_returns_sheet_details_in_hinglish -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 92: test_brand_profile_query_returns_sheet_details_in_hinglish -> FakeSQL -> brand_profile
Purpose / Why: Brand profile: reusable operation.
Line 122: test_brand_or_product_description_search_uses_all_sheet_catalog
Purpose / Why: Test brand or product description search uses all sheet catalog: reusable operation.
Line 123: test_brand_or_product_description_search_uses_all_sheet_catalog -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 124: test_brand_or_product_description_search_uses_all_sheet_catalog -> FakeSQL -> search_catalog
Purpose / Why: Search catalog: reusable operation.
Line 147: test_hinglish_missing_category_query_gets_hinglish_summary
Purpose / Why: Test hinglish missing category query gets hinglish summary: reusable operation.
Line 148: test_hinglish_missing_category_query_gets_hinglish_summary -> FakeComparison
Purpose / Why: Class: related data aur methods ka reusable blueprint.

Line 149: test_hinglish_missing_category_query_gets_hinglish_summary -> FakeComparison -> compare_master_vs_categorization
Purpose / Why: Compare master vs categorization: reusable operation.
Line 175: test_product_audit_chat_requests_full_category_review
Purpose / Why: Test product audit chat requests full category review: reusable operation.
Line 178: test_product_audit_chat_requests_full_category_review -> FakeEngine
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 179: test_product_audit_chat_requests_full_category_review -> FakeEngine -> audit
Purpose / Why: Audit: reusable operation.
Line 195: test_supplier_question_uses_supplier_search_without_brand_profile
Purpose / Why: Test supplier question uses supplier search without brand profile: reusable operation.
Line 196: test_supplier_question_uses_supplier_search_without_brand_profile -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 197: test_supplier_question_uses_supplier_search_without_brand_profile -> FakeSQL -> search_supplier
Purpose / Why: Search supplier: reusable operation.
Line 203: test_supplier_question_uses_supplier_search_without_brand_profile -> FakeSQL -> brand_profile
Purpose / Why: Brand profile: reusable operation.

tests\test_rag_id_lookup.py

Line 4: test_id_query_uses_complete_identifier_lookup
Purpose / Why: Test id query uses complete identifier lookup: reusable operation.
Line 5: test_id_query_uses_complete_identifier_lookup -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 6: test_id_query_uses_complete_identifier_lookup -> FakeSQL -> lookup_identifier
Purpose / Why: Lookup identifier: reusable operation.
Line 31: test_invalid_id_returns_clear_not_found_response
Purpose / Why: Test invalid id returns clear not found response: reusable operation.
Line 32: test_invalid_id_returns_clear_not_found_response -> FakeSQL
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 33: test_invalid_id_returns_clear_not_found_response -> FakeSQL -> lookup_identifier
Purpose / Why: Lookup identifier: reusable operation.

tests\test_sql_search.py

Line 4: FakeDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 5: FakeDB -> __init__
Purpose / Why: init : reusable operation.
Line 8: FakeDB -> query
Purpose / Why: Query: reusable operation.
Line 31: test_lookup_identifier_returns_complete_direct_and_related_records
Purpose / Why: Test lookup identifier returns complete direct and related records: reusable operation.
Line 42: test_lookup_identifier_matches_alternate_upc_and_related_dataset
Purpose / Why: Test lookup identifier matches alternate upc and related dataset: reusable operation.
Line 51: test_search_catalog_passes_offset_for_pagination
Purpose / Why: Test search catalog passes offset for pagination: reusable operation.

tests\test_upc_match_engine.py

Line 4: FakeDB
Purpose / Why: Class: related data aur methods ka reusable blueprint.
Line 5: FakeDB -> __init__
Purpose / Why: init : reusable operation.
Line 11: FakeDB -> query
Purpose / Why: Query: reusable operation.
Line 20: _category_row
Purpose / Why: category row: reusable operation.
Line 35: _master_row
Purpose / Why: master row: reusable operation.
Line 50: test_exact_upc_is_reported_as_verified_without_writing
Purpose / Why: Test exact upc is reported as verified without writing: reusable operation.
Line 64: test_missing_upc_gets_strong_product_match_and_preserves_leading_zero
Purpose / Why: Test missing upc gets strong product match and preserves leading zero: reusable operation.
Line 80: test_brand_only_match_does_not_suggest_an_upc
Purpose / Why: Test brand only match does not suggest an upc: reusable operation.
Line 94: test_blank_brand_does_not_scan_unrelated_master_products
Purpose / Why: Test blank brand does not scan unrelated master products: reusable operation.

`Line 107: test_size_conflict_does_not_suggest_an_upc`
Purpose / Why: Test size conflict does not suggest an upc: reusable operation.
`Line 121: test_liter_and_milliliter_sizes_are_compatible`
Purpose / Why: Test liter and milliliter sizes are compatible: reusable operation.
`Line 136: test_audit_loads_master_candidates_once_for_multiple_rows`
Purpose / Why: Test audit loads master candidates once for multiple rows: reusable operation.

tests\test_upload_endpoints.py

`Line 10: test_chat_rejects_unsupported_upload_before_processing`
Purpose / Why: Test chat rejects unsupported upload before processing: reusable operation.

tests\test_user_prompt.py

`Line 7: main`
Purpose / Why: Main: reusable operation.

6. Kaunsi language kyun use hui?

Python: main backend language. Flask routes, CSV cleaning, UPC normalization, database queries, RAG routing, BM25 scoring, document extraction aur Ollama API calls Python mein hain. Python choose hua kyunki pandas, Flask, SQLite, PDF/DOCX readers aur AI ecosystem strong hai; data-heavy logic readable rehta hai.

HTML: templates/index.html browser ka structure banata hai - header, chat box, tabs, buttons, tables aur upload controls. HTML decide karta hai page par kaunse elements exist karenge.

CSS: index.html ke style section mein colors, dark dashboard layout, cards, buttons, responsive grid, badges aur scrolling tables define hote hain. CSS ka kaam appearance hai, business/data logic nahi.

JavaScript: browser-side interaction handle karta hai. fetch() se /api/chat, /api/compare aur export endpoints call hote hain; FormData file upload karta hai; returned JSON ko chat bubbles aur tables mein convert karta hai. Isse page reload ke bina live UI milti hai.

SQL: SQLite aur MySQL tables se exact search, COUNT, JOIN, GROUP BY, duplicate audit aur missing-record comparison ke liye. SQL deterministic hai, isliye exact UPC/count ke liye LLM se better hai.

JSON: frontend-backend communication format. Flask jsonify() se answer, records, source_files, record_count aur download_token browser ko milte hain. JSON config/analysis reports mein bhi use hua hai.

CSV: original business datasets ka storage format. Pipe (|) aur comma delimiters alag files mein hain; loader delimiter config se read karta hai. CSV reports user download kar sakta hai.

Markdown: assistant answers headings, bullets, tables aur code formatting ke liye banata hai. Frontend ka parseSimpleMarkdown() us text ko HTML presentation mein badalta hai.

.env simple KEY=VALUE settings rakhta hai. requirements.txt Python dependencies ka manifest hai. Pickle binary format persistent local vector index save karne ke liye use hua hai.

7. Languages ka combined request flow

HTML form + JavaScript FormData -> Flask/Python route -> Python RAG/SQL engine -> SQLite SQL result -> Python JSON response -> JavaScript JSON parsing -> HTML table + CSS styling. Agar wording natural banani ho aur Ollama online ho, Python retrieved context ko local LLM ko bhejta hai; final answer Markdown mein aata hai.

8. Kisi ko explain karne ka ready answer

BevIQ local beverage intelligence application hai. CSV files pehle normalize hokar SQLite tables aur indexes banti hain. Flask browser/API requests leta hai. RAG orchestrator query ko route karta hai: exact lookup ke liye SQL, cross-file gap ke liye comparison engine, uploaded PDF ke liye document chunks, product UPC review ke liye UPC match engine, aur fuzzy catalog retrieval ke liye BM25 vector engine. Ollama optional local LLM hai jo retrieved facts ko conversational answer mein likhta hai. Isliye app offline/private, traceable source-row based aur CSV-exportable hai.